

CSCRS Web Portal Software Requirements Specification (SRS)

Version: 1.0 (Draft)

Document Status: In Progress

Chapter 1 — Introduction

1.1 Purpose

The purpose of this document is to define the complete Software Requirements Specification (SRS) for the **Crowdsourced Civic Issue Reporting and Resolution System (CSCRS) Web Portal**.

This document serves as the primary blueprint for designing, developing, testing, maintaining, and enhancing the web portal throughout its lifecycle.

It describes the functional requirements, non-functional requirements, user experience, interface behavior, security decisions, accessibility standards, navigation system, frontend architecture, integration strategy, and future extensibility of the portal.

The objective of this document is to ensure that every stakeholder—including developers, UI/UX designers, testers, project evaluators, and AI-assisted development tools—shares a common understanding of the system before implementation begins.

1.2 Scope

The CSCRS Web Portal is an enterprise-grade administrative portal designed for municipal authorities to monitor, manage, and resolve civic issues reported by citizens through the CSCRS ecosystem.

The portal is intended exclusively for administrative users and shall provide secure, role-based access to system functionalities.

The portal supports the following administrative roles:

- Department Administrator
- City Administrator
- Super Administrator

The web portal does **not** provide interfaces for Citizens or Workers.

Citizen and Worker interactions are handled exclusively through the dedicated CSCRS Mobile Application.

The portal shall provide:

- Administrative dashboards
- Report management

- Department management
- Worker management
- Analytics and statistics
- Notifications
- AI-assisted monitoring
- System administration
- Secure authentication
- Role-Based Access Control (RBAC)

The portal is designed as a modern Single Page Application (SPA) communicating with the existing FastAPI backend through REST APIs.

1.3 Project Vision

The vision of the CSCRS Web Portal is to provide a centralized, secure, scalable, and intelligent administrative platform capable of simplifying municipal operations while maintaining transparency, accountability, and operational efficiency.

Instead of functioning as a conventional complaint management website, the portal shall operate as an enterprise operational dashboard that enables authorized officials to monitor the complete lifecycle of civic issue resolution.

The system aims to reduce manual workload through intelligent automation while providing administrators with real-time visibility into reports, departments, workers, analytics, and AI-generated insights.

1.4 Objectives

The primary objectives of the portal are:

- Provide a unified administrative platform for all municipal authorities.
- Eliminate fragmented management systems.
- Enable secure role-based administration.
- Present real-time operational dashboards.
- Reduce manual administrative workload.
- Improve transparency of civic issue resolution.
- Enable faster decision making through analytics.
- Provide an intuitive enterprise-grade user experience.
- Integrate seamlessly with the existing FastAPI backend.

- Support future scalability without architectural redesign.

1.5 Intended Audience

This document is intended for:

- Frontend Developers
- Backend Developers
- UI/UX Designers
- Software Architects
- Project Evaluators
- Quality Assurance Engineers
- Future Contributors
- AI-assisted Development Systems

1.6 Document Conventions

Throughout this document, the following terminology shall be used consistently:

Term	Meaning
Portal	CSCRS Administrative Web Portal
Mobile Application	Citizen & Worker Mobile Application
Dashboard	Role-specific landing workspace after authentication
Module	A functional section of the portal
Screen	Individual user interface page
Component	Reusable UI element
Widget	Dashboard information card or visualization
Role	User permission category
Administrator	Any authenticated portal user
API	Existing FastAPI REST endpoint
SPA	Single Page Application

1.7 Design Philosophy

The CSCRS Web Portal shall follow a **Modern Government Enterprise Design Philosophy**.

The portal shall not resemble a traditional government website with outdated layouts and static interfaces.

Instead, it shall combine:

- Government-grade trustworthiness
- Enterprise software usability
- Modern SaaS aesthetics
- Accessibility compliance
- Responsive design
- Performance-first architecture

The user experience shall prioritize operational efficiency over decorative elements.

Every interface element must contribute directly to productivity, clarity, or system transparency.

1.8 Guiding Principles

The portal shall be developed according to the following principles:

Simplicity

Interfaces shall minimize unnecessary complexity.

Consistency

All pages shall follow the same visual language and interaction patterns.

Accessibility

The portal shall remain usable by all users, including keyboard-only users and assistive technologies.

Security

Every administrative feature shall respect authentication, authorization, and role-based permissions.

Scalability

The architecture shall accommodate future modules without major redesign.

Maintainability

Reusable components and feature-based architecture shall be preferred throughout development.

Performance

The portal shall prioritize fast loading, efficient rendering, and optimized API communication.

User-Centered Navigation

Users shall reach frequently used features with the minimum possible number of interactions.

Dashboard widgets, quick actions, breadcrumbs, and global search shall support this objective.

1.9 Out of Scope

The following features are explicitly outside the scope of this document:

- Citizen Mobile Application UI
- Worker Mobile Application UI
- Backend API implementation
- Machine Learning model implementation
- Database schema design
- Infrastructure deployment configuration

These systems are documented independently.

1.10 Document Roadmap

The remainder of this specification is organized as follows:

- Chapter 2 — Project Overview
- Chapter 3 — Stakeholders & User Roles
- Chapter 4 — Portal Scope
- Chapter 5 — Information Architecture
- Chapter 6 — Landing Page Specification
- Chapter 7 — Authentication Module
- Chapter 8 — Enterprise Portal Layout
- Chapter 9 — Navigation System

- Chapter 10 — Dashboard Specification
- Chapter 11 — Functional Modules
- Chapter 12 — UI Design System
- Chapter 13 — Accessibility Requirements
- Chapter 14 — Responsive Design
- Chapter 15 — Security Requirements
- Chapter 16 — Frontend Architecture
- Chapter 17 — API Integration Strategy
- Chapter 18 — Error & Loading Strategy
- Chapter 19 — Performance Requirements
- Chapter 20 — Future Enhancements
- Chapter 21 — AI Development Guide

CSCRS Web Portal Software Requirements Specification (SRS)

Chapter 2 — Project Overview

2.1 Background

Municipal corporations receive thousands of civic complaints every day, including reports related to road damage, garbage accumulation, waterlogging, damaged street lights, damaged electric poles, and open manholes. In many municipalities, the complaint management process is still partially manual or relies on fragmented digital systems that lack intelligent verification, automation, centralized monitoring, and transparent resolution tracking.

These limitations often result in:

- Duplicate complaint submissions.
- False or invalid reports.
- Delayed assignment of work.
- Limited administrative visibility.
- Poor inter-department coordination.
- Lack of accountability.
- Slow response times.
- Reduced citizen satisfaction.

To address these challenges, the **Crowdsourced Civic Issue Reporting and Resolution System (CSCRS)** has been designed as an AI-assisted civic issue management platform.

The overall system combines Artificial Intelligence, computer vision, role-based administration, automated workflows, and real-time analytics to streamline the complete lifecycle of civic issue reporting and resolution.

2.2 About CSCRS

The **Crowdsourced Civic Issue Reporting and Resolution System (CSCRS)** is a smart civic issue management platform that enables citizens to report public infrastructure problems using a mobile application while providing municipal authorities with an enterprise-grade administrative portal for efficient monitoring, assignment, verification, and resolution of reported issues.

The system integrates Artificial Intelligence into multiple stages of the workflow, including issue detection, duplicate identification, image verification, and resolution validation, thereby reducing manual effort while improving operational efficiency.

The platform consists of two primary client applications:

Mobile Application

The mobile application is intended exclusively for:

- Citizens
- Municipal Workers

The mobile application enables citizens to report civic issues and allows workers to receive assignments and submit resolution evidence.

Administrative Web Portal

The Administrative Web Portal is intended exclusively for:

- Department Administrators
- City Administrators
- Super Administrators

The portal serves as the centralized operational control center for the entire system.

2.3 System Components

The CSCRS ecosystem consists of the following major components:

Citizen Mobile Application

Provides reporting, report tracking, notifications, profile management, and feedback submission.

Worker Mobile Application

Provides assignment management, work progress tracking, resolution submission, and task updates.

Administrative Web Portal

Provides centralized administration, dashboards, report management, analytics, worker management, department management, notifications, and system monitoring.

FastAPI Backend

Handles authentication, authorization, business logic, workflow orchestration, API services, report lifecycle management, and system operations.

Artificial Intelligence Engine

Provides:

- Civic issue detection
 - Image authenticity verification
 - Duplicate detection
 - Risk scoring
 - Resolution verification
-

Database Layer

Stores users, reports, assignments, timelines, analytics, notifications, departments, workers, and administrative data.

2.4 High-Level System Workflow

The overall system workflow SHALL be as follows:

Citizen

|



Capture Issue Image

|



AI Verification Pipeline



Issue Classification



Risk Assessment



Report Creation



Department Assignment



Worker Assignment



Issue Resolution



Resolution Verification



Citizen Feedback

The Administrative Web Portal SHALL monitor and manage every stage of this workflow.

2.5 Purpose of the Administrative Web Portal

The Administrative Web Portal SHALL function as the centralized operational platform for authorized municipal administrators.

The portal SHALL NOT function as a public complaint submission website.

Its primary responsibilities include:

- Monitoring incoming reports.
 - Managing report lifecycle.
 - Managing departments.
 - Managing workers.
 - Monitoring AI verification results.
 - Reviewing manually flagged reports.
 - Viewing analytics and statistics.
 - Monitoring system performance.
 - Managing administrative operations.
-

2.6 Design Goals

The portal SHALL achieve the following design goals:

Enterprise User Experience

The portal SHALL provide a professional enterprise software experience comparable to modern administrative platforms.

Government Trustworthiness

The portal SHALL maintain a visual identity appropriate for municipal and government organizations while avoiding outdated interface patterns.

Operational Efficiency

Frequently performed administrative tasks SHALL require the minimum possible number of user interactions.

Scalability

The portal SHALL support future expansion without requiring significant architectural changes.

Accessibility

The portal SHALL comply with modern accessibility principles to ensure usability for all authorized users.

Responsiveness

The portal SHALL provide an optimized experience across desktop, tablet, and supported mobile browsers.

2.7 Supported Administrative Roles

The portal SHALL support the following authenticated roles:

Role	Primary Responsibility
Department Administrator	Department-level operations and worker management
City Administrator	City-wide monitoring and administrative control
Super Administrator	Global administration and system-level management

The portal SHALL NOT expose Citizen or Worker interfaces.

2.8 Administrative Philosophy

The portal SHALL follow a **Dashboard-First Operational Model**.

Immediately after successful authentication, users SHALL be redirected to a role-specific dashboard rather than a generic landing page.

The dashboard SHALL serve as the primary operational workspace.

Users SHALL be able to navigate to frequently used modules directly from dashboard widgets without relying solely on sidebar navigation.

This approach minimizes navigation effort while improving operational efficiency.

2.9 Role-Based Access Control (RBAC)

The Administrative Web Portal SHALL implement strict Role-Based Access Control.

Each authenticated user SHALL only view modules, menus, actions, and data that are explicitly authorized for their assigned role.

Unauthorized modules SHALL NOT appear in the user interface.

The portal SHALL avoid displaying disabled or inaccessible administrative modules to unauthorized users in order to reduce interface complexity and prevent unnecessary information disclosure.

2.10 Technology Philosophy

The portal SHALL be developed using modern frontend technologies that emphasize:

- Component reusability.

- Maintainability.
- Type safety.
- Accessibility.
- Performance.
- Scalability.
- Long-term support.

The frontend SHALL integrate seamlessly with the existing FastAPI backend through REST APIs without requiring backend architectural modifications.

2.11 Future Vision

The CSCRS Administrative Web Portal SHALL be designed as a scalable foundation capable of supporting future enhancements, including but not limited to:

- Multi-city deployment.
- Multi-language support.
- Progressive Web Application (PWA) capabilities.
- Advanced analytics.
- AI-powered recommendations.
- Device management.
- Session management.
- Single Sign-On (SSO).
- Two-Factor Authentication (2FA).
- Government identity integration.
- Smart City ecosystem integration.

These enhancements SHALL NOT require major redesign of the portal architecture.

Chapter 3 — Stakeholders and User Roles

3.1 Stakeholder Overview

The CSCRS ecosystem consists of multiple stakeholders who interact with different parts of the overall platform.

Not every stakeholder interacts with the Administrative Web Portal. Therefore, the responsibilities, permissions, and interfaces shall be clearly separated according to the intended operational workflow.

The primary stakeholders of the CSCRS ecosystem are:

- Citizens
- Municipal Workers
- Department Administrators
- City Administrators
- Super Administrators
- Municipal Authorities
- System Administrators (Maintenance)
- Development Team (Future Enhancements)

Among these stakeholders, only administrative users SHALL access the Administrative Web Portal.

3.2 Stakeholder Classification

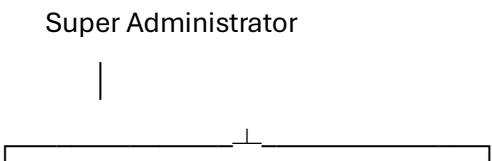
The stakeholders SHALL be classified as follows:

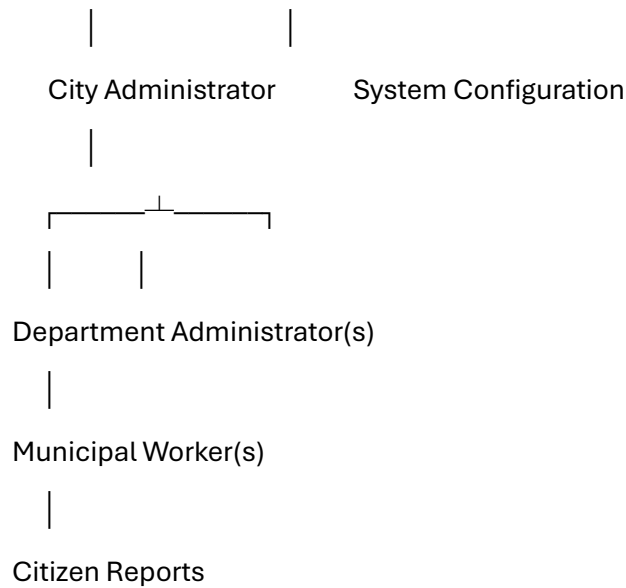
Stakeholder	Mobile Application	Web Portal	Description
Citizen	✓	✗	Reports civic issues and tracks resolution status.
Municipal Worker	✓	✗	Receives assignments and submits resolution evidence.
Department Administrator	✗	✓	Manages departmental operations and workers.
City Administrator	✗	✓	Monitors city-wide operations and administrative activities.
Super Administrator	✗	✓	Performs system-wide administration and oversight.

The Administrative Web Portal SHALL NOT expose interfaces intended for Citizens or Municipal Workers.

3.3 Administrative Hierarchy

The administrative hierarchy SHALL follow the structure illustrated below.





This hierarchy SHALL determine the authority boundaries and responsibilities of each administrative role.

3.4 Role Responsibilities

3.4.1 Citizen

The Citizen is responsible for reporting civic issues using the CSCRS Mobile Application.

Primary responsibilities include:

- Registering an account.
- Reporting civic issues.
- Uploading issue photographs.
- Tracking report status.
- Viewing report timeline.
- Receiving notifications.
- Providing resolution feedback.

The Citizen SHALL NOT access the Administrative Web Portal.

3.4.2 Municipal Worker

Municipal Workers are responsible for resolving assigned civic issues.

Primary responsibilities include:

- Viewing assigned tasks.
- Updating work progress.

- Uploading resolution images.
- Completing assigned work.
- Receiving assignment notifications.

Municipal Workers SHALL interact exclusively through the Mobile Application.

They SHALL NOT access the Administrative Web Portal.

3.4.3 Department Administrator

Department Administrators manage departmental operations.

Their responsibilities SHALL include:

- Monitoring departmental reports.
- Reviewing pending reports.
- Managing municipal workers.
- Assigning work.
- Forwarding reports when necessary.
- Reviewing AI-generated verification results.
- Monitoring departmental performance.
- Viewing departmental analytics.
- Receiving administrative notifications.

Department Administrators SHALL only access resources belonging to their assigned department.

3.4.4 City Administrator

City Administrators supervise the overall civic operations of an entire city.

Their responsibilities SHALL include:

- Monitoring city-wide reports.
- Managing departments.
- Managing Department Administrators.
- Monitoring analytics.
- Viewing operational statistics.
- Reviewing city performance.
- Managing administrative operations.

- Accessing city-level dashboards.

City Administrators SHALL NOT access system-level administrative functions reserved for Super Administrators.

3.4.5 Super Administrator

The Super Administrator is the highest authority within the CSCRS Administrative Portal.

Responsibilities SHALL include:

- System administration.
- Managing City Administrators.
- Monitoring platform health.
- Accessing AI dataset exports.
- Viewing system-wide analytics.
- Managing platform configuration.
- Monitoring operational status.
- Reviewing system logs (future enhancement).
- Managing global administrative settings (future enhancement).

The Super Administrator SHALL have unrestricted administrative access within the system.

3.5 Portal Access Policy

The Administrative Web Portal SHALL only permit authenticated users belonging to the following roles:

- Department Administrator
- City Administrator
- Super Administrator

Any attempt by Citizens or Municipal Workers to access the Administrative Web Portal SHALL be rejected through backend authorization mechanisms.

3.6 Authentication Policy

Every administrative user SHALL authenticate using the centralized login page.

The portal SHALL provide a single authentication interface for all administrative roles.

Role selection SHALL NOT be performed by the user during login.

Instead, the backend authentication service SHALL determine the authenticated user's role after successful credential verification.

Following authentication, the frontend SHALL automatically redirect the user to the appropriate role-specific dashboard.

3.7 Role-Based Interface Policy

The Administrative Web Portal SHALL implement dynamic user interfaces based on authenticated roles.

The following elements SHALL adapt according to RBAC permissions:

- Sidebar navigation.
- Dashboard widgets.
- Quick Actions.
- Management modules.
- Administrative tools.
- Analytics.
- Page visibility.
- Navigation routes.

Unauthorized modules SHALL NOT appear within the interface.

The portal SHALL avoid displaying disabled administrative modules solely for informational purposes.

3.8 Dashboard Assignment

Immediately after successful authentication, users SHALL be redirected to their respective operational dashboard.

Role	Landing Dashboard
Department Administrator	Department Dashboard
City Administrator	City Dashboard
Super Administrator	System Dashboard

The dashboard SHALL function as the primary operational workspace.

3.9 Portal Responsibility Matrix

Capability	Citizen Worker Department Admin City Admin Super Admin				
Login to Web Portal	✗	✗	✓	✓	✓
Dashboard	✗	✗	✓	✓	✓
Report Management	✗	✗	✓	✓	✓
Worker Management	✗	✗	✓	✗	✗
Department Management	✗	✗	✗	✓	✓
City Administration	✗	✗	✗	✗	✓
Analytics	✗	✗	✓	✓	✓
Notifications	✗	✗	✓	✓	✓
Settings	✗	✗	✓	✓	✓

Note: The matrix above represents high-level responsibilities. Detailed permissions and API-level authorization SHALL be specified in the RBAC and Module Specification chapters.

3.10 Separation of Responsibilities

The CSCRS ecosystem SHALL maintain a clear separation between operational environments.

Mobile Application

The Mobile Application SHALL be responsible for:

- Citizen interactions.
- Worker operations.
- Issue reporting.
- Assignment execution.
- Resolution submission.

Administrative Web Portal

The Administrative Web Portal SHALL be responsible for:

- Administrative monitoring.
- Operational management.
- Analytics.
- Administrative workflows.
- Department supervision.

- City supervision.
- System administration.

This separation SHALL simplify user experience while maintaining clear operational boundaries.

3.11 Guiding Principle

The Administrative Web Portal SHALL prioritize operational efficiency over feature exposure.

Users SHALL only interact with the tools required for their responsibilities.

The interface SHALL minimize unnecessary complexity by presenting only role-relevant information, thereby improving usability, reducing cognitive load, and strengthening security through interface-level role isolation.

Chapter 4 — Portal Scope and Information Architecture

4.1 Portal Overview

The CSCRS Administrative Web Portal SHALL serve as the centralized web-based administration platform for the Crowdsourced Civic Issue Reporting and Resolution System (CSCRS).

The portal SHALL provide authorized administrative users with secure access to dashboards, reports, analytics, management modules, notifications, and system operations through a unified enterprise interface.

The portal SHALL be implemented as a Single Page Application (SPA) and SHALL communicate with the existing FastAPI backend through REST APIs.

4.2 Portal Objectives

The Administrative Web Portal SHALL achieve the following objectives:

- Provide a centralized operational workspace.
- Reduce navigation complexity.
- Present real-time operational information.
- Improve administrative productivity.
- Support role-based administration.
- Integrate with AI-assisted workflows.
- Deliver a modern enterprise user experience.
- Maintain accessibility compliance.
- Support future scalability.

4.3 Portal Characteristics

The portal SHALL exhibit the following characteristics:

Enterprise-Oriented

The portal SHALL resemble modern enterprise administration software rather than a traditional government website.

Government Trust

The visual identity SHALL convey professionalism, transparency, and institutional trust appropriate for municipal governance.

Dashboard-Driven

Users SHALL begin their workflow from a role-specific dashboard immediately after authentication.

Role-Aware

Every screen, menu, widget, and navigation element SHALL adapt dynamically according to authenticated user permissions.

Responsive

The portal SHALL support desktop, laptop, tablet, and supported mobile browsers.

Desktop SHALL remain the primary target platform.

Accessible

The portal SHALL comply with modern accessibility guidelines and keyboard navigation standards.

4.4 Portal Operating Model

The Administrative Web Portal SHALL follow the operational model illustrated below.

Visitor



Landing Website

|



Authentication

|



Role Identification

|



Role-Specific Dashboard

|



Operational Modules

|



Detailed Workspaces

The Landing Website and the Administrative Portal SHALL operate as a single web application with separate public and authenticated experiences.

4.5 Public vs Authenticated Areas

The system SHALL be divided into two primary sections.

Public Area

Accessible without authentication.

Includes:

- Home
- About
- Features
- How It Works
- Departments
- Statistics
- FAQ
- Contact

- Download Mobile App
- Login

The Public Area SHALL function as the official public-facing website of CSCRS.

Authenticated Area

Accessible only after successful authentication.

Includes:

- Dashboard
- Reports
- Analytics
- Management
- Notifications
- Settings
- Administrative Modules

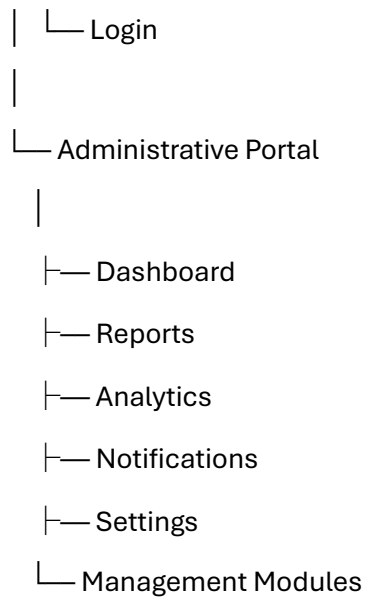
Every authenticated page SHALL require authorization before rendering.

4.6 Portal Information Architecture

The portal SHALL follow the information architecture below.

CSCRS Portal





Detailed module hierarchy SHALL be defined in Chapter 11.

4.7 Navigation Philosophy

The portal SHALL prioritize direct access to operational tasks.

Navigation SHALL be designed to minimize unnecessary clicks.

Users SHALL be able to reach frequently used modules through:

- Dashboard widgets.
- Quick Action cards.
- Sidebar navigation.
- Global Search.
- Notifications.
- Breadcrumb navigation.

The dashboard SHALL act as both an information center and a navigation hub.

4.8 Dashboard-First Experience

Following authentication, users SHALL NOT encounter an intermediate home page.

Instead, the system SHALL immediately display the role-specific dashboard.

The dashboard SHALL provide:

- Operational summaries.
- Quick actions.
- Recent activities.

- Analytics.
 - Alerts.
 - Shortcuts to frequently used modules.
-

4.9 Single Portal Philosophy

The Administrative Web Portal SHALL maintain a single application architecture.

Separate portals SHALL NOT be developed for:

- Department Administrators
- City Administrators
- Super Administrators

Instead, a unified application SHALL dynamically adapt according to authenticated permissions.

This approach improves maintainability, consistency, and scalability.

4.10 Role-Based Navigation

Navigation menus SHALL be generated dynamically after authentication.

Users SHALL only see navigation items for which they possess sufficient permissions.

Administrative modules outside the user's authorization scope SHALL NOT appear in the interface.

The portal SHALL avoid displaying disabled menu items solely for visibility purposes.

4.11 Workspace Concept

The Administrative Portal SHALL operate using a persistent workspace layout.

Every authenticated page SHALL consist of:

- Top Navigation Bar
- Collapsible Sidebar
- Breadcrumb Navigation
- Main Workspace
- Notification System
- Profile Menu

The Main Workspace SHALL dynamically render different modules while preserving the overall application layout.

4.12 Portal Design Philosophy

The portal SHALL emphasize productivity over decoration.

The design SHALL:

- Reduce visual clutter.
- Present only relevant information.
- Maintain consistent interaction patterns.
- Encourage rapid task completion.
- Support long-duration administrative use.

Animations SHALL remain subtle and SHALL never interfere with operational efficiency.

4.13 Functional Boundaries

The Administrative Web Portal SHALL NOT perform the following functions:

- Citizen issue reporting.
- Worker assignment execution.
- Resolution image capture.
- Mobile-specific workflows.
- AI model training.

These operations belong to other components of the CSCRS ecosystem.

4.14 Future Scalability

The information architecture SHALL permit future expansion without restructuring the existing navigation hierarchy.

Future modules MAY include:

- Audit Logs
- System Health
- Device Management
- Active Sessions
- AI Recommendations
- Multi-City Administration
- Government Integrations

- GIS Dashboard
- Advanced Reports

The architecture SHALL accommodate these additions using the existing navigation framework.

4.15 Sitemap (Public Website)

The public website SHALL expose the following sitemap structure.

/

- └─ Home
- └─ About
- └─ Features
- └─ How It Works
- └─ Departments
- └─ Statistics
- └─ FAQ
- └─ Contact
- └─ Download App
- └─ Login
- └─ Privacy Policy
- └─ Terms & Conditions
- └─ Accessibility Statement
- └─ Sitemap

The Sitemap page SHALL provide a structured overview of all publicly accessible pages.

4.16 Sitemap (Authenticated Portal)

The authenticated portal SHALL expose role-specific navigation.

Dashboard

- |
- └─ Reports
- | └─ All Reports
- | └─ Report Details



Additional nodes SHALL be rendered dynamically according to the authenticated user's role.

4.17 Chapter Summary

This chapter establishes the architectural foundation of the CSCRS Administrative Web Portal.

The portal SHALL consist of a single modern enterprise application featuring:

- A public-facing informational website.
- A secure authenticated administration environment.
- Dynamic role-based navigation.
- Dashboard-first workflow.
- Workspace-oriented architecture.
- Future-ready information hierarchy.

This architecture SHALL guide all subsequent frontend development activities.

Chapter 5 — Landing Page Specification

5.1 Overview

The Landing Page SHALL serve as the official public-facing website of the Crowdsourced Civic Issue Reporting and Resolution System (CSCRS).

The Landing Page SHALL function as:

- The official digital identity of the project.
- The primary entry point for visitors.
- An information portal for citizens and municipal authorities.

- A gateway to the Administrative Web Portal.
- A promotional page for the CSCRS Mobile Application.

The Landing Page SHALL remain publicly accessible without requiring authentication.

The page SHALL communicate trust, transparency, professionalism, and technological capability while maintaining a modern government enterprise aesthetic.

5.2 Design Philosophy

The Landing Page SHALL NOT resemble a traditional government website with outdated layouts, excessive text, or cluttered navigation.

Instead, the interface SHALL combine:

- Modern SaaS design principles.
- Government-level credibility.
- Enterprise dashboard aesthetics.
- High readability.
- Accessibility.
- Responsive design.
- Minimal visual clutter.

The overall experience SHALL emphasize clarity and professionalism over decorative elements.

5.3 Page Layout

The Landing Page SHALL follow the structure below.



Solution Overview
How CSCRS Works
Features
Dashboard Preview
Issue Categories
Live Statistics
AI Pipeline
Mobile App Promotion
FAQ
Contact
Footer

This section order SHALL remain consistent throughout development.

5.4 Utility Bar

The Utility Bar SHALL appear at the very top of the page.

Its primary purpose SHALL be accessibility and quick access.

Height:

Approximately 36–40 pixels.

Left Side

The left side SHALL contain:

- Government Identity (configurable)
- Smart City Mission (configurable)

Example:

Government of India | Smart City Mission

These values SHALL be configurable and SHALL NOT be hardcoded.

Right Side

The right side SHALL contain:

- Skip to Main Content
- Language Selector
- Accessibility
- Sitemap

Example:

Skip to Main Content

Language

Accessibility

Sitemap

The "Skip to Main Content" link SHALL become visible when focused using keyboard navigation.

5.5 Sticky Header

The main navigation header SHALL remain fixed at the top while scrolling.

The sticky behavior SHALL activate immediately after page scrolling begins.

The sticky header SHALL improve navigation efficiency and SHALL remain visible throughout the browsing session.

5.6 Header Layout

The Header SHALL consist of three logical sections.

Left

Official Logo

Project Name

CSCRS

Crowdsourced Civic Issue Reporting
and Resolution System

Center

Primary Navigation Menu.

Right

Download Mobile App

Login

The Login button SHALL be visually emphasized as the primary call-to-action.

5.7 Navigation Bar

The navigation menu SHALL include the following items.

- Home
- About
- Features
- How It Works
- Departments
- Statistics
- FAQ
- Contact

Navigation SHALL perform smooth scrolling to corresponding sections.

The active section SHALL be highlighted while scrolling.

5.8 Hero Section

The Hero Section SHALL occupy the first visible viewport.

This section SHALL communicate the platform's purpose within the first few seconds of user interaction.

Left Content

Primary Heading.

Example:

AI Powered Civic Issue Reporting & Resolution System

Subheading describing the mission.

Two primary buttons:

- Download Mobile App
 - Admin Login
-

Right Content

Instead of generic stock photography, the Hero SHALL display a custom illustration representing the CSCRS ecosystem.

The illustration SHOULD include visual representations of:

- City infrastructure.
- Roads.
- Garbage.
- Street Lights.
- Waterlogging.
- Artificial Intelligence.
- Administrative Dashboard.
- Connected workflow.

The artwork SHALL use a clean vector illustration style.

5.9 Departments Section

The Departments section SHALL present the municipal departments supported by CSCRS.

Each department SHALL be represented using an icon card.

Example departments include:

- Roads
- Solid Waste

- Street Lighting
- Electrical
- Water Supply
- Drainage

The list SHALL be configurable.

5.10 Problem Statement

This section SHALL explain the limitations of conventional complaint management systems.

Example comparison:

Traditional System

↓

Manual Verification

↓

Delayed Assignment

↓

Duplicate Complaints

↓

Slow Resolution

↓

Limited Transparency

Beside it, the CSCRS solution SHALL be presented.

5.11 Solution Overview

This section SHALL visually explain how CSCRS solves the identified problems.

A graphical workflow SHALL be displayed.

Example:

Citizen

↓

AI Verification

↓

Issue Classification

↓

Department Assignment

↓

Worker Resolution

↓

Citizen Feedback

5.12 How CSCRS Works

The complete workflow SHALL be displayed as a horizontal timeline.

Example:

1. Capture Issue

↓

2. AI Verification

↓

3. Issue Detection

↓

4. Department Assignment

↓

5. Worker Resolution

↓

6. Citizen Feedback

Each stage SHALL include:

- Icon
 - Title
 - Short description
-

5.13 Key Features

The Landing Page SHALL showcase the primary capabilities of CSCRS.

The section SHALL use responsive feature cards.

Example features:

- AI Detection
- Image Verification
- Duplicate Detection
- Automatic Assignment
- Risk Assessment
- Live Tracking
- Notifications
- Analytics
- Dashboard
- Resolution Verification
- Secure Authentication
- Role-Based Access

Each feature card SHALL contain:

- Icon
 - Title
 - Description
-

5.14 Dashboard Preview

This section SHALL demonstrate the capabilities of the Administrative Portal.

A realistic dashboard mockup SHALL be displayed inside a laptop frame or browser frame.

The preview SHALL highlight:

- Charts

- Statistics
- Reports
- Analytics
- KPIs

This section SHALL encourage confidence in the administrative capabilities of CSCRS.

5.15 Issue Categories

Supported civic issue categories SHALL be displayed using visual cards.

The default categories SHALL include:

- Road Damage
- Garbage
- Waterlogging
- Street Light
- Damaged Electric Pole
- Open Manhole

Future categories SHALL be configurable.

5.16 Live Statistics

The Landing Page SHALL display real-time platform statistics.

Example metrics:

- Total Reports
- Resolved Reports
- Departments
- Workers
- Average Resolution Time
- Citizen Satisfaction

Whenever backend APIs are available, these values SHALL be populated dynamically.

Animated counters MAY be used.

5.17 AI Pipeline Showcase

This section SHALL present the internal AI verification workflow.

Example:

Image Upload



EXIF Validation



GPS Validation



Timestamp Validation



Image Quality



YOLO Detection



Duplicate Detection



Risk Score

↓

Report Generation

This section SHALL communicate the technological sophistication of the platform.

5.18 Mobile Application Promotion

The Landing Page SHALL promote the Citizen and Worker Mobile Application.

The section SHALL contain:

- Mobile application mockup.
- QR Code.
- Google Play button.
- Future App Store placeholder.
- Download APK button (optional).

A concise description of the application's capabilities SHALL accompany the download options.

5.19 FAQ

Frequently Asked Questions SHALL be presented using an accordion interface.

The initial implementation SHOULD include at least six frequently asked questions.

5.20 Contact Section

The Contact section SHALL contain:

- Office Address
 - Contact Email
 - Phone Number
 - Office Hours
 - Embedded Map (future configurable)
 - Contact Form (optional)
-

5.21 Footer

The Footer SHALL be divided into multiple columns.

Recommended sections:

- About CSCRS
- Quick Links
- Resources
- Mobile Application
- Contact Information

The Footer SHALL also include links to:

- Privacy Policy
 - Terms & Conditions
 - Accessibility Statement
 - Sitemap
-

5.22 Responsive Behaviour

The Landing Page SHALL support:

- Desktop
- Laptop
- Tablet
- Mobile

Navigation SHALL automatically collapse into a hamburger menu on smaller screens.

5.23 Accessibility Requirements

The Landing Page SHALL support:

- Keyboard navigation.
 - Skip to Main Content.
 - Screen readers.
 - ARIA labels.
 - High contrast support.
 - Visible keyboard focus.
 - Accessible forms.
 - Semantic HTML.
-

5.24 Performance Requirements

The Landing Page SHOULD prioritize:

- Fast initial loading.
 - Lazy loading of large illustrations.
 - Optimized images.
 - Responsive image formats.
 - Minimal layout shifts.
 - High Lighthouse performance score.
-

5.25 Future Enhancements

The architecture SHALL permit future additions including:

- News Section
- Announcements
- Blog
- Press Releases
- Multi-language translations
- Live municipal alerts
- Public service notifications

Chapter 6 — Authentication & User Session Management

6.1 Overview

The Authentication Module SHALL serve as the secure entry point to the CSCRS Administrative Web Portal.

Only authenticated administrative users SHALL be permitted to access protected resources.

The authentication system SHALL integrate with the existing FastAPI backend without requiring architectural modifications.

The frontend SHALL consume the existing authentication APIs and SHALL NOT implement independent authentication logic.

6.2 Authentication Philosophy

The portal SHALL provide a **single unified authentication interface** for all administrative roles.

Separate login pages SHALL NOT be developed for:

- Department Administrator
- City Administrator
- Super Administrator

Role selection SHALL NOT be performed manually by users.

Instead, the authenticated role SHALL be determined exclusively by the backend after successful credential verification.

6.3 Authentication Flow

The authentication workflow SHALL follow the sequence below.

Visitor

|



Landing Page

|



Login Page

|



Credential Validation

|



FastAPI Authentication API

|



JWT Token Issued

|



User Profile Retrieval

|



Role Detection

|



Role-Specific Dashboard

6.4 Login Page

The Login Page SHALL be minimal, secure, and distraction-free.

The page SHALL contain:

- Project Logo
- Portal Title
- Welcome Message
- Email Address
- Password
- Show/Hide Password
- Remember Me (Future Enhancement)
- Login Button
- Forgot Password
- Back to Home

The login page SHALL NOT include registration functionality.

Administrative accounts SHALL be created by authorized administrators only.

6.5 Authentication Fields

The login form SHALL contain:

Field	Required
-------	----------

Email Address	Yes
---------------	-----

Password	Yes
----------	-----

The frontend SHALL validate:

- Empty fields
- Email format
- Maximum input length
- Minimum password requirements (UI validation only)

Backend validation SHALL remain the ultimate source of truth.

6.6 Login Process

Upon clicking the Login button, the frontend SHALL:

1. Validate user input.
 2. Display loading state.
 3. Disable repeated submissions.
 4. Invoke the Login API.
 5. Process backend response.
 6. Store authentication token.
 7. Retrieve authenticated user information.
 8. Detect user role.
 9. Redirect to the appropriate dashboard.
-

6.7 Authentication States

The Login Page SHALL support the following states:

Default

User has not submitted credentials.

Loading

Login request is in progress.

The Login button SHALL display a loading indicator.

Repeated submissions SHALL be prevented.

Success

Authentication completed successfully.

The user SHALL be redirected immediately.

Error

Authentication failed.

The page SHALL display an appropriate error message without revealing sensitive information.

6.8 Authentication Errors

The frontend SHALL handle the following scenarios:

- Invalid credentials
- Unauthorized account
- Inactive account
- Email not verified (if applicable)
- Network failure
- Server unavailable
- Session expired
- Unexpected server response

Error messages SHALL remain user-friendly.

Sensitive backend details SHALL NOT be exposed.

6.9 Forgot Password

The Login Page SHALL provide access to the Forgot Password workflow.

The workflow SHALL utilize the existing backend implementation.

The frontend SHALL NOT introduce additional password recovery mechanisms.

6.10 Reset Password

The Reset Password page SHALL be accessible only through valid password reset links generated by the backend.

Expired or invalid reset tokens SHALL redirect users to an appropriate error page.

6.11 Session Management

The frontend SHALL maintain the authenticated session using the existing JWT authentication strategy.

No modifications SHALL be made to the backend authentication architecture during the initial frontend implementation.

Future migration to HttpOnly cookies MAY be considered as a production enhancement.

6.12 Token Storage

The frontend SHALL utilize the existing backend authentication mechanism.

The authentication strategy SHALL remain compatible with:

- Existing JWT implementation
- Existing protected APIs
- Existing authorization middleware

Changes to token storage SHALL be deferred until future production hardening phases.

6.13 Protected Routes

Every authenticated route SHALL require successful authentication.

Unauthenticated users attempting to access protected routes SHALL be redirected to the Login Page.

Protected pages SHALL never render before authentication verification completes.

6.14 Role Detection

Immediately after authentication, the frontend SHALL retrieve the authenticated user's role.

The role SHALL determine:

- Dashboard
- Sidebar
- Navigation
- Quick Actions
- Widgets
- Available Modules
- Available Routes

The frontend SHALL NOT infer roles independently.

Backend responses SHALL remain authoritative.

6.15 Role-Based Redirection

Successful authentication SHALL redirect users as follows:

Role	Destination
Department Administrator	Department Dashboard
City Administrator	City Dashboard
Super Administrator	System Dashboard

Manual role selection SHALL NOT be supported.

6.16 Logout

Logout SHALL:

- Clear authentication state.
- Remove stored authentication information.
- Clear cached protected data.
- Redirect to the Landing Page.

Protected browser history SHALL NOT remain accessible after logout.

6.17 Session Expiration

If authentication expires during portal usage:

The frontend SHALL:

- Display an informative message.
 - Redirect the user to Login.
 - Preserve the intended destination when possible.
-

6.18 Unauthorized Access

Attempting to access unauthorized modules SHALL result in:

- Route blocking.
- Appropriate error page (403).
- No exposure of protected content.

Frontend authorization SHALL complement backend authorization.

Backend authorization SHALL remain the final security authority.

6.19 Security Requirements

The Authentication Module SHALL support:

- Secure communication over HTTPS.
- JWT authentication.
- RBAC integration.
- Rate-limited authentication APIs.
- Secure password handling.

- Future session enhancements.

The frontend SHALL never store passwords.

6.20 Captcha Strategy

The initial implementation SHALL NOT require CAPTCHA during normal authentication.

The system SHALL rely upon:

- Existing backend rate limiting.
- Secure authentication.
- JWT authorization.

Future implementations MAY introduce adaptive CAPTCHA mechanisms based on suspicious authentication behavior.

6.21 Future Authentication Enhancements

The authentication architecture SHALL support future implementation of:

- HttpOnly Secure Cookies
- Refresh Token Rotation
- Device Management
- Active Sessions
- Login History
- Two-Factor Authentication (2FA)
- Single Sign-On (SSO)
- Government Identity Integration

These enhancements SHALL be implemented without redesigning the authentication user interface.

6.22 Chapter Summary

The Authentication Module establishes a secure, centralized entry point into the CSCRS Administrative Portal.

The module SHALL:

- Utilize the existing FastAPI authentication system.
- Maintain a unified login experience.
- Automatically determine administrative roles.

- Redirect users to role-specific dashboards.
- Enforce protected routing.
- Preserve compatibility with future authentication enhancements.

Chapter 7 — Enterprise Portal Layout & Navigation Architecture

7.1 Overview

The CSCRS Administrative Portal SHALL adopt a unified enterprise workspace architecture that remains consistent across all authenticated modules.

Every authenticated screen SHALL inherit the same application shell consisting of:

- Top Navigation Bar
- Collapsible Sidebar
- Breadcrumb Navigation
- Main Workspace
- Notification Center
- User Profile Menu

Only the Main Workspace SHALL change when navigating between modules.

This architecture SHALL ensure consistency, familiarity, and operational efficiency throughout the portal.

7.2 Layout Philosophy

The portal SHALL follow a **Workspace-Oriented Design** rather than a page-oriented design.

Users SHALL remain inside the same application shell throughout their session.

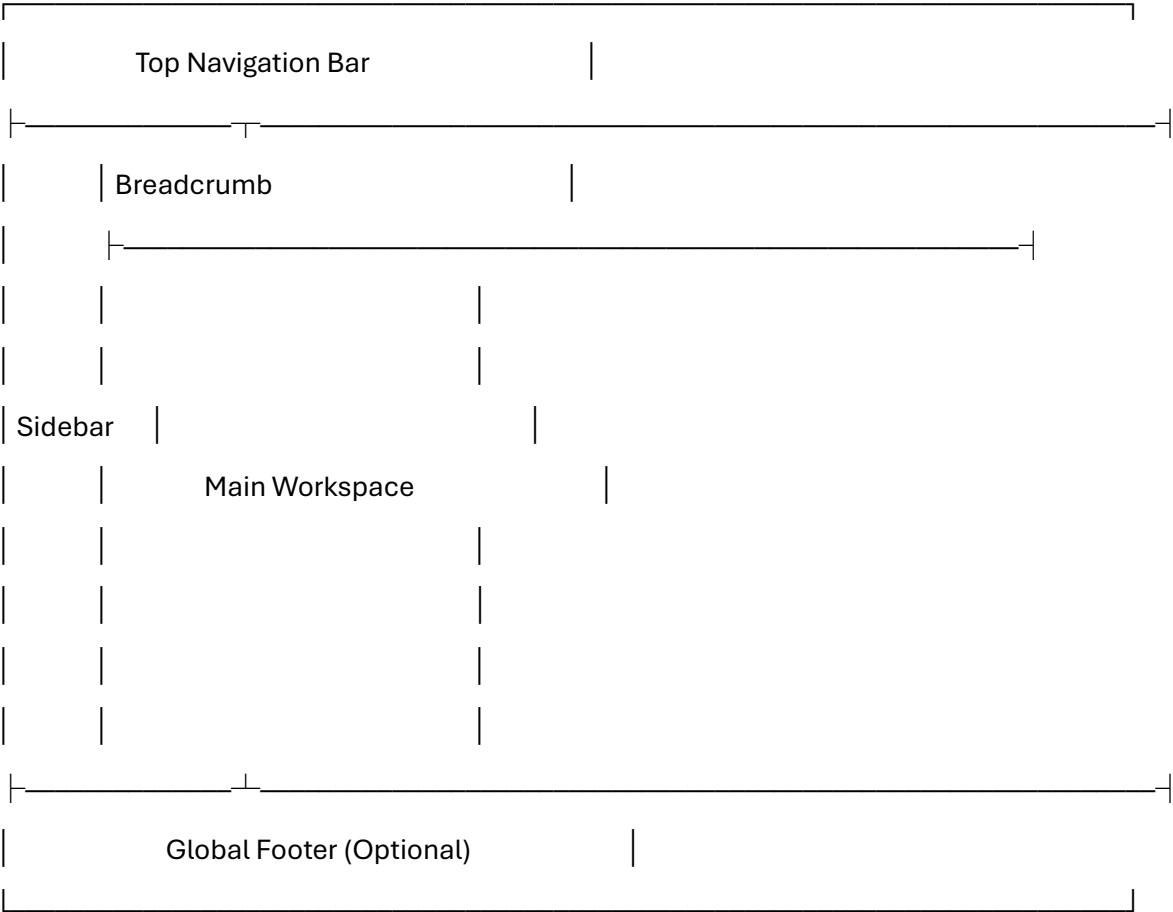
Navigation between modules SHALL update only the workspace content while preserving:

- Sidebar
- Top Navigation
- Search
- Notifications
- Profile Menu

This approach SHALL reduce unnecessary page reloads and maintain user context.

7.3 Overall Portal Layout

The authenticated portal SHALL follow the structure below.



7.4 Top Navigation Bar

The Top Navigation Bar SHALL remain visible throughout the authenticated session.

The navigation bar SHALL contain:

Left Section

- Sidebar Toggle (Hamburger)
- Breadcrumb Navigation

Center Section

- Global Search

Right Section

- Quick Actions (Future)
- Notifications
- Theme Switch (Future)

- User Profile Menu

The navigation bar SHALL remain fixed during scrolling.

7.5 Sidebar Navigation

The Sidebar SHALL serve as the primary navigation mechanism for authenticated users.

The Sidebar SHALL:

- Be collapsible.
- Support nested menus.
- Display icons and labels.
- Highlight the active module.
- Persist its collapsed/expanded state during the session.

The Sidebar SHALL remain visible on desktop devices.

On smaller devices, it SHALL transform into an overlay drawer.

7.6 Sidebar Behavior

The Sidebar SHALL support two modes.

Expanded Mode

Displays:

- Icons
 - Labels
 - Nested navigation
-

Collapsed Mode

Displays:

- Icons only

Labels SHALL appear using tooltips when hovered.

7.7 Role-Based Sidebar

The Sidebar SHALL be dynamically generated after authentication.

Example:

Department Administrator

Dashboard

Reports

Workers

Manual Reviews

Analytics

Notifications

Settings

City Administrator

Dashboard

Reports

Departments

Department Admins

Analytics

Notifications

Settings

Super Administrator

Dashboard

Reports

City Admins

System

Dataset Export

Analytics

Settings

Menus SHALL be generated from permissions rather than hardcoded role checks.

7.8 Main Workspace

The Main Workspace SHALL occupy the majority of the viewport.

The Workspace SHALL dynamically render:

- Dashboard
- Reports
- Analytics
- Forms
- Tables
- Details Pages
- Settings
- Management Modules

The workspace SHALL remain scrollable independently from the Sidebar whenever practical.

7.9 Breadcrumb Navigation

The portal SHALL implement Oracle Cloud–style breadcrumb navigation.

Every page beyond the Dashboard SHALL display its navigation path.

Examples:

Dashboard

Dashboard

Reports

Dashboard > Reports

Pending Reports

Dashboard > Reports > Pending

Report Details

Dashboard > Reports > Pending > Report #CSCRS-2026-001245

Worker Profile

Dashboard > Workers > Rajesh Kumar

Breadcrumbs SHALL be automatically generated from the route hierarchy.

7.10 Dashboard-First Navigation

Immediately after authentication, users SHALL enter their Dashboard.

The Dashboard SHALL function as:

- Information Center
- Navigation Hub
- Operational Workspace

Users SHALL be capable of opening major modules directly from dashboard widgets without navigating through the Sidebar.

7.11 Interactive Dashboard Cards

Dashboard widgets SHALL function as navigation shortcuts.

Example:

Pending Reports

125

Click

↓

Opens

Reports

↓

Pending Reports

Similarly,

High Priority Reports

↓

High Priority Report List

Workers

↓

Worker Management

Departments

↓

Department Overview

Recent Activities

↓

Corresponding Details Page

Dashboard cards SHALL never be informational only.

Whenever applicable, they SHALL navigate to the associated operational screen.

7.12 Global Search

The portal SHALL provide a Global Search located within the Top Navigation Bar.

The search SHALL support:

- Report ID
- Citizen Name (where permitted)
- Worker Name
- Department
- Report Category
- Issue Type

Future enhancements MAY include AI-powered search suggestions.

7.13 Notification Center

The Notification Center SHALL display:

- New Reports

- Assignment Updates
- Resolution Updates
- Administrative Alerts
- System Notifications

Unread notifications SHALL be visually distinguished.

Users SHALL be able to navigate directly to the associated module from a notification.

7.14 User Profile Menu

The User Profile Menu SHALL contain:

- My Profile
- Change Password
- Preferences (Future)
- Help (Future)
- Logout

The profile avatar SHALL remain visible in the Top Navigation Bar.

7.15 Quick Actions

The portal SHALL provide context-aware Quick Actions.

Examples:

Department Administrator

- Create Worker
- View Pending Reports
- Manual Reviews
- Forward Requests

City Administrator

- Create Department
- View Analytics
- Manage Department Admins

Super Administrator

- Create City Administrator
- Export AI Dataset

- System Health

Quick Actions SHALL adapt according to authenticated permissions.

7.16 Workspace Navigation

The portal SHALL operate as a Single Page Application.

Module navigation SHALL update only the Main Workspace.

The following interface elements SHALL remain persistent:

- Sidebar
- Top Navigation
- Breadcrumb
- Notifications
- Profile Menu

This behavior SHALL minimize full-page reloads.

7.17 Page Header

Every authenticated screen SHALL begin with a standardized Page Header.

The Page Header SHALL include:

- Page Title
- Page Description (Optional)
- Primary Action Button
- Secondary Actions
- Filters (Where Applicable)

Example:

Reports

Manage, monitor and review civic issue reports.

[Create Report] [Export]

7.18 Responsive Behavior

Desktop

- Permanent Sidebar
- Full Workspace

Tablet

- Collapsible Sidebar

Mobile Browser

- Drawer Navigation
- Optimized Workspace

The Administrative Portal SHALL primarily target desktop usage.

7.19 Navigation Performance

Navigation SHALL:

- Preserve application state whenever appropriate.
 - Minimize unnecessary API requests.
 - Avoid complete page refreshes.
 - Display loading indicators only when necessary.
-

7.20 User Experience Principles

The portal SHALL prioritize:

- Minimal navigation depth.
- Maximum operational efficiency.
- Clear information hierarchy.
- Predictable interactions.
- Consistent layouts.
- Reduced cognitive load.

No frequently used administrative function SHOULD require excessive navigation.

7.21 Future Enhancements

The navigation architecture SHALL support future additions including:

- Command Palette (Ctrl + K)
- Keyboard Shortcuts
- Favorites

- Recently Visited Pages
- Multiple Workspace Tabs
- Personalized Dashboard Layouts
- Saved Searches

These features SHALL integrate into the existing architecture without requiring structural redesign.

7.22 Chapter Summary

The Enterprise Portal Layout establishes a unified operational workspace for all authenticated administrative users.

The portal SHALL provide:

- Persistent application shell.
- Dashboard-first navigation.
- Role-based Sidebar.
- Oracle-style Breadcrumbs.
- Interactive dashboard widgets.
- Global Search.
- Notification Center.
- Context-aware Quick Actions.
- Single Page Application behavior.

This layout SHALL be inherited by every authenticated module within the CSCRS Administrative Portal.

Chapter 8 — UI Design System & Design Language

8.1 Overview

The CSCRS Administrative Web Portal SHALL follow a unified Design System to ensure visual consistency, usability, accessibility, scalability, and maintainability across all interfaces.

Every page, module, component, and interaction SHALL comply with this design system.

The Design System SHALL act as the single source of truth for frontend development.

8.2 Design Philosophy

The visual identity SHALL combine the following principles:

- Modern Enterprise Software
- Government Trustworthiness
- Minimalism
- Accessibility
- Productivity
- Clarity
- Consistency

The interface SHALL avoid unnecessary visual decoration and SHALL prioritize operational efficiency.

8.3 Visual Identity

The portal SHALL appear as a modern government enterprise application rather than a marketing website.

The visual language SHALL emphasize:

- Clean layouts
 - Consistent spacing
 - High readability
 - Balanced whitespace
 - Professional typography
 - Simple iconography
 - Clear hierarchy
-

8.4 Color System

The portal SHALL define standardized semantic colors rather than hardcoded colors.

The following semantic color tokens SHALL exist.

Token	Usage
Primary	Primary actions
Secondary	Secondary actions
Success	Completed operations
Warning	Pending attention

Token	Usage
Error	Validation and failures
Info	Informational content
Surface	Cards and containers
Background	Application background
Border	Component borders
Text Primary	Primary content
Text Secondary	Secondary content

The actual hexadecimal values SHALL be defined within the implementation theme.

The design SHALL support future rebranding without requiring component modifications.

8.5 Typography

The portal SHALL use a single primary font family throughout the application.

Typography SHALL follow a consistent hierarchy.

Levels SHALL include:

- Display
- Heading 1
- Heading 2
- Heading 3
- Heading 4
- Body Large
- Body
- Caption
- Label

Typography SHALL prioritize readability over decorative styling.

8.6 Spacing System

A standardized spacing scale SHALL be used.

All margins and paddings SHALL reference predefined spacing tokens.

Arbitrary spacing values SHOULD be avoided.

Consistent spacing SHALL improve layout predictability and maintainability.

8.7 Grid System

The application SHALL utilize a responsive grid system.

Desktop layouts SHALL prioritize:

- Multi-column dashboards
- Large data tables
- Side-by-side panels

Tablet layouts SHALL adapt using responsive stacking.

Mobile layouts SHALL minimize horizontal scrolling.

8.8 Border Radius

The portal SHALL maintain consistent corner radii across:

- Buttons
- Cards
- Inputs
- Dialogs
- Tables
- Dropdowns

Different components SHALL NOT use inconsistent rounding styles.

8.9 Shadows & Elevation

Elevation SHALL communicate hierarchy.

Only predefined elevation levels SHALL be used.

Typical levels SHALL include:

- None
- Low
- Medium
- High

Excessive shadow effects SHALL be avoided.

8.10 Iconography

Icons SHALL be:

- Simple
- Consistent
- Vector-based
- Easily recognizable

Icons SHALL always accompany critical navigation items.

Icons SHALL NOT replace textual labels entirely unless sufficient context exists.

8.11 Buttons

The portal SHALL define standardized button variants.

Required variants:

- Primary
- Secondary
- Outline
- Ghost
- Destructive
- Icon Button

Every button SHALL support:

- Default
- Hover
- Focus
- Active
- Disabled
- Loading

States.

8.12 Forms

All forms SHALL follow identical structure.

Each form SHALL contain:

- Label

- Input
- Helper Text
- Validation Message

Required fields SHALL be clearly indicated.

Validation SHALL occur consistently throughout the portal.

8.13 Input Components

Standardized input controls SHALL include:

- Text Input
- Password Input
- Search Input
- Email Input
- Number Input
- Phone Input
- Date Picker
- Time Picker
- Dropdown
- Multi Select
- Text Area
- File Upload
- Image Upload

Input styling SHALL remain identical throughout the application.

8.14 Tables

Administrative data SHALL primarily be displayed using enterprise data tables.

Tables SHALL support:

- Sorting
- Filtering
- Pagination
- Search
- Column Visibility

- Row Selection
- Export (where applicable)

Large datasets SHALL avoid client-side rendering whenever practical.

8.15 Cards

Cards SHALL serve as reusable information containers.

Card types SHALL include:

- Statistic Card
- Feature Card
- Dashboard Widget
- Department Card
- Report Card
- Activity Card

All cards SHALL follow consistent spacing and elevation.

8.16 Status Badges

System status SHALL always be represented using standardized badges.

Examples include:

- Pending
- Approved
- Rejected
- In Progress
- Completed
- Active
- Inactive
- High Priority

Badge appearance SHALL remain consistent throughout the portal.

8.17 Alerts

The portal SHALL support:

- Success Alerts

- Warning Alerts
- Error Alerts
- Information Alerts

Alerts SHALL remain visually distinct while preserving consistency.

8.18 Dialogs & Modals

Dialogs SHALL be used only when user confirmation is required.

Examples:

- Delete Confirmation
- Logout Confirmation
- Block User
- Archive Report

Critical actions SHALL always require explicit confirmation.

8.19 Drawers

Drawers MAY be used for:

- Filters
- Notifications
- Mobile Sidebar
- Quick Details

Drawers SHALL not replace full-page workflows unnecessarily.

8.20 Tabs

Tabs SHALL organize closely related information.

Example:

Report Details

Overview

Timeline

AI Verification

Assignment

Resolution

History

Tabs SHALL avoid page reloads.

8.21 Accordions

Accordions SHALL be used primarily for:

- FAQ
- Advanced Filters
- Expandable Information

They SHALL NOT replace primary navigation.

8.22 Timeline Component

The Timeline component SHALL display chronological events.

Typical usage:

- Report Lifecycle
- Assignment History
- Resolution Updates
- Administrative Activities

Events SHALL display:

- Timestamp
 - Status
 - Description
 - Responsible User (where applicable)
-

8.23 Charts

Charts SHALL present analytics visually.

Supported chart types SHALL include:

- Line Chart
- Bar Chart
- Pie Chart
- Area Chart
- Donut Chart

Charts SHALL always include titles and legends where applicable.

8.24 Empty States

Every module SHALL define an Empty State.

Example:

No Reports Found

Try changing filters

or

Create a New Report

Empty states SHALL guide users toward meaningful actions.

8.25 Loading States

Loading SHALL be communicated using:

- Skeleton Screens
- Inline Loading Indicators
- Progress Indicators

Blocking spinners SHALL be minimized.

Skeleton loading SHALL be preferred for data-intensive pages.

8.26 Error States

Standardized error pages SHALL include:

- 400
- 401
- 403
- 404

- 500

Error pages SHALL remain user-friendly.

8.27 Success Feedback

Successful operations SHALL provide immediate confirmation.

Preferred methods:

- Toast Notifications
- Inline Success Messages

Modal dialogs SHALL not be used for routine success messages.

8.28 Animations

Animations SHALL be subtle and purposeful.

Examples:

- Page transitions
- Card hover
- Dropdown expansion
- Sidebar collapse
- Toast appearance

Decorative animations SHALL be avoided.

8.29 Accessibility Design

Every component SHALL support:

- Keyboard Navigation
- Visible Focus States
- ARIA Labels
- Screen Readers
- High Contrast Compatibility

Accessibility SHALL never be optional.

8.30 Responsive Components

Every reusable component SHALL adapt gracefully across:

- Desktop
- Laptop
- Tablet
- Mobile

Component functionality SHALL remain consistent regardless of screen size.

8.31 Design Consistency Rules

The following practices SHALL be prohibited:

- Multiple button styles for identical actions.
 - Inconsistent spacing.
 - Random typography sizes.
 - Arbitrary colors.
 - Mixed icon styles.
 - Inconsistent border radius.
 - Multiple interaction patterns for identical operations.
-

8.32 Future Design System Expansion

The Design System SHALL support future additions including:

- Dark Mode
- Municipal Branding Themes
- White Label Deployments
- RTL Language Support
- Additional Accessibility Themes

These enhancements SHALL integrate without redesigning existing components.

8.33 Chapter Summary

The UI Design System establishes the visual and interaction standards for the CSCRS Administrative Web Portal.

Every future screen, dashboard, module, and reusable component SHALL inherit this design language, ensuring long-term consistency, maintainability, accessibility, and scalability.

Chapter 9 — Routing, Navigation Flow & Page Architecture

9.1 Overview

The CSCRS Administrative Web Portal SHALL implement a structured routing architecture that provides predictable navigation, secure access control, consistent URL patterns, and efficient page transitions.

The routing system SHALL support:

- Public pages
- Protected pages
- Nested routes
- Dynamic routes
- Role-based routes
- Error routes

The portal SHALL operate as a Single Page Application (SPA), ensuring seamless navigation without full-page reloads.

9.2 Routing Philosophy

The routing architecture SHALL follow these principles:

- Predictable URL structure.
- Human-readable routes.
- Consistent naming conventions.
- Minimal routing depth.
- Secure route protection.
- Lazy-loaded modules where appropriate.
- Deep-link support.
- Browser navigation compatibility.

Every route SHALL correspond to a meaningful business function rather than a backend API endpoint.

9.3 Route Categories

Routes SHALL be categorized into:

Public Routes

Accessible without authentication.

Examples:

- Home
- About
- Features
- Contact
- FAQ
- Login

Protected Routes

Accessible only after successful authentication.

Examples:

- Dashboard
- Reports
- Analytics
- Departments
- Workers
- Notifications
- Settings

Error Routes

Accessible when exceptional conditions occur.

Examples:

- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Internal Error

9.4 Public Route Hierarchy

The public website SHALL expose the following routes.

/

/about

/features

/how-it-works

/departments

/statistics

/faq

/contact

/login

/privacy-policy

/terms

/accessibility

/sitemap

Public routes SHALL remain accessible without authentication.

9.5 Protected Route Hierarchy

All authenticated routes SHALL exist beneath the authenticated application shell.

Example:

/dashboard

/reports

/reports/:reportId

/analytics

/notifications

/settings

Additional routes SHALL be rendered dynamically according to RBAC permissions.

9.6 Nested Routing

Nested routing SHALL be used for complex modules.

Example:

Reports

↓

All Reports

↓

Pending Reports

↓

Report Details

↓

Timeline

Corresponding route hierarchy:

/reports

/reports/pending

/reports/high-priority

/reports/:reportId

/reports/:reportId/timeline

Nested routes SHALL improve navigation clarity.

9.7 Dynamic Routes

Dynamic routes SHALL be used for entity-specific pages.

Examples include:

/reports/:reportId

/workers/:workerId

/departments/:departmentId

/users/:userId

Dynamic parameters SHALL be validated before page rendering.

9.8 Route Naming Convention

Route names SHALL:

- Use lowercase letters.
- Use hyphens for multiple words.
- Avoid underscores.
- Avoid abbreviations whenever possible.

Example:

✓ /manual-reviews

✓ /department-admins

✗ /DeptAdmin

✗ /manual_reviews

9.9 Protected Route Behavior

Protected routes SHALL perform authentication verification before rendering.

The routing flow SHALL follow:

User Requests Route

↓

Authentication Check

↓

Role Verification

↓

Authorized?

↓

Yes → Render Page

↓

No → Redirect

Unauthorized content SHALL never be rendered.

9.10 Role-Based Route Access

Every protected route SHALL define authorized roles.

Example:

Route	Department Admin	City Admin	Super Admin
Dashboard	✓	✓	✓
Workers	✓	✗	✗
Departments	✗	✓	✓
City Admins	✗	✗	✓

Frontend route protection SHALL complement backend authorization.

Backend authorization SHALL remain authoritative.

9.11 Default Route

After successful authentication:

Department Administrator SHALL be redirected to:

/dashboard

City Administrator SHALL be redirected to:

/dashboard

Super Administrator SHALL also be redirected to:

/dashboard

The dashboard SHALL dynamically adapt according to the authenticated role.

Separate dashboard URLs SHALL NOT be created.

9.12 Browser Navigation

Browser Back and Forward navigation SHALL function normally.

The application SHALL preserve:

- Current module
- Current filters
- Current pagination
- Current search query
- Current tab

whenever practical.

9.13 Deep Linking

Users SHALL be able to directly access bookmarked URLs.

Example:

/reports/CSCRS-2026-000451

The application SHALL restore the requested page after authentication.

9.14 Breadcrumb Integration

Breadcrumbs SHALL synchronize automatically with the active route hierarchy.

Example:

Dashboard

>

Reports

>

Pending

>

Report Details

Manual breadcrumb management SHOULD be avoided.

9.15 Route Transitions

Navigation SHALL prioritize speed.

Route transitions SHALL:

- Avoid unnecessary reloads.
- Preserve layout.
- Display loading indicators only when required.
- Maintain application responsiveness.

9.16 Page Refresh Behavior

Refreshing the browser SHALL NOT log the user out unless the authentication session has expired.

The application SHALL restore the current route after successful authentication verification.

9.17 Scroll Restoration

Navigation SHALL follow:

- New page → Scroll to top.
- Detail navigation → Scroll to top.
- Browser Back → Restore previous scroll position whenever feasible.

This behavior SHALL improve user experience in long tables and dashboards.

9.18 Route-Level Error Handling

Invalid routes SHALL redirect to standardized error pages.

Examples:

401

Unauthorized

403

Access Denied

404

Page Not Found

500

Something Went Wrong

Error pages SHALL maintain the portal's visual identity.

9.19 Lazy Loading

Large modules SHOULD be lazy-loaded.

Examples:

- Analytics
- AI Dataset Export
- Large Management Modules

Critical routes SHALL load immediately.

9.20 URL Stability

Public URLs SHALL remain stable across releases whenever possible.

Frequent URL restructuring SHOULD be avoided to preserve bookmarks and documentation references.

9.21 Future Route Expansion

The routing architecture SHALL support future additions including:

- GIS Dashboard
- Audit Logs
- System Health
- Active Sessions
- AI Recommendations
- Smart City Integrations

without requiring fundamental restructuring.

9.22 Route Inventory (High-Level)

The portal SHALL include, at minimum, the following top-level routes:

/

login

/dashboard

/reports

/analytics

/notifications

/settings

Module-specific routes SHALL be documented in later chapters.

9.23 Navigation Principles

The routing system SHALL prioritize:

- Predictability
- Consistency
- Security
- Performance
- Maintainability
- Discoverability

Every route SHALL represent a logical business capability rather than technical implementation details.

9.24 Chapter Summary

The Routing and Navigation Architecture establishes the structural framework of the CSCRS Administrative Portal.

It defines:

- Public and protected routes.
- Nested navigation.
- Dynamic routing.
- Role-based access.
- Browser behavior.
- Deep linking.
- URL conventions.
- Error handling.
- Future scalability.

This routing architecture SHALL serve as the implementation blueprint for the frontend navigation system.

Chapter 10 — Dashboard Specification

10.1 Overview

The Dashboard SHALL serve as the primary operational workspace of the CSCRS Administrative Portal.

Immediately after successful authentication, every authenticated administrator SHALL be redirected to the Dashboard.

The Dashboard SHALL function as:

- Information Center
- Operational Control Center
- Navigation Hub
- Monitoring Workspace
- Decision Support Interface

The Dashboard SHALL present role-specific information while maintaining a consistent layout across all administrative roles.

10.2 Dashboard Philosophy

The Dashboard SHALL prioritize operational efficiency over visual decoration.

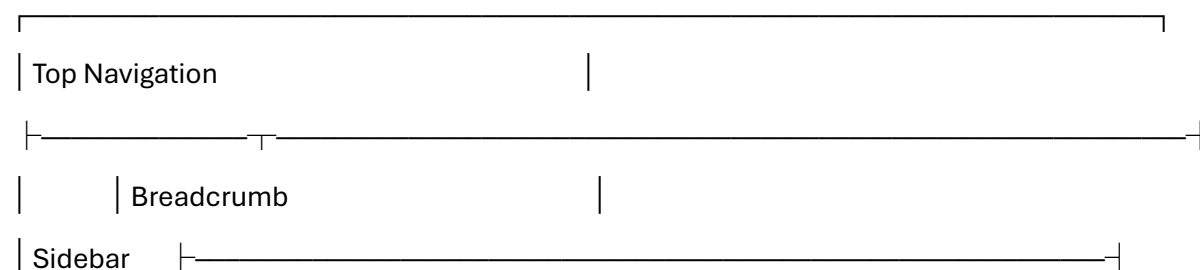
Every widget displayed on the Dashboard SHALL either:

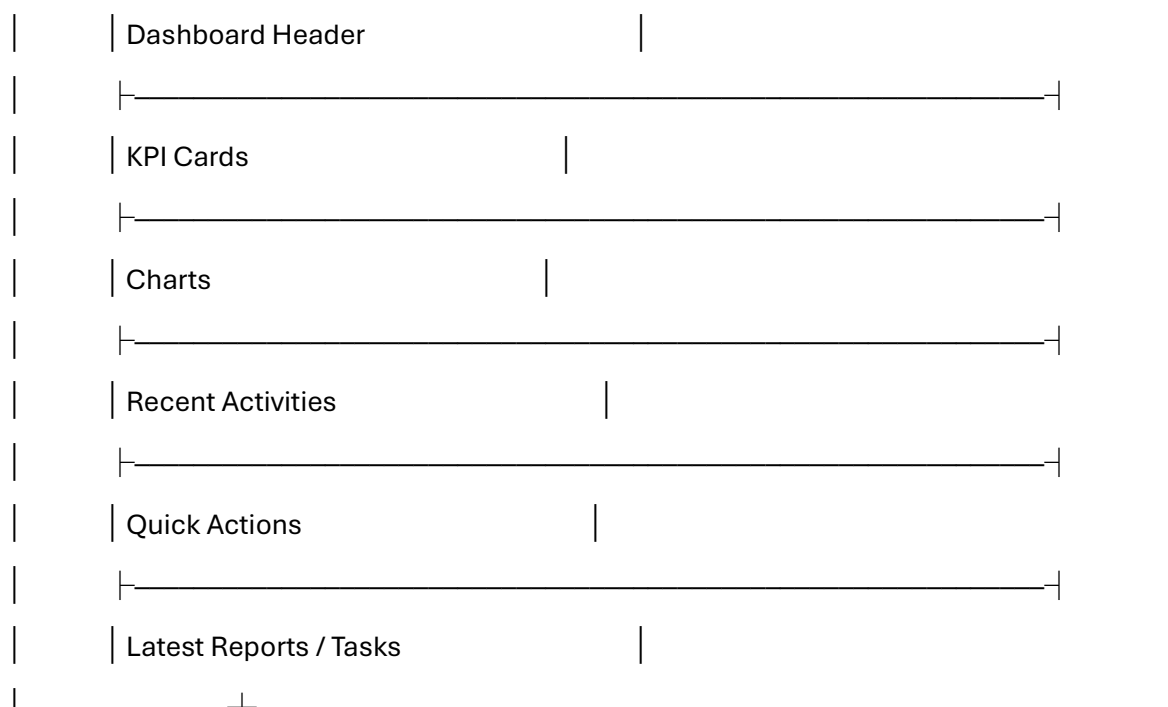
- Provide actionable information, or
- Navigate the user to a related operational module.

Widgets SHALL NOT exist solely for aesthetic purposes.

10.3 Common Dashboard Layout

Every dashboard SHALL inherit the Enterprise Portal Layout.





10.4 Dashboard Components

Every dashboard SHALL contain the following logical sections.

- Page Header
- Welcome Section
- KPI Cards
- Analytics Widgets
- Charts
- Recent Activities
- Quick Actions
- Operational Tables

Additional sections SHALL appear according to authenticated permissions.

10.5 Dashboard Header

The Dashboard Header SHALL display:

- Dashboard Title
- Current Date
- Welcome Message
- Organization Name (Future Configurable)

Example:

Good Morning, Rahul Sharma

Department Administrator

Monday, 28 July 2026

10.6 KPI Cards

KPI cards SHALL provide high-level operational summaries.

Typical cards include:

- Total Reports
- Pending Reports
- In Progress
- Resolved Reports
- High Priority Reports
- Average Resolution Time

Each KPI SHALL include:

- Icon
- Value
- Label
- Trend Indicator (Future)
- Click Navigation

10.7 Interactive Widgets

Every dashboard card SHALL be clickable whenever meaningful.

Example:

Pending Reports

125

↓

Navigate to

Reports



Pending Reports

Similarly,

Resolved Reports



Resolved Report List

Workers



Worker Management

Departments



Department Management

10.8 Dashboard Charts

The Dashboard SHALL support analytical visualizations.

Typical charts MAY include:

- Daily Reports
- Monthly Reports
- Resolution Trends
- Department Performance
- Issue Distribution
- Category Analysis
- Resolution Time Analysis

Charts SHALL support responsive resizing.

10.9 Recent Activities

The Dashboard SHALL present recent administrative activities.

Examples:

- New Report Submitted
- Worker Assigned
- Resolution Uploaded
- Report Closed
- Department Updated

Each activity SHALL link to its corresponding detail page.

10.10 Quick Actions

Quick Actions SHALL provide direct access to frequently used operations.

The displayed actions SHALL depend upon authenticated role.

Examples:

Department Administrator

- View Pending Reports
- Manage Workers
- Manual Reviews
- Forward Requests

City Administrator

- Manage Departments
- Manage Department Administrators
- View Analytics
- High Priority Reports

Super Administrator

- Manage City Administrators
 - Export Dataset
 - View System Health
 - Platform Configuration
-

10.11 Department Administrator Dashboard

The Department Administrator Dashboard SHALL focus on departmental operations.

Primary widgets SHALL include:

- Pending Reports

- Assigned Workers
- Manual Reviews
- Forward Requests
- Department Performance
- Resolution Rate
- Recent Reports
- Worker Availability

The dashboard SHALL emphasize operational efficiency.

10.12 City Administrator Dashboard

The City Administrator Dashboard SHALL provide city-wide visibility.

Widgets SHALL include:

- Total Reports
- Active Departments
- Department Performance
- Resolution Statistics
- Citizen Satisfaction
- High Priority Reports
- Department Comparison
- Monthly Trends

The dashboard SHALL support strategic monitoring rather than daily operational tasks.

10.13 Super Administrator Dashboard

The Super Administrator Dashboard SHALL provide system-wide oversight.

Widgets SHALL include:

- City Administrators
- Department Count
- Platform Usage
- Dataset Export Status
- System Health
- Storage Utilization (Future)

- API Health (Future)
 - Platform Statistics
-

10.14 Dashboard Tables

The Dashboard MAY contain operational tables.

Examples:

- Recent Reports
- Pending Reviews
- Worker Status
- Department Status

Tables SHALL support:

- Sorting
 - Pagination
 - Quick Navigation
-

10.15 Dashboard Filters

Where applicable, dashboard data SHALL support filtering.

Examples:

- Today
- Last 7 Days
- Last 30 Days
- Current Month
- Current Year

Filter selections SHALL refresh dashboard widgets accordingly.

10.16 Empty Dashboard State

If no operational data exists,

the Dashboard SHALL present an informative empty state.

Example:

No Reports Available

Reports submitted by citizens

will appear here.

Empty dashboards SHALL never appear broken.

10.17 Loading State

During data retrieval,

dashboard widgets SHALL display Skeleton Loaders.

Loading SHALL occur independently for each widget whenever practical.

The entire dashboard SHOULD NOT be blocked by a single slow API response.

10.18 Error Handling

Dashboard failures SHALL gracefully display:

- Widget-level errors
- Retry option
- Friendly error messages

Failure of one widget SHALL NOT prevent other widgets from loading.

10.19 Dashboard Refresh Strategy

Dashboard data SHALL refresh:

- On first load.
- On manual refresh.
- After critical operations affecting displayed metrics.

Future versions MAY support automatic background refresh.

10.20 Personalization (Future Enhancement)

Future versions MAY allow users to:

- Rearrange widgets
- Hide widgets
- Pin favorite widgets
- Save dashboard layouts

The initial implementation SHALL use a standardized layout.

10.21 Dashboard Performance

The Dashboard SHALL:

- Load efficiently.
- Minimize API calls.
- Reuse cached data where appropriate.
- Lazy load non-critical widgets.
- Maintain smooth scrolling.

10.22 Dashboard Accessibility

Dashboard widgets SHALL support:

- Keyboard Navigation
- Screen Readers
- Accessible Charts
- High Contrast Mode

Interactive widgets SHALL expose appropriate ARIA labels.

10.23 Dashboard Security

Dashboard information SHALL respect Role-Based Access Control.

Users SHALL only receive:

- Authorized metrics.
- Authorized reports.
- Authorized analytics.

Sensitive administrative information SHALL never be exposed to unauthorized roles.

10.24 Dashboard API Integration

Dashboard widgets SHALL consume dedicated backend APIs.

Multiple related APIs MAY be aggregated through a frontend service layer to minimize unnecessary component complexity.

Dashboard components SHALL remain independent from API implementation details.

10.25 Future Dashboard Enhancements

Future versions MAY introduce:

- Real-Time Updates
- WebSocket Notifications
- AI Recommendations
- Predictive Analytics
- Interactive Maps
- Custom KPIs
- Multi-City Monitoring
- Personalized Insights

These enhancements SHALL integrate into the existing dashboard architecture.

10.26 Chapter Summary

The Dashboard SHALL function as the operational command center of the CSCRS Administrative Portal.

It SHALL provide:

- Role-specific operational information.
- Interactive KPI widgets.
- Charts and analytics.
- Recent activities.
- Quick actions.
- Navigation shortcuts.
- Secure, role-aware data presentation.

The Dashboard SHALL become the primary workspace from which administrators perform the majority of daily operations.

Chapter 11 — Report Management Module

11.1 Overview

The Report Management Module SHALL serve as the central operational module of the CSCRS Administrative Portal.

This module SHALL provide authorized administrators with complete visibility into the lifecycle of civic issue reports submitted through the Citizen Mobile Application.

The module SHALL enable administrators to:

- Monitor reports.
- Review report details.
- Track report progress.
- Review AI verification results.
- Manage assignments.
- Monitor resolutions.
- Analyze report status.
- Perform administrative actions according to RBAC permissions.

This SHALL be the most frequently used module within the Administrative Portal.

11.2 Objectives

The Report Management Module SHALL:

- Centralize report operations.
 - Improve administrative efficiency.
 - Reduce report processing time.
 - Improve issue visibility.
 - Simplify report tracking.
 - Provide complete lifecycle transparency.
 - Integrate AI verification results.
 - Support future analytics.
-

11.3 Module Entry Points

Users SHALL be able to enter the Report Module through multiple navigation paths.

Dashboard Widget

↓

Reports Sidebar

↓

Global Search

↓

Notifications

↓

Quick Actions

↓

Deep Link

Every entry SHALL navigate to the same Report Management module.

11.4 Module Structure

The module SHALL contain the following screens.

Reports

|

|— All Reports

|— Pending Reports

|— In Progress

|— Resolved

|— High Priority

|— Manual Review

|— Duplicate Reports

|— Archived Reports (Future)

└ Report Details

11.5 Report List Page

The Report List SHALL display civic issue reports in a structured enterprise data table.

The page SHALL become the default landing page of the Report Management Module.

11.6 Report Table

The Report Table SHALL support enterprise-grade functionality.

Columns SHALL include:

- Report ID
- Issue Category
- Current Status
- Priority
- Department
- Assigned Worker
- Submitted By
- Submission Date
- Last Updated
- Actions

Future columns MAY include configurable visibility.

11.7 Table Features

The Report Table SHALL support:

- ✓ Pagination
- ✓ Search
- ✓ Multi-column Sorting
- ✓ Filtering
- ✓ Export
- ✓ Row Selection
- ✓ Column Visibility
- ✓ Refresh
- ✓ Responsive Layout

The table SHALL support large datasets efficiently.

11.8 Search

The Report Module SHALL provide instant searching.

Users SHALL be able to search by:

- Report ID
- Citizen Name (subject to permissions)
- Worker Name
- Department
- Issue Category
- Location
- Status

Search SHALL be server-side whenever practical.

11.9 Filters

The module SHALL provide advanced filters.

Filters SHALL include:

Status

Priority

Department

Issue Category

Date Range

Assigned Worker

AI Verification Result

Risk Level

Duplicate Status

Future filters MAY be added without redesigning the interface.

11.10 Sorting

Users SHALL be able to sort reports by:

Newest

Oldest

Priority

Status

Department

Submission Date

Resolution Date

Sorting SHALL be server-side.

11.11 Report Status

Reports SHALL visually indicate their current lifecycle stage.

Examples:

Pending

AI Verification

Manual Review

Assigned

In Progress

Resolved

Rejected

Closed

Every status SHALL use standardized status badges.

11.12 Report Card (Responsive)

On smaller screens,

the Report Table SHALL automatically transform into responsive Report Cards.

Each card SHALL contain:

Report ID

Issue

Priority

Status

Department

Date

Quick Actions

11.13 Report Details

Selecting a report SHALL open the Report Details workspace.

The Report Details page SHALL become the central workspace for a single civic issue.

11.14 Report Detail Layout

The Report Details page SHALL use a tab-based layout.

Overview

Timeline

AI Verification

Assignment

Resolution

History

Tabs SHALL update content without reloading the page.

11.15 Overview Tab

The Overview SHALL present:

Report Summary

Citizen Information (RBAC Controlled)

Issue Details

Department

Priority

Location

Images

Status

Submission Information

11.16 Timeline Tab

The Timeline SHALL display the complete lifecycle of the report.

Example:

Citizen Submitted

↓

AI Verification

↓

Department Assigned

↓

Worker Assigned

↓

Resolution Submitted

↓

Verification Completed

↓

Closed

Each event SHALL display:

Timestamp

Status

Description

Responsible User

11.17 AI Verification Tab

This tab SHALL present AI-generated insights.

Examples:

Issue Detection

Confidence Score

Image Quality

GPS Validation

Timestamp Validation

Duplicate Detection

Risk Score

Verification Decision

The frontend SHALL only display information received from backend APIs.

11.18 Assignment Tab

The Assignment section SHALL display:

Assigned Department

Assigned Worker

Assignment Date

Current Assignment Status

Assignment History

Authorized administrators MAY perform assignment-related actions according to RBAC permissions.

11.19 Resolution Tab

The Resolution section SHALL display:

Resolution Images

Resolution Notes

Completion Date

Verification Status

Citizen Feedback (if available)

11.20 History Tab

The History tab SHALL record administrative actions.

Examples:

Status Changed

Assignment Updated

Priority Modified

Department Changed

Administrative Notes

This information SHALL support auditing and traceability.

11.21 Quick Actions

Depending upon RBAC,
the Report Details page MAY display actions including:

Assign Worker

Forward Report

Change Priority

Update Status

Add Notes

Download Report

Export Details

Actions SHALL only appear when authorized.

11.22 Image Viewer

Uploaded images SHALL support:

Preview

Zoom

Fullscreen

Image Navigation (Multiple Images)

Download (Authorized Roles Only)

Future versions MAY include annotation capabilities.

11.23 Map Integration (Future)

Future versions MAY display:

Interactive Map

GPS Coordinates

Location Accuracy

Nearby Reports

The layout SHALL reserve space for future GIS integration.

11.24 Notifications

Administrative actions affecting reports SHALL generate notifications according to backend workflows.

The frontend SHALL reflect notification status but SHALL NOT independently generate business events.

11.25 Empty State

When no reports exist,
the module SHALL display:

No Reports Found

Reports submitted by citizens

will appear here.

The interface SHALL never appear broken or incomplete.

11.26 Loading State

The module SHALL use:

Skeleton Tables

Skeleton Cards

Skeleton Detail Views

Blocking loading indicators SHOULD be avoided.

11.27 Error Handling

Errors SHALL be displayed using standardized alert components.

Typical scenarios:

Network Failure

Permission Denied

Invalid Report

Deleted Report

Server Error

Users SHALL receive actionable guidance where appropriate.

11.28 Performance

The Report Module SHALL support:

Large datasets

Server-side Pagination

Server-side Filtering

Server-side Sorting

Efficient API usage

Lazy Loading

11.29 Accessibility

The Report Module SHALL support:

Keyboard Navigation

Screen Readers

Accessible Tables

Accessible Tabs

ARIA Labels

High Contrast Mode

11.30 Security

The module SHALL respect:

Authentication

RBAC

Protected Routes

Secure API Communication

Frontend SHALL never expose unauthorized report information.

11.31 API Integration

The module SHALL consume existing backend APIs for:

- Report Listing
- Report Details
- Timeline

- Assignment
- Resolution
- AI Verification
- Search
- Filtering
- Status Updates (Authorized)
- Export (Where Applicable)

The frontend SHALL not duplicate business logic that already exists in the backend.

11.32 Future Enhancements

The Report Module SHALL support future additions including:

- GIS Map View
 - Bulk Operations
 - AI Recommendations
 - Smart Filters
 - Saved Searches
 - Report Comparison
 - Heat Maps
 - Offline Export
 - Report Sharing (Role Controlled)
-

11.33 Chapter Summary

The Report Management Module SHALL function as the operational core of the CSCRS Administrative Portal.

It SHALL provide:

- Enterprise-grade report management.
- Complete lifecycle visibility.
- AI verification insights.
- Timeline tracking.
- Assignment monitoring.
- Resolution management.

- Secure role-based operations.
- Scalable architecture for future enhancements.

Chapter 12 — Worker Management Module

12.1 Overview

The Worker Management Module SHALL provide authorized administrators with the ability to manage municipal workers responsible for resolving civic issues reported through the CSCRS ecosystem.

The module SHALL support the complete lifecycle of worker administration, including creation, activation, profile management, availability monitoring, assignment visibility, status management, and administrative actions.

The module SHALL integrate with the existing backend Worker Management APIs and SHALL NOT duplicate business logic already implemented on the server.

12.2 Objectives

The Worker Management Module SHALL:

- Centralize worker administration.
 - Simplify workforce management.
 - Improve assignment visibility.
 - Monitor worker availability.
 - Track worker performance.
 - Maintain accurate worker records.
 - Support secure administrative operations.
-

12.3 Business Workflow

The worker management workflow SHALL follow the sequence below.

Create Worker

|

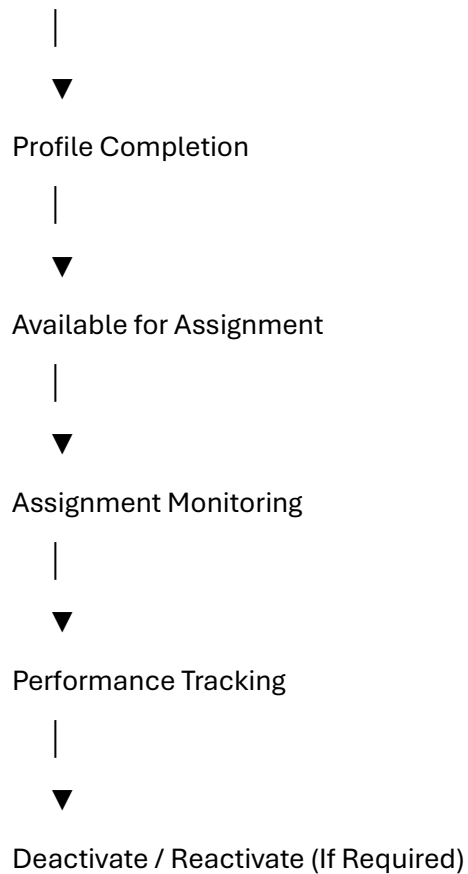


Invitation Sent

|



Worker Activates Account



12.4 Module Entry Points

The Worker Management Module SHALL be accessible through:

- Sidebar Navigation
- Dashboard Quick Actions
- Dashboard KPI Widgets
- Global Search
- Notification Links

All entry points SHALL navigate to the same module.

12.5 Module Structure

The module SHALL consist of the following screens.

Workers

|

└─ All Workers

└─ Active Workers

└─ Inactive Workers

└─ Available Workers

└─ Busy Workers

└─ Worker Details

└─ Create Worker

└─ Edit Worker

12.6 RBAC

Capability	Department Admin	City Admin	Super Admin
View Workers	✓	View Only	View Only
Create Worker	✓	✗	✗
Edit Worker	✓	✗	✗
Activate Worker	✓	✗	✗
Deactivate Worker	✓	✗	✗
Delete Worker	✗ (Soft Delete Only if Supported)	✗	✗

The frontend SHALL hide actions not permitted for the authenticated role.

12.7 Worker List Page

The Worker List SHALL display all authorized workers in an enterprise data table.

Columns SHALL include:

- Employee ID
- Full Name
- Department
- Phone Number
- Email
- Availability
- Current Assignment Count
- Account Status
- Last Active
- Actions

Future columns MAY be configured dynamically.

12.8 Search & Filters

The Worker Module SHALL support:

Search by:

- Worker Name
- Employee ID
- Email
- Phone Number

Filters:

- Department
- Availability
- Account Status
- Assignment Status
- Date Joined

Search and filtering SHALL be server-side.

12.9 Worker Details

Selecting a worker SHALL open the Worker Details workspace.

The page SHALL present information using tab navigation.

Overview

Assignments

Performance

Availability

Activity

History

12.10 Overview Tab

The Overview tab SHALL display:

- Profile Photo (if available)
- Employee ID
- Full Name
- Contact Information
- Department
- Designation
- Joining Date
- Current Status
- Availability
- Current Assignment Count

12.11 Assignments Tab

The Assignments tab SHALL display:

- Current Assignments
- Completed Assignments
- Pending Assignments
- Average Resolution Time
- Assignment Timeline

Each assignment SHALL provide direct navigation to the corresponding report.

12.12 Performance Tab

The Performance section SHALL display performance indicators.

Examples include:

- Total Assigned Reports
- Completed Reports
- Average Resolution Time
- Resolution Success Rate
- Citizen Feedback Rating (if available)
- Monthly Performance

This data SHALL be retrieved from backend APIs.

12.13 Availability Tab

The Availability section SHALL display:

- Current Availability
- Last Status Update
- Current Workload
- Assignment Capacity (Future)

Department Administrators MAY update availability when permitted by backend rules.

12.14 Activity Timeline

The Activity Timeline SHALL record important worker events.

Examples:

- Worker Created
- Invitation Sent
- Account Activated
- Assignment Accepted
- Assignment Completed
- Availability Updated
- Profile Updated

Events SHALL include timestamps.

12.15 Create Worker

The Create Worker page SHALL include:

- Full Name
- Email Address
- Phone Number
- Employee ID
- Department
- Designation (Future)
- Submit Button

The frontend SHALL validate all required fields before submission.

12.16 Worker Invitation

Upon successful creation,

the frontend SHALL display confirmation that an invitation has been sent.

Invitation handling SHALL remain entirely managed by the backend.

The frontend SHALL only display the resulting status.

12.17 Edit Worker

Authorized administrators SHALL be able to edit permitted worker information.

Fields SHALL be pre-populated using backend data.

Read-only fields SHALL remain visually distinguishable.

12.18 Worker Status Management

The module SHALL visually distinguish worker states.

Examples:

- Active
- Inactive
- Available
- Busy

- Suspended (Future)

Standardized status badges SHALL be used throughout the portal.

12.19 Quick Actions

Authorized actions MAY include:

- View Details
- Edit Worker
- Activate
- Deactivate
- View Assignments
- View Performance
- Resend Invitation (if backend supports it)

Actions SHALL only appear when permitted.

12.20 Empty State

When no workers exist,
the module SHALL display:
No Workers Found

Create a Worker to begin assigning civic issue reports.

A primary action button SHALL navigate directly to the Create Worker page.

12.21 Loading State

The Worker Module SHALL utilize:

- Skeleton Tables
- Skeleton Profile Cards
- Skeleton Detail Views

Loading indicators SHALL appear independently for individual sections whenever possible.

12.22 Error Handling

The module SHALL gracefully handle:

- Network failures
- Permission denied
- Worker not found
- Duplicate employee ID
- Duplicate email
- Server errors

Error messages SHALL be user-friendly and SHALL NOT expose backend implementation details.

12.23 Performance Requirements

The Worker Module SHALL support:

- Server-side pagination
 - Server-side filtering
 - Efficient search
 - Lazy-loaded detail sections
 - Optimized API utilization
-

12.24 Accessibility

The module SHALL support:

- Keyboard navigation
 - Screen readers
 - Accessible tables
 - Accessible forms
 - ARIA-compliant controls
 - High-contrast compatibility
-

12.25 Security

The Worker Module SHALL enforce:

- Authentication
- RBAC
- Protected routes

- Secure API communication

The frontend SHALL never expose worker information outside the authenticated user's authorized scope.

12.26 API Integration

The module SHALL integrate with backend APIs responsible for:

- Worker Listing
- Worker Details
- Worker Creation
- Worker Update
- Worker Activation
- Worker Deactivation
- Invitation Status
- Worker Performance
- Worker Assignments

Business rules SHALL remain exclusively implemented within the backend.

12.27 Future Enhancements

The architecture SHALL support future additions including:

- Worker Location Tracking
- GPS Attendance
- Shift Management
- Leave Management
- Attendance Calendar
- Workload Prediction
- AI-based Worker Recommendation
- Performance Leaderboards

These enhancements SHALL integrate without redesigning the existing module.

12.28 Chapter Summary

The Worker Management Module SHALL provide a centralized interface for managing municipal workers throughout their operational lifecycle.

It SHALL enable authorized administrators to:

- Create and activate workers.
- Monitor assignments.
- Review performance.
- Manage availability.
- Perform secure administrative actions.
- Maintain workforce visibility.

The module SHALL remain scalable and compatible with future workforce management capabilities.

Chapter 13 — Department Management Module

13.1 Overview

The Department Management Module SHALL provide authorized administrators with the ability to manage municipal departments responsible for resolving civic issues within the CSCRS ecosystem.

The module SHALL serve as the administrative center for department configuration, monitoring, operational statistics, and department-level performance.

Only authorized administrative roles SHALL be permitted to perform department management operations.

The frontend SHALL integrate with the existing backend Department Management APIs and SHALL not duplicate backend business logic.

13.2 Objectives

The Department Management Module SHALL:

- Centralize department administration.
 - Maintain department information.
 - Monitor departmental performance.
 - Provide department-level analytics.
 - Improve administrative visibility.
 - Support scalable organizational growth.
 - Simplify inter-department coordination.
-

13.3 Business Workflow

Department Created



Department Activated



Department Administrator Assigned



Workers Assigned



Reports Assigned



Department Performance Monitoring



Department Updated / Archived (Future)

13.4 Module Entry Points

The Department Management Module SHALL be accessible through:

- Sidebar Navigation
- Dashboard Widgets
- Quick Actions
- Global Search
- Analytics
- Notifications

All entry points SHALL navigate to the same Department Management workspace.

13.5 RBAC

Capability	Department Admin	City Admin	Super Admin
View Departments	View Own	✓	✓
Create Department	✗	✓	✓
Edit Department	✗	✓	✓
Activate Department	✗	✓	✓
Deactivate Department	✗	✓	✓
Delete Department	✗	✗	✗ (Future Policy Based)

Department Administrators SHALL only view information relevant to their assigned department.

13.6 Module Structure

Departments

|

|— All Departments

|— Active Departments

|— Inactive Departments

|— Department Details

|— Create Department

|— Edit Department

└ Department Analytics

13.7 Department List Page

The Department List SHALL present all authorized departments in an enterprise data table.

Columns SHALL include:

- Department Name
- Department Code
- Department Administrator
- Worker Count
- Active Reports
- Completed Reports
- Status
- Created Date
- Actions

Future columns MAY be configurable.

13.8 Search & Filters

Search SHALL support:

- Department Name
- Department Code
- Department Administrator

Filters SHALL include:

- Status
- Active Reports
- Performance
- Created Date

Filtering SHALL be server-side.

13.9 Department Details

Selecting a department SHALL open the Department Details workspace.

The page SHALL contain the following tabs.

Overview

Department Admin

Workers

Reports

Performance

History

13.10 Overview Tab

The Overview SHALL display:

- Department Name
 - Department Code
 - Description
 - Current Status
 - Department Administrator
 - Total Workers
 - Active Reports
 - Completed Reports
 - Pending Reports
-

13.11 Department Administrator Tab

This section SHALL display:

- Assigned Department Administrator
- Contact Information
- Assignment Date
- Current Status

Authorized users MAY change the assigned administrator according to backend permissions.

13.12 Workers Tab

The Workers tab SHALL display:

- Total Workers
- Active Workers
- Available Workers
- Busy Workers

Each worker SHALL provide navigation to the Worker Details module.

13.13 Reports Tab

The Reports tab SHALL display department-specific reports.

Examples include:

- Pending Reports
- Assigned Reports
- High Priority Reports
- Resolved Reports

Each report SHALL provide navigation to Report Details.

13.14 Performance Tab

Performance metrics SHALL include:

- Resolution Rate
- Average Resolution Time
- Pending Report Count
- Monthly Performance
- Workload Distribution
- Citizen Satisfaction (Future)

Charts MAY be displayed where appropriate.

13.15 History Tab

The History tab SHALL maintain an audit timeline.

Typical events include:

- Department Created
- Administrator Assigned
- Status Updated

- Configuration Modified
- Department Reactivated

Each event SHALL include timestamp and responsible administrator where available.

13.16 Create Department

The Create Department page SHALL contain:

- Department Name
- Department Code
- Description
- Initial Status

The frontend SHALL validate mandatory fields before submission.

13.17 Edit Department

Authorized users SHALL be able to update editable department information.

Read-only fields SHALL be visually distinguished.

13.18 Department Status

Departments SHALL use standardized status badges.

Supported states include:

- Active
 - Inactive
 - Under Maintenance (Future)
 - Archived (Future)
-

13.19 Quick Actions

Depending upon RBAC,

available actions MAY include:

- View Details
- Edit Department
- Assign Administrator
- View Workers

- View Reports
- View Analytics

Unauthorized actions SHALL NOT appear.

13.20 Empty State

When no departments exist,
the module SHALL display:

No Departments Available

Create a department to begin managing civic operations.

A primary action SHALL navigate to the Create Department page.

13.21 Loading State

The module SHALL utilize:

- Skeleton Tables
- Skeleton Cards
- Skeleton Detail Views

Independent sections SHOULD load independently.

13.22 Error Handling

The module SHALL gracefully handle:

- Department not found
- Duplicate department code
- Network failure
- Permission denied
- Server error

User-friendly messages SHALL be displayed.

13.23 Performance Requirements

The Department Module SHALL support:

- Server-side pagination

- Server-side filtering
 - Efficient searching
 - Lazy-loaded analytics
 - Optimized API requests
-

13.24 Accessibility

The Department Module SHALL support:

- Keyboard navigation
 - Screen readers
 - Accessible tables
 - Accessible forms
 - ARIA-compliant controls
 - High-contrast themes
-

13.25 Security

The module SHALL enforce:

- Authentication
- Role-Based Access Control
- Secure API communication
- Protected routes

Department Administrators SHALL never access information belonging to other departments.

13.26 API Integration

The module SHALL integrate with backend APIs responsible for:

- Department Listing
- Department Details
- Department Creation
- Department Update
- Department Activation
- Department Analytics
- Department Statistics

- Department Reports

The frontend SHALL remain independent of backend implementation details.

13.27 Future Enhancements

The architecture SHALL support future additions including:

- Multi-City Department Mapping
- Department Budget Tracking
- Resource Allocation Dashboard
- GIS-based Department Coverage
- AI Performance Recommendations
- Capacity Planning

These enhancements SHALL integrate without requiring structural redesign.

13.28 Chapter Summary

The Department Management Module SHALL provide a centralized administrative interface for managing municipal departments within the CSCRS ecosystem.

It SHALL enable authorized administrators to:

- Create and manage departments.
- Monitor operational performance.
- Track reports and workers.
- Review departmental analytics.
- Maintain organizational structure.

The module SHALL support future expansion while maintaining a consistent enterprise user experience.

Chapter 14 — Department Administrator Management Module

14.1 Overview

The Department Administrator Management Module SHALL provide authorized users with the capability to create, manage, monitor, and administer Department Administrator accounts within the CSCRS ecosystem.

Department Administrators act as the operational heads of their assigned departments and are responsible for supervising workers, managing departmental reports, monitoring issue resolution, and coordinating daily operations.

The module SHALL integrate with the existing backend APIs responsible for Department Administrator management.

14.2 Objectives

The Department Administrator Management Module SHALL:

- Centralize Department Administrator account management.
 - Simplify onboarding.
 - Manage administrator lifecycle.
 - Monitor administrator activity.
 - Support secure invitation workflows.
 - Improve organizational transparency.
 - Maintain RBAC compliance.
-

14.3 Business Workflow

Create Department Administrator



Generate Invitation



Email Invitation Sent



Invitation Accepted



Password Setup



Account Activated



Department Assigned



Operational Activities



Deactivate / Reactivate (If Required)

14.4 RBAC

Capability	Department Admin	City Admin	Super Admin
View Department Administrators	✗	✓	✓
Create Department Administrator	✗	✓	✓
Edit Department Administrator	✗	✓	✓
Activate Account	✗	✓	✓
Deactivate Account	✗	✓	✓
Resend Invitation	✗	✓	✓
Reset Invitation	✗	✓	✓
Delete Account	✗	✗	✗ (Future Policy Based)

14.5 Module Structure

Department Administrators



└─ All Administrators

└─ Active Administrators

└─ Pending Invitations

|— Inactive Administrators

|— Administrator Details

|— Create Administrator

└— Edit Administrator

14.6 Module Entry Points

The module SHALL be accessible through:

- Sidebar Navigation
- Dashboard Quick Actions
- Dashboard Widgets
- Global Search

All entry points SHALL navigate to the Department Administrator Management module.

14.7 Administrator List

The Administrator List SHALL display all authorized Department Administrator accounts.

Columns SHALL include:

- Full Name
 - Employee ID (if applicable)
 - Email
 - Phone Number
 - Assigned Department
 - Invitation Status
 - Account Status
 - Created Date
 - Last Login
 - Actions
-

14.8 Search & Filters

Search SHALL support:

- Name
- Email
- Department
- Phone Number

Filters SHALL include:

- Department
- Account Status
- Invitation Status
- Date Created
- Last Login

Server-side filtering SHALL be preferred.

14.9 Create Department Administrator

The Create Administrator page SHALL contain:

- Full Name
- Email Address
- Phone Number
- Assigned Department
- Designation (Future)
- Submit Button

Mandatory fields SHALL be validated before submission.

14.10 Invitation Workflow

Upon successful creation,

the frontend SHALL display:

- Invitation Generated
- Email Sent Status
- Invitation Expiration (if provided by backend)

The frontend SHALL not generate invitations independently.

Invitation logic SHALL remain exclusively within backend services.

14.11 Pending Invitations

The Pending Invitations page SHALL display administrators who have not yet activated their accounts.

Information SHALL include:

- Name
- Email
- Department
- Invitation Date
- Expiration Date
- Current Status

Available actions MAY include:

- Resend Invitation
 - Cancel Invitation
 - Refresh Status
-

14.12 Administrator Details

Selecting an administrator SHALL open the Administrator Details workspace.

Tabs SHALL include:

Overview

Department

Activity

History

14.13 Overview Tab

The Overview SHALL display:

- Profile Photo (if available)

- Full Name
 - Contact Information
 - Assigned Department
 - Account Status
 - Invitation Status
 - Last Login
 - Created Date
-

14.14 Department Assignment

The Department tab SHALL display:

- Assigned Department
- Assignment Date
- Department Status

Authorized users MAY reassign administrators according to backend business rules.

14.15 Activity

The Activity tab SHALL summarize administrative operations.

Examples:

- Reports Managed
- Workers Managed
- Pending Reports
- Average Resolution Performance
- Recent Administrative Actions

All metrics SHALL be retrieved through backend APIs.

14.16 History

The History tab SHALL maintain an audit trail.

Typical events include:

- Account Created
- Invitation Sent
- Invitation Accepted

- Account Activated
- Department Changed
- Account Deactivated
- Profile Updated

Each event SHALL include timestamp information.

14.17 Account Status

Standardized status badges SHALL be used.

Supported statuses include:

- Pending Invitation
 - Active
 - Inactive
 - Suspended (Future)
-

14.18 Quick Actions

Authorized users MAY perform:

- View Details
- Edit Information
- Activate Account
- Deactivate Account
- Resend Invitation
- View Department

Actions SHALL only appear when permitted by RBAC.

14.19 Empty State

If no Department Administrators exist,
the module SHALL display:

No Department Administrators Found

Create a Department Administrator
to begin managing departmental operations.

A primary action button SHALL navigate to the Create Administrator page.

14.20 Loading State

The module SHALL use:

- Skeleton Tables
- Skeleton Profile Views
- Skeleton Detail Pages

Independent content sections SHOULD load asynchronously whenever practical.

14.21 Error Handling

The module SHALL gracefully handle:

- Duplicate email
- Invalid department
- Invitation expired
- Administrator not found
- Network failure
- Permission denied
- Server errors

User-friendly messages SHALL be displayed.

14.22 Performance Requirements

The module SHALL support:

- Server-side pagination
 - Efficient searching
 - Lazy-loaded detail sections
 - Optimized API utilization
-

14.23 Accessibility

The module SHALL comply with:

- Keyboard navigation
- Screen reader compatibility

- Accessible tables
 - Accessible forms
 - High-contrast themes
 - ARIA-compliant controls
-

14.24 Security

The module SHALL enforce:

- Authentication
- Role-Based Access Control
- Protected routes
- Secure API communication

Only authorized administrative roles SHALL access Department Administrator information.

14.25 API Integration

The module SHALL integrate with backend APIs responsible for:

- Department Administrator Listing
- Administrator Details
- Administrator Creation
- Invitation Management
- Account Activation
- Account Deactivation
- Department Assignment
- Activity Summary

Business rules SHALL remain implemented exclusively within backend services.

14.26 Future Enhancements

The architecture SHALL support:

- Multiple Department Assignments
- Temporary Acting Administrator
- Delegation During Leave
- Digital Signature

- Approval Workflow
- Workload Dashboard
- Administrator Performance Score

These enhancements SHALL integrate without redesigning the existing module.

14.27 Chapter Summary

The Department Administrator Management Module SHALL provide a centralized administrative interface for managing departmental leadership across the CSCRS ecosystem.

It SHALL enable authorized administrators to:

- Create and invite Department Administrators.
- Monitor invitation status.
- Manage department assignments.
- Review activity history.
- Securely administer administrator accounts.

The module SHALL remain fully integrated with the existing backend architecture while supporting future organizational growth.

Chapter 15 — City Administrator Management Module

15.1 Overview

The City Administrator Management Module SHALL provide the Super Administrator with centralized control over City Administrator accounts operating within the CSCRS ecosystem.

City Administrators serve as the highest operational authority for a city and are responsible for supervising departments, monitoring city-wide civic issue resolution, reviewing analytics, and ensuring efficient municipal operations.

This module SHALL enable the complete lifecycle management of City Administrator accounts while maintaining strict Role-Based Access Control (RBAC).

The frontend SHALL integrate with the existing backend APIs without duplicating business logic.

15.2 Objectives

The City Administrator Management Module SHALL:

- Centralize City Administrator management.
- Simplify administrator onboarding.
- Monitor city-level administrative operations.

- Support invitation-based account activation.
- Maintain secure administrative hierarchy.
- Improve governance and accountability.
- Support future multi-city deployments.

15.3 Business Workflow

Create City Administrator



Generate Invitation



Invitation Email Sent



Invitation Accepted



Password Setup



Account Activated



City Assignment



City-wide Operations



Deactivate / Reactivate (If Required)

15.4 RBAC

Capability	Department Admin	City Admin	Super Admin
View City Administrators	✗	✗	✓
Create City Administrator	✗	✗	✓
Edit City Administrator	✗	✗	✓
Activate Account	✗	✗	✓
Deactivate Account	✗	✗	✓
Resend Invitation	✗	✗	✓
Assign City	✗	✗	✓
Delete Account	✗	✗	✗ (Future Policy Based)

The entire module SHALL remain accessible exclusively to Super Administrators.

15.5 Module Structure

City Administrators

|

|— All Administrators

|— Active Administrators

|— Pending Invitations

|— Inactive Administrators

|— Administrator Details

|— Create Administrator

└ Edit Administrator

15.6 Module Entry Points

The module SHALL be accessible through:

- Sidebar Navigation
- Dashboard Quick Actions
- Dashboard KPI Widgets
- Global Search

The module SHALL NOT appear for unauthorized roles.

15.7 City Administrator List

The City Administrator List SHALL display all City Administrator accounts.

Columns SHALL include:

- Full Name
 - Email Address
 - Phone Number
 - Assigned City
 - Number of Departments
 - Invitation Status
 - Account Status
 - Last Login
 - Created Date
 - Actions
-

15.8 Search & Filters

Search SHALL support:

- Administrator Name
- Email
- Phone Number
- Assigned City

Filters SHALL include:

- City
- Account Status
- Invitation Status
- Created Date
- Last Login

Filtering SHALL be performed server-side.

15.9 Create City Administrator

The Create City Administrator page SHALL contain:

- Full Name
- Email Address
- Phone Number
- Assigned City
- Initial Status

The frontend SHALL validate all mandatory fields before submission.

15.10 Invitation Workflow

The invitation workflow SHALL remain identical to the backend implementation.

The frontend SHALL display:

- Invitation Sent
- Invitation Pending
- Invitation Accepted
- Invitation Expired (if applicable)

Available actions MAY include:

- Resend Invitation
 - Cancel Invitation
 - Refresh Status
-

15.11 City Assignment

Each City Administrator SHALL be associated with one administrative jurisdiction.

The module SHALL display:

- Assigned City
- Assignment Date
- Current Status
- Number of Departments
- Number of Department Administrators
- Number of Workers (Summary)
- Number of Active Reports (Summary)

Future versions MAY support multiple jurisdictions if backend architecture evolves.

15.12 Administrator Details

Selecting a City Administrator SHALL open the Administrator Details workspace.

Tabs SHALL include:

Overview

City

Departments

Analytics

Activity

History

15.13 Overview Tab

The Overview SHALL display:

- Profile Photo (if available)
- Full Name
- Contact Information
- Assigned City

- Account Status
 - Invitation Status
 - Last Login
 - Created Date
-

15.14 City Tab

The City tab SHALL summarize operational responsibility.

Information SHALL include:

- City Name
- Total Departments
- Active Departments
- Department Administrators
- Total Workers
- Active Reports
- Resolved Reports
- Pending Reports

This section SHALL provide a high-level operational overview.

15.15 Departments Tab

The Departments tab SHALL display all departments under the assigned city.

Each department SHALL include:

- Department Name
- Department Administrator
- Active Workers
- Pending Reports
- Current Performance

Each department SHALL link directly to the Department Management module.

15.16 Analytics Tab

The Analytics tab SHALL display city-level operational insights.

Typical widgets SHALL include:

- Total Reports
- Resolution Rate
- Monthly Trends
- Department Comparison
- Report Category Distribution
- Average Resolution Time

Charts SHALL remain responsive and interactive.

15.17 Activity Tab

The Activity tab SHALL summarize administrative operations.

Examples include:

- Department Created
 - Department Administrator Assigned
 - Reports Reviewed
 - Administrative Actions
 - Configuration Updates
-

15.18 History Tab

The History tab SHALL maintain an administrative audit trail.

Typical events SHALL include:

- Account Created
- Invitation Sent
- Invitation Accepted
- Account Activated
- City Assignment Updated
- Account Deactivated

Every event SHALL include timestamps and responsible users where available.

15.19 Account Status

Standardized status badges SHALL be used.

Supported statuses include:

- Pending Invitation
- Active
- Inactive
- Suspended (Future)

Status colors SHALL comply with the Design System defined in Chapter 8.

15.20 Quick Actions

Authorized users MAY perform:

- View Details
- Edit Administrator
- Change City Assignment
- Activate Account
- Deactivate Account
- Resend Invitation
- View Analytics

Only valid actions SHALL be displayed.

15.21 Organization Hierarchy

The module SHALL visually represent the administrative hierarchy.

Super Administrator

|



City Administrator

|



Department Administrator

|



Workers

|



Citizen Reports

This visualization SHALL assist in understanding administrative responsibility.

15.22 Empty State

If no City Administrators exist,
the module SHALL display:

No City Administrators Found

Create a City Administrator

to begin city-wide administration.

A primary action SHALL navigate directly to the Create Administrator page.

15.23 Loading State

The module SHALL use:

- Skeleton Tables
- Skeleton Cards
- Skeleton Detail Pages
- Skeleton Charts

Individual dashboard sections SHOULD load independently.

15.24 Error Handling

The module SHALL gracefully handle:

- Duplicate Email
- Invalid City Assignment
- Invitation Expired
- Administrator Not Found
- Network Failure
- Permission Denied
- Server Error

User-facing messages SHALL remain concise and informative.

15.25 Performance Requirements

The module SHALL support:

- Server-side Pagination
 - Server-side Filtering
 - Lazy-loaded Analytics
 - Efficient API Requests
 - Responsive Data Rendering
-

15.26 Accessibility

The module SHALL support:

- Keyboard Navigation
 - Screen Reader Compatibility
 - Accessible Tables
 - Accessible Charts
 - Accessible Forms
 - ARIA-compliant Controls
 - High Contrast Mode
-

15.27 Security

The module SHALL enforce:

- Authentication
- RBAC
- Protected Routes
- Secure API Communication

City Administrator information SHALL only be accessible to Super Administrators.

15.28 API Integration

The module SHALL integrate with backend APIs responsible for:

- City Administrator Listing
- Details
- Creation

- Update
- Invitation Management
- Account Activation
- City Assignment
- Activity Summary
- Analytics

The frontend SHALL not implement business logic already present within backend services.

15.29 Future Enhancements

The architecture SHALL support:

- Multi-City Assignment
- Regional Administration
- Administrative Delegation
- Organization Chart Visualization
- Approval Workflows
- Administrative Performance Dashboards
- Regional Analytics
- Smart Governance Recommendations

These enhancements SHALL integrate without requiring architectural redesign.

15.30 Chapter Summary

The City Administrator Management Module SHALL provide centralized administration of city-level leadership across the CSCRS ecosystem.

The module SHALL enable Super Administrators to:

- Create and invite City Administrators.
- Assign cities.
- Monitor city-wide operational responsibilities.
- Review administrative analytics.
- Manage administrator lifecycle.
- Maintain organizational governance.

The module SHALL remain scalable for future expansion into multi-city and regional deployments.

Chapter 16 — Enterprise Notification Center

16.1 Overview

The Enterprise Notification Center SHALL function as the centralized communication hub of the CSCRS Administrative Portal.

The Notification Center SHALL aggregate operational events, administrative alerts, workflow updates, and system messages into a unified interface.

Rather than functioning as a simple notification list, the Notification Center SHALL provide contextual navigation, notification management, and actionable insights.

16.2 Objectives

The Notification Center SHALL:

- Centralize administrative communication.
 - Reduce response time.
 - Improve operational awareness.
 - Eliminate missed administrative events.
 - Support actionable notifications.
 - Maintain notification history.
 - Support future real-time communication.
-

16.3 Notification Sources

Notifications SHALL originate from backend business events.

Examples include:

- New Report Submitted
- High Priority Report Created
- Manual Review Required
- Worker Assigned
- Assignment Updated
- Resolution Submitted
- Resolution Verified
- Department Created
- Worker Invitation Accepted

- Administrator Invitation Accepted
- Account Activated
- System Announcement
- Maintenance Notification
- Security Alert (Future)

The frontend SHALL never generate business notifications independently.

16.4 Notification Entry Points

Users SHALL access the Notification Center through:

- Top Navigation Bell Icon
- Dashboard Widget
- Sidebar Navigation
- Deep Link from Dashboard

All entry points SHALL navigate to the same Notification Center.

16.5 Module Structure

Notifications

|

|— All Notifications

|— Unread

|— High Priority

|— Administrative

|— System

|— Archive (Future)

16.6 Notification Bell

The Notification Bell SHALL remain visible in the Top Navigation Bar.

The bell SHALL display:

- Notification Icon
- Unread Count Badge

Example:



The unread badge SHALL automatically update when backend data changes.

16.7 Notification Panel

Clicking the Notification Bell SHALL open a dropdown panel.

The panel SHALL display:

- Latest Notifications
- Timestamp
- Priority Indicator
- Read / Unread Status
- View All Notifications

The panel SHALL NOT require page navigation.

16.8 Notification List

The Notification Center SHALL display notifications using an enterprise list layout.

Each notification SHALL include:

- Icon
- Title
- Short Description
- Timestamp
- Category
- Priority

- Read Status

Unread notifications SHALL be visually distinguished.

16.9 Notification Categories

Notifications SHALL be categorized.

Examples include:

- Reports
- Assignments
- Departments
- Workers
- Administrators
- Analytics
- System
- Security
- Announcements

Categories SHALL support filtering.

16.10 Priority Levels

Notifications SHALL indicate priority.

Supported priorities:

- Low
- Normal
- High
- Critical

Critical notifications SHALL receive additional visual emphasis.

16.11 Read Status

Every notification SHALL maintain one of the following states:

- Unread
- Read

Users SHALL be able to:

- Mark as Read
- Mark All as Read

Future versions MAY support "Unread Again."

16.12 Notification Details

Selecting a notification SHALL display detailed information.

Information MAY include:

- Full Description
- Related Entity
- Creation Date
- Responsible User
- Current Status

The detail view SHALL preserve navigation context.

16.13 Deep Linking

Notifications SHALL support direct navigation.

Examples:

New Report

↓

Report Details

Worker Assigned

↓

Worker Details

Department Updated

↓

Department Details

Every business notification SHALL navigate directly to the associated module whenever applicable.

16.14 Search & Filters

The Notification Center SHALL support:

Search by:

- Title
- Description
- Related Entity

Filters:

- Category
- Priority
- Read Status
- Date Range

Search SHALL be server-side where practical.

16.15 Bulk Operations

Users MAY perform:

- Mark Selected as Read
- Mark All as Read

Bulk deletion SHALL NOT be supported during the initial implementation unless explicitly supported by backend APIs.

16.16 Notification Retention

The frontend SHALL display notifications according to backend retention policies.

Notification deletion SHALL remain governed by backend business rules.

16.17 Dashboard Integration

Dashboard widgets MAY display:

- Recent Notifications
- High Priority Alerts

- Pending Administrative Actions

Each widget SHALL provide direct navigation to the Notification Center.

16.18 Real-Time Updates (Future Enhancement)

Future implementations MAY support:

- WebSocket Notifications
- Server-Sent Events (SSE)
- Push Notifications

The Notification Center architecture SHALL accommodate real-time delivery without redesign.

16.19 Email Integration (Future)

Future versions MAY synchronize notifications with email.

Examples:

- Critical Alerts
- Invitation Emails
- Security Warnings
- Maintenance Announcements

Email preferences SHALL remain configurable.

16.20 Notification Preferences (Future)

Users MAY configure:

- Email Notifications
- Push Notifications
- Notification Categories
- Quiet Hours
- Priority Alerts

Preferences SHALL be managed through the Profile & Settings module.

16.21 Empty State

When no notifications exist,

the Notification Center SHALL display:

No Notifications

You're all caught up.

The interface SHALL communicate a completed state rather than appearing empty.

16.22 Loading State

The module SHALL use:

- Skeleton Notification List
- Skeleton Detail View

Notification loading SHALL remain independent of other dashboard components.

16.23 Error Handling

The module SHALL gracefully handle:

- Notification not found
- Network failure
- Server error
- Permission denied

Friendly error messages SHALL be displayed.

16.24 Performance Requirements

The Notification Center SHALL support:

- Server-side pagination
- Lazy loading
- Efficient polling (where applicable)
- Optimized API requests

Large notification histories SHALL remain performant.

16.25 Accessibility

The module SHALL support:

- Keyboard navigation
- Screen readers

- ARIA-compliant controls
 - Accessible notification badges
 - High-contrast compatibility
-

16.26 Security

The Notification Center SHALL enforce:

- Authentication
- RBAC
- Secure API communication

Users SHALL receive only notifications relevant to their authorized scope.

16.27 API Integration

The module SHALL integrate with backend APIs responsible for:

- Notification Listing
- Notification Details
- Mark as Read
- Mark All as Read
- Notification Counts
- Notification Categories

Business event generation SHALL remain exclusively handled by backend services.

16.28 Future Enhancements

The architecture SHALL support:

- WebSocket Live Updates
- Push Notifications
- Email Synchronization
- Notification Snoozing
- AI-Prioritized Notifications
- Smart Notification Grouping
- Mention System
- Administrative Broadcasts

16.29 Chapter Summary

The Enterprise Notification Center SHALL provide a centralized, role-aware communication hub for the CSCRS Administrative Portal.

It SHALL enable administrators to:

- Monitor operational events.
- Track administrative activities.
- Navigate directly to affected modules.
- Manage notification status.
- Prepare for future real-time communication.

The Notification Center SHALL improve responsiveness while maintaining a consistent enterprise user experience.

Chapter 17 — Analytics, Insights & Business Intelligence Module

17.1 Overview

The Analytics, Insights & Business Intelligence Module SHALL provide authorized administrators with a centralized analytical workspace for monitoring operational performance, evaluating municipal efficiency, identifying trends, and supporting data-driven decision making.

Unlike operational modules that focus on individual reports or administrative tasks, this module SHALL present aggregated information through Key Performance Indicators (KPIs), dashboards, analytical visualizations, comparative reports, and executive summaries.

The module SHALL transform operational data into actionable insights for administrative decision-making.

17.2 Objectives

The Analytics Module SHALL:

- Provide city-wide operational visibility.
- Support data-driven governance.
- Measure departmental performance.
- Evaluate workforce efficiency.
- Monitor report resolution trends.
- Present AI-generated operational insights.
- Generate executive-level reports.

- Support future predictive analytics.

17.3 Business Workflow

Operational Data



Backend Analytics APIs



Aggregation



Visualization



Administrative Insights



Executive Decisions

17.4 RBAC

Capability	Department Admin	City Admin	Super Admin
Department Analytics	✓ (Own Department Only)	✓	✓
City Analytics	✗	✓	✓
Cross-Department Comparison	✗	✓	✓
System Analytics	✗	✗	✓
Export Reports	✓ (Own Scope)	✓	✓

Analytics SHALL always respect backend authorization.

17.5 Module Structure

Analytics

|

|— Dashboard

|— Reports

|— Departments

|— Workers

|— AI Insights

|— Executive Reports

└ Exports

17.6 KPI Dashboard

The Analytics Dashboard SHALL display high-level KPIs.

Examples include:

- Total Reports
- Pending Reports
- Active Reports
- Resolved Reports
- Resolution Rate
- Average Resolution Time
- Worker Utilization
- Department Performance

Each KPI SHALL support drill-down navigation.

17.7 Operational Analytics

Operational Analytics SHALL provide insight into daily municipal activities.

Visualizations MAY include:

- Daily Report Volume
- Weekly Report Trends
- Monthly Resolution Trends
- Resolution Time Distribution
- Report Category Distribution

Charts SHALL support responsive resizing.

17.8 Department Analytics

Department Analytics SHALL provide comparative performance metrics.

Examples:

- Active Departments
- Pending Workload
- Average Resolution Time
- Department Ranking
- Resolution Success Rate
- Monthly Department Performance

Department comparison SHALL be available only to authorized roles.

17.9 Worker Analytics

Worker Analytics SHALL summarize workforce performance.

Examples:

- Active Workers
- Available Workers
- Busy Workers
- Average Assignment Completion
- Worker Productivity
- Monthly Performance

Department Administrators SHALL only view analytics for workers within their own department.

17.10 AI Insights

The AI Insights section SHALL present analytics generated from backend AI services.

Examples include:

- AI Detection Accuracy
- Duplicate Detection Rate
- Manual Review Percentage
- Image Verification Results
- Risk Score Distribution
- Detection Category Distribution

The frontend SHALL only visualize backend-provided metrics.

17.11 Resolution Analytics

Resolution Analytics SHALL measure operational effectiveness.

Examples include:

- Average Resolution Time
- Resolution Success Rate
- Delayed Reports
- High Priority Resolution Time
- Department Resolution Comparison

Charts SHALL support configurable time ranges.

17.12 Geographic Analytics (Future)

The architecture SHALL support geographic visualizations.

Future widgets MAY include:

- Heat Maps
- Ward-wise Distribution
- Zone Performance
- Area-wise Report Density
- GIS Integration

The current layout SHALL reserve space for these future capabilities.

17.13 Executive Dashboard

The Executive Dashboard SHALL provide summarized information suitable for senior administrative officials.

Typical sections SHALL include:

- Executive KPIs
- Department Comparison
- Monthly Summary
- Operational Health
- Performance Highlights
- Critical Issues

The Executive Dashboard SHALL minimize operational detail while maximizing decision-support information.

17.14 Report Builder (Future)

Future versions MAY support customizable analytical reports.

Administrators MAY select:

- Date Range
- Department
- Issue Category
- Report Type
- Output Format

The initial implementation SHALL use predefined reports.

17.15 Export Center

Authorized users SHALL export analytical data in supported formats.

Supported formats MAY include:

- PDF
- Excel
- CSV

Export permissions SHALL follow backend authorization.

17.16 Time Range Filters

Analytics SHALL support predefined ranges.

Examples:

- Today
- Last 7 Days
- Last 30 Days
- Current Month
- Previous Month
- Current Year
- Custom Range

Changing the time range SHALL refresh all relevant widgets.

17.17 Drill-Down Navigation

Charts and KPI cards SHALL support drill-down navigation.

Example:

Pending Reports

↓

Click

↓

Pending Report List

Example:

Road Damage

↓

Click

↓

Filtered Reports

Users SHALL transition directly from analytics to operational modules.

17.18 Dashboard Personalization (Future)

Future versions MAY allow users to:

- Rearrange charts.
- Hide widgets.
- Save layouts.
- Pin favorite KPIs.

The initial implementation SHALL provide a standardized layout.

17.19 Empty State

When analytical data is unavailable,
the module SHALL display:

No Analytics Available

Data will appear as reports are processed.

The interface SHALL remain informative.

17.20 Loading State

The module SHALL utilize:

- Skeleton KPI Cards
- Skeleton Charts
- Skeleton Tables

Individual widgets SHOULD load independently.

17.21 Error Handling

The Analytics Module SHALL gracefully handle:

- Missing datasets

- Network failures
- Unauthorized access
- Invalid filters
- Export failures
- Server errors

Charts SHALL display localized error messages without affecting unrelated widgets.

17.22 Performance Requirements

The module SHALL support:

- Lazy-loaded charts
- Cached datasets where appropriate
- Efficient API utilization
- Progressive loading
- Responsive rendering

Large analytical datasets SHALL remain performant.

17.23 Accessibility

Analytics SHALL support:

- Keyboard navigation
- Accessible charts
- Screen readers
- High-contrast themes
- Descriptive chart labels
- ARIA-compliant controls

Alternative textual summaries SHOULD accompany graphical visualizations wherever practical.

17.24 Security

The Analytics Module SHALL enforce:

- Authentication
- RBAC
- Protected routes

- Secure API communication

Users SHALL only access analytics corresponding to their authorized scope.

17.25 API Integration

The module SHALL integrate with backend APIs responsible for:

- Dashboard Analytics
- Department Statistics
- Worker Statistics
- Resolution Metrics
- AI Analytics
- Export Operations

Business calculations SHALL remain exclusively implemented within backend services.

17.26 Future Enhancements

The architecture SHALL support:

- Predictive Analytics
- AI Recommendations
- Trend Forecasting
- Heat Maps
- GIS Integration
- Executive Email Reports
- Scheduled Report Generation
- Comparative Benchmarking
- Machine Learning Insights

These enhancements SHALL integrate without redesigning the module.

17.27 Chapter Summary

The Analytics, Insights & Business Intelligence Module SHALL transform operational data into meaningful administrative intelligence.

The module SHALL provide:

- Executive dashboards.

- Department performance analysis.
- Worker productivity insights.
- AI-generated metrics.
- Resolution analytics.
- Exportable reports.
- Future-ready predictive capabilities.

The Analytics Module SHALL function as the primary decision-support system for municipal administrators.

Chapter 18 — User Profile & Account Management Module

18.1 Overview

The User Profile & Account Management Module SHALL provide authenticated administrators with a centralized interface for viewing and managing their personal account information.

This module SHALL allow users to maintain their profile details, review account information, change passwords, and manage personal preferences while respecting Role-Based Access Control (RBAC).

The module SHALL focus exclusively on user-specific account management and SHALL NOT include system-wide configuration options.

18.2 Objectives

The User Profile Module SHALL:

- Centralize account information.
- Allow secure profile updates.
- Support password management.
- Improve account transparency.
- Maintain consistent user identity.
- Prepare the platform for future account-related enhancements.

18.3 Module Entry Points

The module SHALL be accessible through:

- Top Navigation Profile Avatar
- User Dropdown Menu
- Dashboard Shortcut (Future)

- Global Search (Future)

Selecting **My Profile** SHALL navigate to the User Profile workspace.

18.4 RBAC

All authenticated administrative roles SHALL have access to their own profile.

Capability	Department Admin	City Admin	Super Admin
View Own Profile	✓	✓	✓
Edit Own Profile	✓	✓	✓
Change Password	✓	✓	✓
View Login History (Future)	✓	✓	✓
Manage Sessions (Future)	✓	✓	✓

Users SHALL NOT access another user's profile through this module.

18.5 Module Structure

My Profile

|

|— Overview

|— Personal Information

|— Security

|— Activity

|— Preferences (Future)

|— Sessions (Future)

18.6 Overview

The Overview page SHALL present a summary of the authenticated user's account.

Information SHALL include:

- Profile Picture
- Full Name
- Role
- Assigned Organization
- Department / City (as applicable)
- Email Address
- Phone Number
- Account Status
- Last Login
- Account Creation Date

The Overview SHALL provide quick access to frequently used account actions.

18.7 Personal Information

The Personal Information section SHALL allow users to view and update editable profile information.

Editable fields MAY include:

- Full Name
- Phone Number
- Profile Picture (Future)

Read-only fields SHALL include:

- Email Address
- Employee ID (if applicable)
- Assigned Role
- Assigned Department / City
- Account Creation Date

Field editability SHALL follow backend policies.

18.8 Change Password

The Change Password page SHALL integrate with the existing backend password management APIs.

The form SHALL contain:

- Current Password
- New Password
- Confirm Password

The frontend SHALL perform:

- Required field validation
- Password confirmation validation
- Basic password strength validation

The backend SHALL remain the source of truth for password policies.

18.9 Password Requirements

The frontend SHALL communicate password requirements before submission.

Examples MAY include:

- Minimum length
- Uppercase letter
- Lowercase letter
- Numeric character
- Special character

Validation messages SHALL be clear and user-friendly.

18.10 Activity

The Activity section SHALL summarize recent account-related events.

Examples include:

- Password Changed
- Profile Updated
- Login Successful
- Failed Login Attempt (Future)
- Invitation Accepted
- Account Activated

Activity information SHALL be retrieved from backend APIs where available.

18.11 Preferences (Future)

Future implementations MAY allow users to configure:

- Preferred Language
- Time Zone
- Date Format
- Dashboard Preferences
- Notification Preferences Shortcut
- Accessibility Preferences

The current release SHALL reserve space for these enhancements.

18.12 Active Sessions (Future)

Future versions MAY display:

- Current Session
- Browser Information
- Device Name
- Login Location
- Login Time

Users MAY be allowed to terminate active sessions according to backend capabilities.

18.13 Profile Picture

The architecture SHALL support future profile image uploads.

Supported capabilities MAY include:

- Upload
- Replace
- Remove
- Crop
- Preview

Image validation SHALL follow backend file validation rules.

18.14 Quick Actions

The Profile module SHALL provide quick access to:

- Edit Profile
- Change Password
- View Activity
- Logout

Future actions MAY include session management and security settings.

18.15 Empty State

If profile information is unavailable,
the module SHALL display:

Profile Information Unavailable

Please contact your system administrator if the issue persists.

18.16 Loading State

The module SHALL utilize:

- Skeleton Profile Cards
- Skeleton Forms
- Skeleton Activity Timeline

Individual sections SHOULD load independently.

18.17 Error Handling

The module SHALL gracefully handle:

- Invalid password
- Password mismatch
- Profile update failure
- Network failure
- Permission denied
- Server error

Validation messages SHALL remain clear and actionable.

18.18 Performance Requirements

The module SHALL support:

- Efficient API utilization
- Lazy loading of secondary tabs
- Optimized profile retrieval
- Responsive rendering

18.19 Accessibility

The module SHALL support:

- Keyboard navigation
- Accessible forms
- Screen reader compatibility
- ARIA-compliant controls
- High-contrast themes

18.20 Security

The User Profile Module SHALL enforce:

- Authentication
- RBAC
- Secure API communication

Sensitive account information SHALL never be exposed beyond the authenticated user's authorized scope.

Passwords SHALL never be displayed or stored by the frontend.

18.21 API Integration

The module SHALL integrate with backend APIs responsible for:

- Profile Retrieval
- Profile Update
- Change Password
- User Information

- Activity Summary

Business validation SHALL remain implemented within backend services.

18.22 Future Enhancements

The architecture SHALL support:

- Multi-factor Authentication (2FA)
- Login History
- Active Session Management
- Device Trust
- Security Notifications
- Digital Identity Verification
- Profile Completion Score

These enhancements SHALL integrate without redesigning the existing module.

18.23 Chapter Summary

The User Profile & Account Management Module SHALL provide authenticated administrators with a secure and centralized interface for managing their personal account information.

The module SHALL enable users to:

- View and update profile information.
- Change passwords securely.
- Review account activity.
- Prepare for future security and personalization enhancements.

The module SHALL maintain a consistent, secure, and user-centric experience while integrating seamlessly with the existing backend authentication and account management services.

Chapter 19 — System Settings & Configuration Module

19.1 Overview

The System Settings & Configuration Module SHALL provide authorized users with a centralized interface for managing configurable aspects of the CSCRS Administrative Portal.

The module SHALL separate **personal preferences** from **system-wide configuration**, ensuring that only authorized administrative roles can modify operational settings.

The module SHALL integrate with existing backend configuration APIs and SHALL remain extensible for future platform capabilities.

19.2 Objectives

The System Settings Module SHALL:

- Centralize platform configuration.
 - Support role-based settings management.
 - Maintain secure configuration updates.
 - Improve administrative efficiency.
 - Enable future scalability.
 - Preserve configuration consistency across the platform.
-

19.3 RBAC

Capability	Department Admin	City Admin	Super Admin
View Personal Settings	✓	✓	✓
Change Personal Preferences	✓	✓	✓
View Department Settings	✓ (Own Department Only)	✓	✓
Modify Department Settings	✓ (Own Department Only)	✓	✓
View System Configuration	✗	View Limited	✓
Modify System Configuration	✗	✗	✓

Every configuration SHALL respect backend authorization.

19.4 Module Structure

Settings

|

|— General

|— Notifications

└— Language

└— Appearance

└— Department Settings

└— Security

└— System Configuration

└— Integrations (Future)

└— Audit Logs (Future)

19.5 General Settings

The General Settings section SHALL include configurable preferences such as:

- Display Language
- Date Format
- Time Format
- Time Zone
- Number Format

These settings SHALL apply to the authenticated user's portal experience unless defined as organization-wide settings.

19.6 Notification Settings

Users SHALL be able to configure personal notification preferences.

Supported options MAY include:

- In-App Notifications
- Email Notifications (Future)
- Critical Alerts

- Assignment Notifications
- Resolution Updates
- Administrative Announcements

Notification preferences SHALL NOT affect mandatory security notifications.

19.7 Language Settings

The portal SHALL support multilingual capability.

Initial implementation SHALL include:

- English
- Hindi

The architecture SHALL support additional languages without redesign.

Changing language SHALL update the interface dynamically where technically feasible.

19.8 Appearance

The Appearance section SHALL define user interface preferences.

Initial implementation SHALL include:

- System Theme
- Light Theme

Future enhancements MAY include:

- Dark Theme
- High Contrast Theme
- Municipal Branding Themes

Appearance changes SHALL not affect business functionality.

19.9 Department Settings

Department-level configuration SHALL be available according to RBAC.

Examples MAY include:

- Department Display Name
- Contact Information
- Working Hours (Future)
- Escalation Preferences (Future)

Department Administrators SHALL only manage settings within their assigned department.

19.10 Security Settings

The Security section SHALL provide access to account-related security options.

Examples include:

- Change Password (Shortcut)
- Password Policy Information
- Multi-Factor Authentication Status (Future)
- Active Sessions (Future)
- Trusted Devices (Future)

Sensitive operations SHALL require backend verification.

19.11 System Configuration

The System Configuration section SHALL be available only to Super Administrators.

Examples MAY include:

- Platform Information
- Application Version
- Maintenance Mode (Future)
- Default Configuration Values
- Feature Flags (Future)
- Global Platform Preferences

The frontend SHALL not expose unauthorized configuration options.

19.12 Integration Settings (Future)

Future implementations MAY support configuration of:

- Email Service
- SMS Gateway
- Push Notification Service
- GIS Services
- Government APIs
- AI Model Providers

The architecture SHALL remain extensible.

19.13 Audit Log Access (Future)

Authorized administrators MAY access configuration-related audit logs.

Typical entries SHALL include:

- Configuration Changed
- Security Setting Updated
- Language Updated
- System Preference Modified

Audit logs SHALL remain read-only.

19.14 Save & Reset Behavior

Configuration pages SHALL provide:

- Save Changes
- Cancel
- Reset to Default (where applicable)

Users SHALL receive confirmation before resetting configuration values.

19.15 Validation

The frontend SHALL validate:

- Required fields
- Supported values
- Input formats

Business validation SHALL remain implemented by backend services.

19.16 Empty State

If a settings category contains no configurable items,
the module SHALL display:

No Configurable Settings Available

This section currently has no configurable options.

19.17 Loading State

The module SHALL utilize:

- Skeleton Forms
- Skeleton Settings Cards

Configuration categories SHOULD load independently.

19.18 Error Handling

The module SHALL gracefully handle:

- Configuration update failure
- Permission denied
- Invalid values
- Network failure
- Server errors

The previous configuration SHALL remain visible until a successful update is confirmed.

19.19 Performance Requirements

The Settings Module SHALL support:

- Efficient API requests
 - Lazy loading of categories
 - Responsive rendering
 - Minimal unnecessary refreshes
-

19.20 Accessibility

The module SHALL support:

- Keyboard navigation
 - Screen readers
 - Accessible forms
 - High Contrast Mode
 - ARIA-compliant controls
-

19.21 Security

The Settings Module SHALL enforce:

- Authentication
- RBAC
- Secure API communication

Only authorized users SHALL modify configuration values.

Configuration changes SHALL follow backend authorization policies.

19.22 API Integration

The module SHALL integrate with backend APIs responsible for:

- User Preferences
- System Settings
- Department Settings
- Notification Preferences
- Configuration Updates

The frontend SHALL not implement configuration business rules independently.

19.23 Future Enhancements

The architecture SHALL support:

- Dark Mode
- White Label Branding
- Municipal Themes
- AI Configuration
- Regional Settings
- Feature Toggles
- Plugin Configuration
- Multi-Tenant Configuration

These enhancements SHALL integrate without architectural redesign.

19.24 Chapter Summary

The System Settings & Configuration Module SHALL provide a centralized interface for managing platform and user configuration while maintaining strict role-based access control.

The module SHALL enable:

- Personal preferences.
- Department-level configuration.
- System-wide settings.
- Notification preferences.
- Future integrations.
- Secure configuration management.

The architecture SHALL remain scalable and compatible with future platform enhancements.

Chapter 20 — Platform Administration Module

20.1 Overview

The Platform Administration Module SHALL provide the Super Administrator with centralized control over the CSCRS platform.

This module SHALL serve as the operational headquarters for platform-wide administration, governance, monitoring, configuration, auditing, and future system expansion.

Only authenticated users possessing the Super Administrator role SHALL be permitted to access this module.

The frontend SHALL integrate with the existing backend administrative APIs while maintaining strict Role-Based Access Control (RBAC).

20.2 Objectives

The Platform Administration Module SHALL:

- Centralize platform governance.
 - Provide complete system visibility.
 - Monitor platform health.
 - Manage administrative hierarchy.
 - Support AI dataset administration.
 - Maintain auditability.
 - Prepare the platform for future large-scale deployments.
-

20.3 RBAC

Capability	Department Admin	City Admin	Super Admin
View Platform Dashboard	✗	✗	✓
View System Health	✗	✗	✓
View Audit Logs	✗	✗	✓
Manage Platform Configuration	✗	✗	✓
AI Dataset Export	✗	✗	✓
Platform Statistics	✗	✗	✓
Maintenance Mode	✗	✗	✓ (Future)

This module SHALL NOT be visible to unauthorized users.

20.4 Module Structure

Platform Administration

|

|— Platform Dashboard

|— Organization

|— AI Dataset

|— Audit Center

|— Platform Statistics

|— System Health

|— Configuration

└─ Maintenance (Future)

└─ Feature Management (Future)

20.5 Platform Dashboard

The Platform Dashboard SHALL present a complete overview of the CSCRS ecosystem.

Examples include:

- Total Cities
- Total Departments
- Total Department Administrators
- Total Workers
- Total Reports
- Pending Reports
- Resolved Reports
- Platform Uptime (Future)

Every widget SHALL support drill-down navigation where applicable.

20.6 Organization Overview

The Organization section SHALL summarize the administrative hierarchy.

Examples:

- Cities
- Departments
- Department Administrators
- Workers

The frontend SHALL provide navigation to the corresponding management modules.

20.7 AI Dataset Management

The AI Dataset section SHALL provide access to AI-related administrative capabilities supported by the backend.

Examples MAY include:

- Export Approved Dataset

- Export Training Dataset
- Dataset Statistics
- Dataset Version Information
- Annotation Progress (Future)

The frontend SHALL not perform dataset processing independently.

20.8 Audit Center

The Audit Center SHALL display platform-wide administrative events.

Examples include:

- User Created
- Administrator Activated
- Configuration Changed
- Report Status Updated
- Department Created
- Worker Created
- Login Events (Future)
- Security Events (Future)

Audit information SHALL be read-only.

20.9 Platform Statistics

Platform-wide statistics SHALL include:

- Reports by Category
- Reports by Department
- Reports by City
- Resolution Rate
- Worker Productivity
- Department Performance
- AI Verification Statistics

Charts SHALL support drill-down where appropriate.

20.10 System Health

The System Health section SHALL summarize platform operational status.

Examples MAY include:

- Backend Service Status
- Database Status
- Storage Usage
- API Availability (Future)
- Background Jobs (Future)
- AI Service Status (Future)

The frontend SHALL display backend-provided health information without independently performing infrastructure monitoring.

20.11 Configuration

The Configuration section SHALL provide centralized access to platform-wide configuration.

Examples include:

- Platform Information
- Global Defaults
- Supported Languages
- Notification Defaults
- AI Configuration (Future)
- Platform Branding (Future)

Only backend-supported configuration options SHALL be exposed.

20.12 Dataset Export

Authorized users SHALL be able to export AI datasets.

Supported export formats SHALL depend upon backend implementation.

The frontend SHALL provide:

- Export Status
 - Progress Indicator
 - Completion Notification
 - Download Link (when available)
-

20.13 Administrative Search

Platform Administration SHALL support global administrative search.

Search MAY include:

- Cities
- Departments
- Administrators
- Workers
- Reports

Search SHALL respect RBAC and backend authorization.

20.14 Quick Actions

Quick Actions MAY include:

- Create City Administrator
- View Platform Analytics
- Export AI Dataset
- View Audit Logs
- Manage Configuration

Actions SHALL be displayed according to backend-supported capabilities.

20.15 Empty State

When no platform data exists,

the module SHALL display:

No Platform Data Available

Platform information will appear as the system becomes operational.

20.16 Loading State

The module SHALL utilize:

- Skeleton KPI Cards
- Skeleton Tables
- Skeleton Charts

- Skeleton Dashboard Widgets

Independent sections SHOULD load asynchronously.

20.17 Error Handling

The module SHALL gracefully handle:

- Platform data unavailable
- Permission denied
- Export failure
- Network failure
- Server errors

Errors SHALL be localized to the affected component whenever possible.

20.18 Performance Requirements

The Platform Administration Module SHALL support:

- Efficient API utilization
 - Server-side pagination
 - Lazy-loaded analytics
 - Responsive rendering
 - Progressive loading
-

20.19 Accessibility

The module SHALL support:

- Keyboard navigation
 - Screen readers
 - Accessible charts
 - Accessible tables
 - High Contrast Mode
 - ARIA-compliant controls
-

20.20 Security

The Platform Administration Module SHALL enforce:

- Authentication
- RBAC
- Protected routes
- Secure API communication

Platform-wide information SHALL remain accessible only to Super Administrators.

20.21 API Integration

The module SHALL integrate with backend APIs responsible for:

- Platform Statistics
- AI Dataset Operations
- System Health
- Audit Logs
- Configuration
- Administrative Search

The frontend SHALL remain independent of backend implementation details.

20.22 Future Enhancements

The architecture SHALL support:

- Multi-Tenant Administration
- Regional Governance
- State-Level Administration
- Central Government Dashboard
- AI Model Management
- Background Job Monitoring
- Feature Flags
- Maintenance Mode
- Disaster Recovery Dashboard
- Infrastructure Monitoring

These enhancements SHALL integrate without requiring architectural redesign.

20.23 Chapter Summary

The Platform Administration Module SHALL provide Super Administrators with centralized governance capabilities for the entire CSCRS ecosystem.

The module SHALL enable:

- Platform-wide monitoring.
- Administrative governance.
- AI dataset administration.
- System health visibility.
- Audit management.
- Executive-level oversight.

The architecture SHALL support future expansion from a single-city deployment to multi-city, district, state, and national implementations.

Chapter 21 — Frontend Architecture & API Integration

21.1 Overview

The CSCRS Administrative Web Portal SHALL adopt a scalable, modular, component-driven frontend architecture.

The architecture SHALL emphasize:

- Scalability
- Maintainability
- Reusability
- Testability
- Performance
- Security
- Separation of Concerns

The frontend SHALL integrate seamlessly with the existing FastAPI backend through REST APIs without introducing business logic duplication.

21.2 Architectural Principles

The frontend SHALL follow the following principles:

- Component-Based Development
- Single Responsibility Principle
- Separation of UI and Business Logic

- API-first Integration
- Reusable UI Components
- Centralized State Management
- Modular Folder Structure
- Lazy Loading
- Type Safety
- Accessibility by Design

21.3 Technology Stack

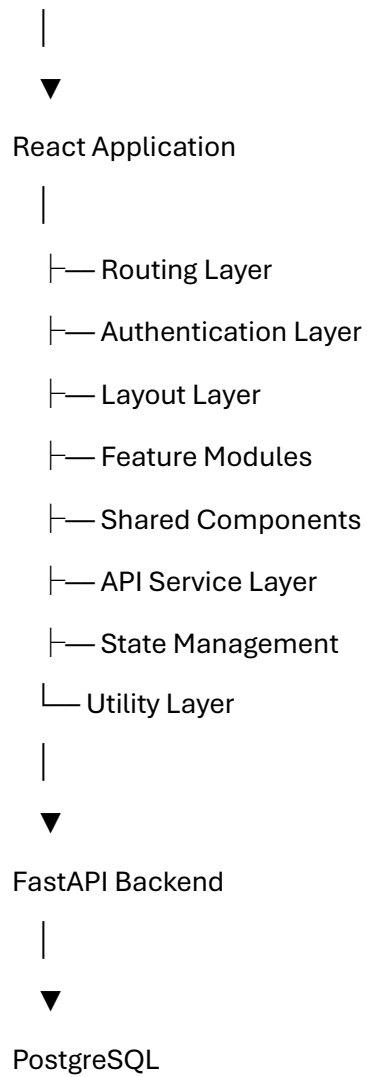
The portal SHALL use the following technologies.

Layer	Technology
Framework	React 19+
Language	TypeScript
Build Tool	Vite
Routing	React Router
UI Components	shadcn/ui
Styling	Tailwind CSS
Icons	Lucide React
Data Fetching	TanStack Query
Forms	React Hook Form
Validation	Zod
Charts	Recharts
Tables	TanStack Table
Notifications	Sonner
HTTP Client	Axios

Alternative technologies SHALL require architectural review before adoption.

21.4 High-Level Frontend Architecture

Browser



21.5 Recommended Folder Structure

The frontend SHALL follow a feature-oriented folder organization.

src/

└─ app/

└─ assets/

└─ components/

└─ common/

| └─ layout/

| └─ charts/

| └─ forms/

| └─ tables/

| └─ feedback/

└─ features/

| └─ authentication/

| └─ dashboard/

| └─ reports/

| └─ workers/

| └─ departments/

| └─ department-admins/

| └─ city-admins/

| └─ notifications/

| └─ analytics/

| |— profile/

| |— settings/

| |— platform/

|— hooks/

|— layouts/

|— lib/

|— services/

|— store/

|— styles/

|— types/

|— utils/

|— main.tsx

Every feature SHALL remain self-contained.

21.6 Feature Module Structure

Each feature SHALL follow the same internal structure.

Example:

reports/

|— components/

|— pages/

|— hooks/

|— services/

|— types/

|— validation/

└ index.ts

This structure SHALL be consistent across all modules.

21.7 API Service Layer

The frontend SHALL communicate with the backend exclusively through a centralized API Service Layer.

Direct Axios calls from UI components SHALL be prohibited.

The API Service Layer SHALL:

- Encapsulate HTTP communication.
 - Handle authentication headers.
 - Normalize API responses.
 - Centralize error handling.
 - Support request cancellation.
 - Support retry policies.
-

21.8 API Integration Strategy

Every frontend module SHALL map to one or more backend services.

Examples:

Dashboard

↓

Dashboard Service

↓

Dashboard APIs

Reports

↓

Report Service

↓

Report APIs

UI components SHALL never consume backend endpoints directly.

21.9 State Management

State SHALL be categorized into:

Global State

Examples:

- Authentication
- User Profile
- Sidebar State
- Theme

Server State

Examples:

- Reports
- Workers
- Departments
- Analytics

Local State

Examples:

- Form Inputs
- Dialog Visibility
- Selected Row

Appropriate state management techniques SHALL be applied based on data ownership.

21.10 Data Fetching

Server communication SHALL use TanStack Query.

The architecture SHALL support:

- Automatic caching
 - Background refresh
 - Query invalidation
 - Retry
 - Loading state management
 - Optimistic updates (where appropriate)
-

21.11 Authentication Layer

The Authentication Layer SHALL:

- Store authentication state.
- Attach JWT tokens.
- Protect routes.
- Handle session expiration.
- Redirect unauthorized users.

Authentication logic SHALL remain centralized.

21.12 Route Guards

Protected routes SHALL verify:

- Authentication
- Authorization
- Session validity

before rendering protected content.

21.13 Layout Layer

Layouts SHALL remain independent of feature modules.

Examples:

- Public Layout
- Authentication Layout
- Portal Layout
- Error Layout

Layouts SHALL not contain business logic.

21.14 Shared Components

Reusable components SHALL include:

- Buttons
- Cards
- Tables
- Forms
- Inputs
- Modals
- Drawers
- Badges
- Alerts
- Skeletons
- Empty States

Duplicate UI components SHALL be avoided.

21.15 Form Validation

All forms SHALL utilize:

- React Hook Form
- Zod validation

Validation SHALL occur both:

- Client-side
- Server-side

Backend validation SHALL remain authoritative.

21.16 Error Handling

Application errors SHALL be categorized.

Examples:

- Validation Errors
- Authentication Errors
- Authorization Errors
- Network Errors
- Unexpected Errors

Standardized error handling SHALL be applied throughout the application.

21.17 Loading Strategy

Loading SHALL use:

- Skeleton Screens
- Inline Loaders
- Progressive Loading

Blocking loading spinners SHOULD be minimized.

21.18 API Error Strategy

The API Service Layer SHALL:

- Normalize backend errors.
- Display user-friendly messages.
- Preserve backend error codes for debugging.

Business-specific messages SHALL originate from backend responses whenever practical.

21.19 Caching Strategy

The frontend SHALL cache:

- User Profile

- Dashboard Data
- Departments
- Settings

Cached information SHALL be invalidated after relevant updates.

21.20 File Upload Strategy

Uploads SHALL support:

- Progress Indicators
- Validation
- Preview
- Cancellation (Future)

Backend SHALL remain responsible for file validation.

21.21 API Mapping Philosophy

Backend APIs SHALL be grouped by business modules rather than pages.

Example:

Reports Module

↓

Report List API

Report Details API

Timeline API

Resolution API

One backend API MAY support multiple frontend screens.

21.22 Logging Strategy

Frontend logging SHALL remain minimal.

Examples:

- Unexpected Errors
- Failed Requests
- Component Failures

Sensitive information SHALL never be logged.

21.23 Environment Configuration

The application SHALL support environment-based configuration.

Examples:

- Development
- Testing
- Staging
- Production

Sensitive configuration SHALL NOT be hardcoded.

21.24 Build Strategy

The frontend SHALL support:

- Development Build
 - Production Build
 - Code Splitting
 - Tree Shaking
 - Asset Optimization
-

21.25 Future Architecture

The architecture SHALL support:

- Micro Frontends
- Plugin System
- Offline Mode
- PWA
- WebSockets
- AI Assistant

- GIS Integration

without major restructuring.

21.26 API Integration Matrix (High-Level)

Frontend Module	Backend API Group
Authentication	Authentication APIs
Dashboard	Dashboard APIs
Reports	Report APIs
Workers	Worker APIs
Departments	Department APIs
Department Administrators	Department Admin APIs
City Administrators	City Admin APIs
Notifications	Notification APIs
Analytics	Analytics APIs
Profile	User APIs
Settings	Configuration APIs
Platform Administration	Platform APIs

A detailed endpoint-level mapping SHALL be maintained as an implementation artifact and synchronized with backend documentation.

21.27 Chapter Summary

The Frontend Architecture & API Integration chapter defines the technical foundation of the CSCRS Administrative Web Portal.

It establishes:

- Modular architecture.
- Feature-based organization.
- Component reuse.
- Centralized API communication.
- Scalable state management.
- Secure authentication integration.

- Maintainable frontend development practices.

The architecture SHALL remain compatible with future platform expansion while preserving consistency across all modules.

Chapter 22 — Non-Functional Requirements (NFR)

22.1 Overview

The Non-Functional Requirements define the quality attributes of the CSCRS Administrative Web Portal.

These requirements specify how the system SHALL perform rather than what functionality it SHALL provide.

The portal SHALL satisfy enterprise-grade expectations for:

- Performance
 - Scalability
 - Availability
 - Reliability
 - Security
 - Maintainability
 - Accessibility
 - Usability
-

22.2 Performance Requirements

The portal SHALL provide a responsive user experience.

Target performance goals:

Metric	Target
Initial Application Load	≤ 3 seconds (Typical Broadband)
Route Navigation	≤ 500 ms (Cached Resources)
API Response Rendering	≤ 1 second after successful API response
Table Search	≤ 2 seconds (Server-side)
Dashboard Rendering	≤ 2 seconds
Filter Updates	≤ 1 second after API response

Performance SHALL be measured under normal operating conditions.

22.3 Scalability

The frontend architecture SHALL support future growth without requiring major redesign.

The portal SHALL support expansion in terms of:

- Additional modules
- Additional user roles
- Additional cities
- Additional departments
- Increased datasets
- Additional API endpoints

The architecture SHALL remain modular and extensible.

22.4 Availability

The portal SHALL remain available whenever backend services are operational.

Frontend failures SHALL be isolated wherever practical.

A failure in one module SHALL NOT prevent unrelated modules from functioning.

Example:

If Analytics fails,

Dashboard, Reports, and Worker Management SHOULD continue functioning.

22.5 Reliability

The portal SHALL provide consistent behavior across repeated operations.

Users SHALL receive predictable responses for identical actions.

Unexpected application crashes SHALL be minimized through:

- Error Boundaries
 - API Error Handling
 - Component Isolation
-

22.6 Maintainability

The frontend SHALL emphasize maintainability through:

- Modular architecture

- Reusable components
- Feature-based organization
- Centralized services
- Consistent naming conventions
- Clear separation of concerns

Future developers SHALL be capable of extending modules without widespread code modifications.

22.7 Extensibility

The architecture SHALL support future addition of:

- New administrative modules
- New report categories
- New analytics
- New dashboards
- AI assistants
- GIS integration
- Plugin architecture

Future enhancements SHALL integrate without restructuring the existing application.

22.8 Usability

The portal SHALL prioritize ease of use.

Typical administrative tasks SHOULD require minimal navigation.

Frequently used operations SHOULD remain accessible within two or three interactions whenever practical.

The interface SHALL remain consistent throughout the application.

22.9 Learnability

New administrators SHOULD become productive with minimal training.

The interface SHALL use:

- Predictable layouts
- Familiar interaction patterns
- Clear navigation

- Standard terminology
-

22.10 Accessibility

The portal SHALL comply with modern accessibility practices.

Support SHALL include:

- Keyboard navigation
- Visible focus indicators
- Screen readers
- Semantic HTML
- ARIA attributes
- Sufficient color contrast

Accessibility SHALL be considered during initial development rather than retrofitted later.

22.11 Browser Compatibility

The portal SHALL support current stable versions of major desktop browsers.

Supported browsers SHALL include:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Apple Safari

The application SHALL gracefully handle unsupported browsers.

22.12 Responsive Design

The portal SHALL support:

- Desktop (Primary Target)
- Laptop
- Tablet
- Mobile Browser (Limited Administrative Use)

Complex administrative workflows SHALL remain optimized for desktop environments.

22.13 Internationalization

The architecture SHALL support multilingual interfaces.

Initial languages:

- English
- Hindi

Additional languages SHALL be introduced without requiring redesign.

22.14 Localization

The portal SHALL support localized presentation of:

- Dates
- Time
- Numbers
- Language
- Time Zone

Localization SHALL remain configurable.

22.15 Consistency

The application SHALL maintain consistency in:

- Navigation
- Buttons
- Forms
- Tables
- Icons
- Colors
- Typography
- Error Messages

Every feature SHALL comply with the Design System defined in Chapter 8.

22.16 Data Integrity

The frontend SHALL accurately present backend-provided information.

The frontend SHALL NOT alter, fabricate, or reinterpret business data.

Backend responses SHALL remain the authoritative source.

22.17 Error Recovery

Whenever practical,

the portal SHALL allow users to recover from recoverable errors.

Examples include:

- Retry failed requests
- Refresh data
- Correct validation errors
- Resume interrupted operations

22.18 Resource Utilization

The frontend SHALL minimize unnecessary:

- API requests
- Re-renders
- Memory usage
- Asset downloads

Efficient resource utilization SHALL improve responsiveness.

22.19 Logging

Frontend logging SHALL remain minimal.

Logs MAY include:

- Unexpected component failures
- Network request failures
- Runtime exceptions

Sensitive information SHALL never be logged.

22.20 Compatibility

The frontend SHALL remain compatible with:

- Existing FastAPI backend
- Existing JWT authentication
- Existing PostgreSQL schema

- Existing RBAC implementation

The frontend SHALL not require backend architectural redesign for initial deployment.

22.21 Backup & Recovery

The frontend SHALL recover gracefully from:

- Browser refresh
- Temporary network interruption
- Session restoration (subject to authentication state)

Recovery SHALL preserve user workflow whenever practical.

22.22 Quality Attributes

The portal SHALL prioritize:

- Simplicity
- Stability
- Predictability
- Performance
- Security
- Maintainability
- Scalability

These attributes SHALL guide future development decisions.

22.23 Technical Debt

New implementations SHOULD minimize technical debt by:

- Reusing shared components
- Avoiding duplicate logic
- Following established architecture
- Maintaining documentation

Refactoring SHALL be performed only when it provides measurable architectural benefit.

22.24 Future Readiness

The architecture SHALL remain compatible with future technologies including:

- Progressive Web Applications (PWA)
 - AI Assistants
 - WebSockets
 - GIS Integration
 - Offline Synchronization
 - Plugin Ecosystem
 - Multi-Tenant Deployments
-

22.25 Chapter Summary

The Non-Functional Requirements establish the quality standards for the CSCRS Administrative Web Portal.

The portal SHALL prioritize:

- High performance.
- Maintainability.
- Scalability.
- Reliability.
- Accessibility.
- Usability.
- Extensibility.

Compliance with these requirements SHALL ensure that the platform remains suitable for long-term production deployment and future expansion.

Chapter 23 — Security, Privacy & Compliance

23.1 Overview

The CSCRS Administrative Web Portal SHALL implement security, privacy, and compliance measures appropriate for an enterprise-grade government administrative platform.

Security SHALL be implemented using a defense-in-depth approach in which frontend and backend responsibilities complement each other.

The frontend SHALL never replace backend security mechanisms.

Backend validation SHALL remain the authoritative enforcement layer.

23.2 Security Objectives

The portal SHALL:

- Protect authenticated users.
- Protect sensitive administrative information.
- Prevent unauthorized access.
- Reduce common web vulnerabilities.
- Maintain data confidentiality.
- Preserve data integrity.
- Support future security enhancements.

23.3 Security Architecture

The portal SHALL implement layered security.

User

↓

HTTPS

↓

Authentication

↓

Route Protection

↓

Role Verification

↓

API Authorization



Backend Validation



Database

Frontend SHALL participate only in presentation-layer security.

Business authorization SHALL remain enforced by backend services.

23.4 Authentication Security

The frontend SHALL:

- Require authentication before accessing protected routes.
- Verify authentication state.
- Handle session expiration.
- Redirect unauthenticated users.
- Remove authentication information during logout.

The frontend SHALL NOT implement authentication independently.

23.5 Authorization

Every protected page SHALL verify user permissions.

Unauthorized users SHALL NOT:

- View protected modules.
- Trigger protected actions.
- Access unauthorized routes.

Frontend authorization SHALL improve user experience.

Backend authorization SHALL remain the final authority.

23.6 Role-Based Access Control

The portal SHALL enforce Role-Based Access Control.

Supported administrative roles:

- Department Administrator
- City Administrator
- Super Administrator

The frontend SHALL dynamically render only authorized interface elements.

23.7 Session Security

Authenticated sessions SHALL support:

- Session validation
- Session expiration
- Secure logout
- Automatic unauthorized redirect

Future implementations MAY include:

- Session timeout warning
 - Active session management
 - Device management
-

23.8 Secure Communication

All communication SHALL occur using HTTPS.

The frontend SHALL never communicate with backend APIs over unsecured HTTP in production environments.

Mixed-content requests SHALL be avoided.

23.9 JWT Handling

The frontend SHALL integrate with the existing backend JWT implementation.

The authentication strategy SHALL remain compatible with future migration to HttpOnly Secure Cookies.

Authentication implementation SHALL remain replaceable without redesigning application modules.

23.10 Client-Side Storage Policy

The frontend SHALL minimize storage of sensitive information.

Sensitive data SHALL NOT be stored unnecessarily.

The frontend SHALL avoid storing:

- Passwords
- OTPs
- Secret Keys
- API Secrets
- Personally Sensitive Information

Storage SHALL remain limited to authenticated session requirements.

23.11 Route Protection

Protected routes SHALL verify:

- Authentication
- Authorization
- Session validity

before rendering protected content.

Unauthorized pages SHALL never become visible before verification completes.

23.12 Input Validation

Client-side validation SHALL improve usability.

Examples include:

- Required fields
- Input format
- Email format
- Length validation
- Numeric validation

Business validation SHALL remain implemented exclusively by backend services.

23.13 Output Encoding

The frontend SHALL safely render backend-provided content.

User-generated content SHALL never be inserted into the DOM using unsafe rendering techniques.

Where dynamic HTML rendering is unavoidable, content SHALL first be sanitized according to backend-approved policies.

23.14 Cross-Site Scripting (XSS)

The application SHALL minimize XSS risk by:

- Avoiding unsafe HTML injection.
- Escaping dynamic content.
- Rendering user input safely.
- Following framework security best practices.

The frontend SHALL avoid bypassing built-in rendering protections.

23.15 Cross-Site Request Forgery (CSRF)

The current JWT-based authentication strategy SHALL follow the backend implementation.

If future authentication migrates to cookie-based sessions,
appropriate CSRF protection SHALL be implemented by backend services.

The frontend SHALL remain compatible with such enhancements.

23.16 Content Security Policy (CSP)

Production deployments SHOULD implement a Content Security Policy.

The frontend SHALL avoid practices that conflict with restrictive CSP rules.

Examples:

- Inline scripts SHOULD be avoided.
- Dynamic script injection SHOULD be minimized.
- Third-party scripts SHALL be reviewed before integration.

CSP enforcement SHALL be configured by deployment infrastructure.

23.17 Secure HTTP Headers

The production environment SHOULD support secure response headers including:

- Content-Security-Policy
- X-Content-Type-Options

- Referrer-Policy
- Permissions-Policy
- Strict-Transport-Security (HTTPS deployments)
- X-Frame-Options (or equivalent CSP frame-ancestors directive)

These headers SHALL be configured by the backend or reverse proxy.

The frontend SHALL remain compatible with these security controls.

23.18 File Upload Security

The frontend SHALL perform basic file validation before upload.

Validation MAY include:

- File size
- File type
- Supported extensions

The backend SHALL remain responsible for authoritative validation including MIME verification, malware scanning (if implemented), and storage policies.

23.19 Privacy

The frontend SHALL display only information authorized for the authenticated user.

Personally identifiable information (PII) SHALL be minimized wherever practical.

Information SHALL never be displayed outside the authenticated user's authorization scope.

23.20 Sensitive Data Handling

Sensitive information SHALL NOT appear in:

- Console logs
- Error dialogs
- URLs
- Browser history (where avoidable)

Detailed technical errors SHALL remain restricted to backend logging.

23.21 Auditability

Administrative actions SHALL be traceable through backend audit mechanisms.

The frontend SHALL accurately display audit information provided by backend APIs.

The frontend SHALL NOT fabricate audit records.

23.22 Error Messages

Security-related error messages SHALL remain generic.

Examples:

✓ "Authentication failed."

✓ "Access denied."

The interface SHALL NOT disclose:

- Internal server details
 - Stack traces
 - Database information
 - Authentication internals
-

23.23 Third-Party Dependencies

Frontend dependencies SHALL:

- Be actively maintained.
- Receive security updates.
- Be reviewed before adoption.

Unused dependencies SHOULD be removed.

23.24 Browser Security

The portal SHALL rely on modern browser security capabilities.

Users SHOULD access the portal using supported browser versions.

Legacy browser compatibility SHALL not compromise application security.

23.25 Privacy Compliance

The portal SHALL support future privacy compliance initiatives.

Examples MAY include:

- Privacy Policy
- Cookie Notice

- Data Retention Policies
- User Consent (if applicable)
- Right-to-Information support (where applicable)

Legal implementation SHALL follow applicable organizational and governmental policies.

23.26 Security Testing

Security verification SHOULD include:

- Authentication testing
- Authorization testing
- Session testing
- Route protection testing
- File upload testing
- Input validation testing
- Dependency vulnerability scanning

Backend security testing SHALL remain independent of frontend testing.

23.27 Incident Readiness

The architecture SHALL support future security enhancements including:

- Account lockout notifications
- Security alerts
- Login anomaly detection
- Device verification
- Risk-based authentication

These capabilities SHALL integrate without requiring frontend architectural redesign.

23.28 Future Enhancements

The architecture SHALL support:

- Multi-Factor Authentication (MFA)
- Hardware Security Keys (WebAuthn/FIDO2)
- Single Sign-On (SSO)
- Government Identity Integration

- Conditional Access Policies
- Security Dashboard

These enhancements SHALL remain compatible with the existing architecture.

23.29 Chapter Summary

The Security, Privacy & Compliance chapter establishes the security expectations for the CSCRS Administrative Web Portal.

The frontend SHALL:

- Protect authenticated sessions.
- Respect Role-Based Access Control.
- Safely render application data.
- Minimize client-side exposure of sensitive information.
- Integrate securely with backend services.
- Support future enterprise security enhancements.

Security SHALL remain a shared responsibility between the frontend, backend, infrastructure, and deployment environment, with backend services retaining authoritative control over authentication, authorization, business validation, and audit enforcement.

Chapter 24 — Testing Strategy, Quality Assurance & Acceptance Criteria

24.1 Overview

The CSCRS Administrative Web Portal SHALL undergo comprehensive verification and validation before being considered production-ready.

Testing SHALL verify:

- Functional correctness
- Security
- Performance
- Accessibility
- Compatibility
- Reliability
- User experience

The testing strategy SHALL combine automated testing, manual verification, and User Acceptance Testing (UAT).

24.2 Testing Objectives

The testing process SHALL ensure that:

- Every functional requirement is implemented correctly.
 - All user roles behave according to RBAC policies.
 - The frontend integrates correctly with backend APIs.
 - Security requirements are satisfied.
 - Performance targets are achieved.
 - Accessibility requirements are fulfilled.
-

24.3 Testing Levels

The frontend SHALL be tested at multiple levels.

Testing Level	Purpose
Unit Testing	Verify individual components
Integration Testing	Verify interaction between modules
API Integration Testing	Verify backend communication
End-to-End Testing	Verify complete user workflows
User Acceptance Testing	Validate business requirements
Regression Testing	Prevent previously fixed issues
Accessibility Testing	Verify WCAG compliance
Performance Testing	Verify responsiveness

24.4 Unit Testing

Unit tests SHALL verify:

- UI Components
- Utility Functions
- Validation Logic
- Hooks
- Helper Functions
- Shared Components

Unit tests SHALL isolate components from backend dependencies using mocks.

24.5 Component Testing

Reusable UI components SHALL be independently verified.

Examples include:

- Button
- Table
- Form
- Modal
- Drawer
- Card
- Badge
- Input
- Breadcrumb
- Notification

Each reusable component SHALL behave consistently throughout the portal.

24.6 Integration Testing

Integration tests SHALL verify interactions between:

- Components
- Layouts
- Feature Modules
- Routing
- Authentication
- API Services

Integration testing SHALL ensure correct communication between frontend layers.

24.7 API Integration Testing

Every backend integration SHALL be verified.

Typical scenarios include:

- Successful API responses
- Validation failures

- Authentication failures
- Authorization failures
- Network interruptions
- Timeout handling
- Empty responses

The frontend SHALL gracefully handle all supported backend responses.

24.8 End-to-End Testing

End-to-End testing SHALL simulate complete business workflows.

Examples include:

Administrator Login

↓

Dashboard

↓

Reports

↓

Report Details

↓

Logout

Additional workflows SHALL include:

- Worker Management
- Department Management
- Analytics
- Notifications
- Profile Update

24.9 User Acceptance Testing (UAT)

User Acceptance Testing SHALL verify that the portal satisfies business requirements.

Representative administrative users SHOULD validate:

- Ease of navigation
- Workflow efficiency

- Dashboard usability
- Report management
- Administrative operations

Acceptance SHALL be based on documented business scenarios.

24.10 Regression Testing

Regression testing SHALL be performed:

- Before each release
- After major feature implementation
- After backend API modifications
- After dependency upgrades

Previously verified functionality SHALL continue operating correctly.

24.11 Role-Based Testing

Every feature SHALL be verified using each administrative role.

Roles include:

- Department Administrator
- City Administrator
- Super Administrator

Testing SHALL confirm:

- Authorized access
 - Unauthorized restrictions
 - Hidden navigation
 - Correct dashboards
-

24.12 Authentication Testing

Authentication testing SHALL verify:

- Login
- Logout
- Session expiration
- Protected routes

- Invalid credentials
- Password change
- Forgot Password workflow

Authentication SHALL remain consistent with backend implementation.

24.13 Navigation Testing

Testing SHALL verify:

- Sidebar navigation
- Breadcrumb updates
- Dashboard shortcuts
- Global Search
- Browser navigation
- Deep linking

Navigation SHALL remain predictable throughout the application.

24.14 Responsive Testing

The portal SHALL be tested across:

- Desktop
- Laptop
- Tablet
- Mobile Browser

Critical administrative workflows SHALL remain fully usable on supported screen sizes.

24.15 Browser Compatibility Testing

Testing SHALL verify compatibility with:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Apple Safari

The portal SHALL remain functional across supported browsers.

24.16 Accessibility Testing

Accessibility verification SHALL include:

- Keyboard Navigation
- Focus Indicators
- Screen Reader Compatibility
- Semantic HTML
- ARIA Labels
- Color Contrast

Testing SHOULD utilize automated accessibility tools supplemented by manual verification.

24.17 Performance Testing

Performance SHALL be evaluated using metrics including:

- Initial Load Time
- Dashboard Rendering
- Route Navigation
- Large Table Rendering
- API Response Rendering

Performance targets SHALL align with Chapter 22.

24.18 Error Handling Testing

Testing SHALL verify:

- Invalid Inputs
- API Failures
- Network Loss
- Unauthorized Access
- Empty States
- Unexpected Errors

The portal SHALL remain stable during error conditions.

24.19 Notification Testing

Notification testing SHALL verify:

- Notification List
- Read Status
- Deep Linking
- Priority Indicators
- Notification Counts

Future real-time notification testing SHALL be documented separately.

24.20 Security Testing

Frontend security verification SHALL include:

- Route Protection
- Session Handling
- Unauthorized Access
- Input Validation
- Secure Storage
- Dependency Vulnerability Review

Backend penetration testing SHALL remain outside the scope of frontend verification.

24.21 Visual Regression Testing

Visual consistency SHALL be verified after significant UI changes.

Testing SHALL ensure:

- Layout consistency
- Typography consistency
- Color consistency
- Responsive behavior
- Component appearance

Unexpected UI regressions SHOULD be identified before release.

24.22 Acceptance Criteria

The frontend SHALL be considered production-ready only when:

- All critical functional tests pass.
- All high-severity defects are resolved.

- Authentication functions correctly.
 - RBAC behaves correctly.
 - API integrations operate successfully.
 - Accessibility requirements are met.
 - Performance targets are achieved.
 - Regression testing passes.
 - User Acceptance Testing is approved.
-

24.23 Defect Classification

Defects SHOULD be categorized as:

Severity Description

Critical System unusable or security risk

High Major functionality unavailable

Medium Feature works with limitations

Low Cosmetic or minor usability issue

Release decisions SHALL prioritize resolution based on severity.

24.24 Release Readiness Checklist

Before production deployment, the following SHALL be verified:

- Functional Testing Complete
 - API Integration Verified
 - Security Review Complete
 - Accessibility Review Complete
 - Browser Compatibility Verified
 - Performance Targets Met
 - Documentation Updated
 - Release Approved
-

24.25 Test Documentation

Testing SHALL produce:

- Test Plan
- Test Cases
- Test Execution Report
- Defect Report
- Regression Report
- UAT Sign-Off

Documentation SHALL be maintained throughout the project lifecycle.

24.26 Future Enhancements

Future testing improvements MAY include:

- Automated End-to-End Pipelines
 - Visual Regression Automation
 - Continuous Accessibility Testing
 - Performance Monitoring Dashboards
 - Synthetic User Monitoring
 - AI-Assisted Test Generation
-

24.27 Chapter Summary

The Testing Strategy, Quality Assurance & Acceptance Criteria chapter establishes the verification framework for the CSCRS Administrative Web Portal.

It ensures that the portal is:

- Functionally correct.
- Secure.
- Performant.
- Accessible.
- Maintainable.
- Production-ready.

The portal SHALL only be released after successfully satisfying all defined acceptance criteria.

Chapter 25 — Deployment, Environment & Release Management

25.1 Overview

The CSCRS Administrative Web Portal SHALL support a structured deployment strategy that enables reliable development, testing, staging, and production releases.

The deployment architecture SHALL prioritize:

- Stability
- Security
- Repeatability
- Scalability
- Recoverability

The deployment process SHALL minimize downtime while maintaining compatibility with the existing backend infrastructure.

25.2 Deployment Objectives

The deployment strategy SHALL:

- Ensure predictable releases.
- Separate development from production.
- Support rollback.
- Reduce deployment risk.
- Enable continuous improvement.
- Prepare for CI/CD automation.

25.3 Deployment Environments

The frontend SHALL support multiple deployment environments.

Environment	Purpose
Local Development	Individual developer environment
Testing	QA verification
Staging	Pre-production validation
Production	Live deployment

Each environment SHALL maintain independent configuration.

25.4 Environment Configuration

Environment-specific configuration SHALL be provided using environment variables.

Examples include:

- API Base URL
- Application Name
- Environment Name
- Logging Level
- Feature Flags (Future)

Sensitive configuration SHALL NOT be hardcoded into the application.

25.5 Build Process

The production build SHALL:

- Compile TypeScript.
- Optimize JavaScript bundles.
- Minify assets.
- Remove development code.
- Generate hashed asset filenames.
- Optimize images where applicable.

Build artifacts SHALL be immutable once generated.

25.6 Deployment Workflow

The recommended deployment workflow SHALL be:

Developer

↓

Git Repository

↓

Build

↓

Testing

↓

Staging

↓

Approval

↓

Production Deployment

↓

Monitoring

Each deployment SHALL be traceable.

25.7 Versioning

The frontend SHALL follow **Semantic Versioning (SemVer)**.

Version format:

MAJOR.MINOR.PATCH

Example

1.0.0

Examples:

- MAJOR → Breaking changes
- MINOR → New features

- PATCH → Bug fixes

Frontend and backend versioning SHOULD remain independently manageable.

25.8 Release Strategy

Every production release SHALL include:

- Release Notes
- Deployment Checklist
- Rollback Plan
- Version Tag
- Change Summary

Critical releases SHOULD undergo staging validation before production deployment.

25.9 Rollback Strategy

If deployment fails,

the deployment process SHALL support rollback to the previously verified version.

Rollback SHALL restore:

- Previous frontend assets
- Previous application version

Rollback SHALL NOT require backend schema changes for frontend-only releases.

25.10 Static Asset Management

The frontend SHALL optimize static assets.

Examples include:

- Images
- Fonts
- Icons
- CSS
- JavaScript

Assets SHALL utilize cache-friendly filenames generated during production builds.

25.11 Caching Strategy

The deployment SHALL support efficient caching.

Recommendations include:

- Long-term caching for versioned assets.
- Short caching for HTML entry files.
- Controlled cache invalidation after new releases.

The caching strategy SHALL minimize unnecessary downloads while ensuring users receive updated application versions.

25.12 Reverse Proxy Compatibility

The frontend SHALL remain compatible with deployment behind a reverse proxy such as:

- Nginx
- Apache HTTP Server
- Cloud-based reverse proxies

Reverse proxy configuration SHALL support HTTPS and Single Page Application (SPA) routing.

25.13 HTTPS Requirements

Production deployments SHALL use HTTPS.

HTTP requests SHOULD redirect to HTTPS.

TLS certificate management SHALL be handled by deployment infrastructure.

25.14 Logging & Monitoring

The frontend SHALL support integration with monitoring systems.

Typical monitoring MAY include:

- JavaScript runtime errors
- Failed API requests
- Performance metrics
- Client-side exceptions

Sensitive information SHALL NOT be transmitted through monitoring systems.

25.15 Health Verification

After deployment,

basic verification SHOULD confirm:

- Application loads successfully.
 - Authentication functions correctly.
 - API connectivity is operational.
 - Protected routes function correctly.
 - Static assets load successfully.
-

25.16 CI/CD Readiness

The architecture SHALL support future Continuous Integration and Continuous Deployment pipelines.

Future automation MAY include:

- Automated Build
- Automated Testing
- Automated Deployment
- Automated Rollback
- Quality Gates

The initial implementation MAY use manual deployment procedures.

25.17 Release Approval

Production deployment SHOULD occur only after:

- Functional testing passed.
 - Security review completed.
 - Regression testing passed.
 - UAT approval received.
 - Release documentation completed.
-

25.18 Backup Considerations

Frontend deployment SHALL preserve:

- Previous release artifacts.
- Configuration files.
- Deployment metadata.

Backup of backend data SHALL remain outside the scope of frontend deployment.

25.19 Downtime Strategy

Frontend deployments SHOULD minimize service interruption.

Whenever practical,

deployment SHOULD be completed without noticeable user disruption.

25.20 Browser Cache Refresh

After deployment,

users SHOULD automatically receive updated application assets after cache invalidation.

Hard refresh SHOULD NOT be routinely required.

25.21 Production Checklist

Before production deployment, verify:

- Environment variables configured.
 - API endpoint verified.
 - HTTPS enabled.
 - Production build generated.
 - Static assets validated.
 - Browser compatibility confirmed.
 - Accessibility verification completed.
 - Monitoring configured.
 - Rollback package available.
-

25.22 Documentation

Each release SHALL include updated documentation.

Examples:

- Release Notes
- Deployment Notes
- Known Issues
- Configuration Changes

- Version History
-

25.23 Future Enhancements

The deployment architecture SHALL support:

- Docker-based frontend deployment
- Kubernetes deployments
- CDN integration
- Blue-Green deployments
- Canary releases
- Automated rollback
- Multi-region hosting

These enhancements SHALL integrate without requiring architectural redesign.

25.24 Chapter Summary

The Deployment, Environment & Release Management chapter establishes the operational framework for delivering the CSCRS Administrative Web Portal into production.

The deployment strategy SHALL provide:

- Structured environments.
- Repeatable releases.
- Secure configuration.
- Rollback capability.
- Monitoring readiness.
- Future CI/CD compatibility.

The architecture SHALL remain scalable for future enterprise deployment requirements.

Chapter 26 — Frontend Development Standards & AI Implementation Guide

26.1 Overview

This chapter establishes the official engineering standards, architectural rules, coding conventions, and AI-assisted development guidelines for the CSCRS Administrative Web Portal.

All frontend development SHALL comply with these standards to ensure:

- Consistency
- Maintainability

- Scalability
- Readability
- Production readiness

These standards SHALL apply equally to human developers and AI-assisted code generation.

26.2 Development Philosophy

Frontend development SHALL prioritize:

- Simplicity
- Reusability
- Predictability
- Performance
- Accessibility
- Maintainability

Developers SHALL prefer improving existing architecture over introducing unnecessary abstractions.

26.3 Technology Standards

Only approved technologies SHALL be used.

Category	Approved Technology
----------	---------------------

Framework	React
Language	TypeScript
Build Tool	Vite
CSS	Tailwind CSS
Components	shadcn/ui
Icons	Lucide React
API Client	Axios
Server State	TanStack Query
Forms	React Hook Form
Validation	Zod

Alternative libraries SHALL require architectural review.

26.4 Folder Standards

Every feature SHALL follow the standardized module structure.

Example:

feature/

components/

pages/

hooks/

services/

types/

validation/

index.ts

Folder organization SHALL remain consistent across the entire project.

26.5 Naming Conventions

Developers SHALL follow consistent naming.

Examples:

Components

ReportCard.tsx

WorkerTable.tsx

DashboardLayout.tsx

Hooks

useReports.ts

useDashboard.ts

Services

reportService.ts

workerService.ts

Types

report.types.ts

worker.types.ts

Naming SHALL remain descriptive.

26.6 Component Design Principles

Every component SHALL have a single responsibility.

Large components SHOULD be divided into smaller reusable components.

Reusable UI elements SHALL reside within shared component libraries whenever appropriate.

26.7 Page Design Rules

Pages SHALL:

- Compose components.
- Trigger data loading.
- Coordinate layouts.

Pages SHALL NOT contain large amounts of business logic.

26.8 Business Logic

Business logic SHALL remain outside UI components.

Business operations SHALL be implemented within:

- Services
- Hooks
- Utility functions

UI components SHALL remain presentation-focused.

26.9 API Integration Rules

All backend communication SHALL occur through Service classes.

The following SHALL NOT be permitted:

✗ Axios directly inside components

✗ Fetch directly inside JSX

Correct flow:

Page

↓

Hook

↓

Service

↓

API

↓

Backend

26.10 State Management Rules

Developers SHALL separate state.

Global State

Authentication

Theme

Sidebar

User

Server State

Reports

Workers

Departments

Analytics

Local State

Dialogs

Forms

Dropdowns

Filters

State duplication SHOULD be avoided.

26.11 Component Reuse

Before creating a component,

developers SHALL verify whether an equivalent shared component already exists.

Duplicate UI components SHOULD NOT be created.

26.12 Styling Standards

Tailwind CSS SHALL be used consistently.

Inline styles SHOULD be avoided except where technically necessary.

Spacing, typography, and colors SHALL follow the Design System defined earlier in this specification.

26.13 Forms

All forms SHALL utilize:

- React Hook Form
- Zod Validation

Validation SHALL remain centralized.

26.14 Tables

All data tables SHALL utilize the approved table implementation.

Tables SHALL support:

- Pagination
- Sorting
- Search
- Filters
- Empty State
- Loading State

Table behavior SHALL remain consistent throughout the portal.

26.15 Error Handling

All user-facing errors SHALL use standardized UI components.

Developers SHALL NOT expose:

- Stack traces
- API internals
- Database errors

Unexpected runtime errors SHALL be captured by Error Boundaries.

26.16 Loading States

Every asynchronous page SHALL implement loading states.

Preferred techniques include:

- Skeletons
- Progressive rendering

Blocking loaders SHOULD be minimized.

26.17 Accessibility Standards

Every component SHALL support:

- Keyboard navigation
- Focus management
- Screen readers

- ARIA labels
- High contrast compatibility

Accessibility SHALL be considered during development rather than after implementation.

26.18 Documentation Standards

Every major feature SHALL include:

- Purpose
- Component description
- API dependencies
- Important assumptions

Complex business logic SHOULD include concise explanatory comments where they improve maintainability.

26.19 Git Standards

Recommended branch naming:

feature/report-module

feature/dashboard

bugfix/login

refactor/sidebar

Commit messages SHOULD follow a consistent convention.

Examples:

feat: add worker management

fix: resolve dashboard loading issue

refactor: optimize report table

26.20 Code Review Checklist

Before merging code, verify:

- Architecture followed.
 - No duplicate components.
 - API layer used correctly.
 - TypeScript errors resolved.
 - Responsive design verified.
 - Accessibility reviewed.
 - Loading states implemented.
 - Error handling present.
-

26.21 AI-Assisted Development Guidelines

AI-generated code SHALL comply with the existing architecture.

AI SHALL:

- Reuse existing components.
 - Preserve folder structure.
 - Respect naming conventions.
 - Follow TypeScript standards.
 - Avoid architectural redesign.
 - Integrate with approved services.
-

26.22 AI Guardrails

When generating code,

AI SHALL NOT:

- Rename existing folders.
- Rename existing components.
- Move files unnecessarily.
- Replace project architecture.
- Introduce unapproved libraries.
- Duplicate business logic.
- Duplicate components.
- Modify backend APIs.

AI SHALL prefer extending the existing implementation.

26.23 AI Prompting Standard

Implementation prompts SHOULD include:

- Module name
- Existing architecture
- Relevant APIs
- UI requirements
- RBAC requirements
- Validation requirements

AI SHALL receive sufficient architectural context before generating implementation code.

26.24 Definition of Done (DoD)

A feature SHALL be considered complete only when:

- Functionality implemented.
 - API integration completed.
 - Loading states implemented.
 - Error handling implemented.
 - Empty states implemented.
 - Responsive behavior verified.
 - Accessibility reviewed.
 - TypeScript compilation successful.
 - No linting errors.
 - Documentation updated.
-

26.25 Pull Request Checklist

Every Pull Request SHOULD include:

- Description
- Screenshots (if UI changes)
- Related issue/task
- Testing summary

- Breaking changes (if any)

Reviewers SHALL verify compliance with this specification.

26.26 Future Enhancements

The development process SHALL support:

- Automated code generation
- AI-assisted refactoring
- AI code review
- Automated documentation
- Automated testing
- Continuous quality analysis

These capabilities SHALL integrate without modifying the established architecture.

26.27 Chapter Summary

The Frontend Development Standards & AI Implementation Guide establishes the engineering rules governing the development of the CSCRS Administrative Web Portal.

It defines:

- Approved technologies.
- Folder structure.
- Coding conventions.
- Component architecture.
- API integration rules.
- AI development guardrails.
- Quality standards.
- Definition of Done.

Compliance with this chapter SHALL ensure that both human developers and AI-assisted tools produce consistent, maintainable, and production-ready code aligned with the overall architecture of the CSCRS platform.

Chapter 27 — AI Implementation Playbook

27.1 Overview

The AI Implementation Playbook establishes the official workflow for AI-assisted frontend development within the CSCRS project.

This chapter defines standardized procedures that SHALL be followed whenever AI systems are used to generate, modify, review, or refactor frontend code.

The objective is to ensure that AI-generated code remains consistent with the approved architecture, coding standards, and project requirements.

27.2 Objectives

The AI Implementation Playbook SHALL:

- Standardize AI-assisted development.
 - Preserve architectural consistency.
 - Reduce duplicate implementations.
 - Improve maintainability.
 - Minimize AI hallucinations.
 - Protect project structure.
 - Increase development productivity.
-

27.3 Approved AI Tools

The project MAY utilize AI development assistants including:

- ChatGPT
- Claude
- GitHub Copilot
- Gemini
- Cursor AI
- Windsurf
- Future approved AI tools

Selection of AI tooling SHALL not affect the established architecture.

27.4 AI Development Philosophy

AI SHALL function as an implementation assistant.

AI SHALL NOT function as an architectural decision-maker.

Project architecture SHALL always take precedence over AI-generated suggestions.

27.5 Standard AI Workflow

Every implementation SHALL follow the sequence below.

Read SRS

↓

Identify Module

↓

Review Existing Structure

↓

Generate Implementation

↓

Developer Review

↓

Testing

↓

Merge

No implementation SHALL skip the architecture review stage.

27.6 Required Context

Before requesting implementation,
AI SHALL receive sufficient project context.

Minimum context SHOULD include:

- Module name
- Existing architecture
- Folder structure
- Backend APIs
- UI requirements
- RBAC rules
- Validation rules

Incomplete context MAY produce incomplete implementations.

27.7 Module-Based Development

Frontend implementation SHALL occur module by module.

Example:

Authentication

↓

Dashboard

↓

Reports

↓

Workers

↓

Departments

↓

Analytics

Large multi-module prompts SHOULD be avoided.

27.8 Existing Code Review

Before generating code,

AI SHALL review the existing implementation.

AI SHALL identify:

- Existing components
- Existing hooks
- Existing services
- Existing layouts
- Existing utilities

Duplicate implementation SHALL be avoided.

27.9 File Creation Rules

AI SHALL create new files only when required.

Before creating any file,

AI SHALL determine whether:

- The functionality already exists.
- An existing file can be extended.
- A shared component can be reused.

Unnecessary file creation SHALL be prohibited.

27.10 Modification Rules

Whenever possible,

AI SHALL modify existing implementations instead of replacing them.

Large-scale rewrites SHALL require explicit approval.

27.11 Architectural Boundaries

AI SHALL NOT:

- Change folder structure.
- Rename existing folders.
- Rename feature modules.
- Replace state management.
- Replace routing architecture.
- Replace UI framework.
- Replace API layer.

Architectural changes SHALL require explicit authorization.

27.12 API Rules

AI SHALL:

- Reuse existing API services.
- Preserve endpoint contracts.
- Avoid duplicating backend logic.
- Respect backend validation.

Business rules SHALL remain implemented exclusively within backend services.

27.13 Component Rules

AI SHALL:

- Prefer reusable components.
- Extend existing components.
- Maintain consistent styling.
- Preserve accessibility.

Duplicate components SHALL be avoided.

27.14 Styling Rules

AI SHALL:

- Follow the Design System.

- Use Tailwind CSS.
- Reuse utility classes.
- Avoid inline styling unless technically required.

Visual consistency SHALL remain a priority.

27.15 Error Handling Rules

Every AI-generated feature SHALL include:

- Loading state
- Empty state
- Error state
- Success state

No asynchronous workflow SHALL omit these states.

27.16 Form Implementation Rules

Forms SHALL include:

- Validation
- Required indicators
- Error messages
- Submission feedback

AI SHALL use approved validation libraries.

27.17 Accessibility Requirements

AI-generated components SHALL support:

- Keyboard navigation
- Focus indicators
- Screen readers
- Semantic HTML
- ARIA attributes where appropriate

Accessibility SHALL not be treated as an optional enhancement.

27.18 Code Quality Checklist

Before considering generated code complete,

AI SHALL verify:

- No TypeScript errors.
 - No lint errors.
 - No duplicate logic.
 - Correct API usage.
 - Responsive layout.
 - Accessibility compliance.
 - Consistent naming.
-

27.19 Review Checklist

Developers SHOULD review AI-generated code for:

- Architectural compliance.
- Performance.
- Security.
- Readability.
- Maintainability.
- Testability.

AI-generated code SHALL not bypass human review.

27.20 Definition of Ready (DoR)

Before implementation begins,

the following SHALL be available:

- Approved SRS section.
- Backend APIs.
- UI requirements.
- RBAC requirements.
- Validation requirements.
- Acceptance criteria.

Implementation SHOULD NOT begin without sufficient requirements.

27.21 Definition of Done (DoD)

A feature SHALL be complete only when:

- UI implemented.
 - API integrated.
 - Error handling completed.
 - Loading states completed.
 - Empty states completed.
 - Responsive behavior verified.
 - Accessibility reviewed.
 - TypeScript passes.
 - Lint passes.
 - Tests completed.
 - Documentation updated.
-

27.22 Prompting Guidelines

AI prompts SHOULD:

- Reference the relevant SRS chapter.
- Specify the target module.
- Mention backend APIs.
- State architectural constraints.
- Request incremental implementation.

Developers SHOULD avoid broad prompts such as:

"Build the whole frontend."

Incremental implementation SHALL be preferred.

27.23 Incremental Development Strategy

Implementation SHALL proceed in small, verifiable increments.

Recommended order:

1. Layout
2. Routing
3. Authentication

4. Dashboard
5. Reports
6. Workers
7. Departments
8. Notifications
9. Analytics
10. Remaining Modules

Each completed module SHALL be reviewed before proceeding.

27.24 AI Limitations

Developers SHALL recognize that AI:

- May misunderstand requirements.
- May introduce duplicate code.
- May recommend unnecessary refactoring.
- May generate outdated patterns.
- May overlook project-specific constraints.

Human oversight SHALL remain mandatory.

27.25 Project-Specific Guardrails

For the CSCRS project, AI SHALL:

- Reuse the approved folder structure.
 - Preserve the React + TypeScript + Vite architecture.
 - Use Tailwind CSS and shadcn/ui components.
 - Use TanStack Query for server state.
 - Keep API communication within the centralized service layer.
 - Respect the existing RBAC implementation.
 - Preserve accessibility and responsive behavior.
 - Avoid introducing libraries unless explicitly approved.
 - Never change backend API contracts.
-

27.26 Future AI Enhancements

The development workflow SHALL support future AI capabilities such as:

- Automated code reviews.
- AI-generated unit tests.
- AI-assisted documentation.
- AI-based accessibility audits.
- Automated performance optimization suggestions.
- Intelligent refactoring recommendations.

These capabilities SHALL augment, not replace, established engineering practices.

27.27 Chapter Summary

The AI Implementation Playbook establishes a standardized methodology for AI-assisted frontend development within the CSCRS project.

It ensures that AI-generated code:

- Adheres to the approved architecture.
- Preserves project consistency.
- Respects backend boundaries.
- Follows engineering standards.
- Supports maintainable, production-ready implementations.

Human review SHALL remain an essential part of the development lifecycle.

Chapter 28 — Master Reference Catalog

28.1 Overview

The Master Reference Catalog SHALL serve as the centralized reference index for the CSCRS Administrative Web Portal.

Unlike previous chapters that define functional and non-functional requirements, this chapter SHALL consolidate the application's structural information into a single reference section.

The Master Reference Catalog SHALL assist:

- Frontend Developers
- Backend Developers
- QA Engineers
- UI/UX Designers
- Technical Writers

- AI Development Assistants
- Future Project Maintainers

28.2 Portal Module Inventory

The Administrative Portal SHALL consist of the following primary modules.

Module	Status
Landing Portal	✓
Authentication	✓
Dashboard	✓
Reports	✓
Workers	✓
Departments	✓
Department Administrators	✓
City Administrators	✓
Notifications	✓
Analytics	✓
Profile	✓
Settings	✓
Platform Administration	✓

Additional modules MAY be introduced in future versions.

28.3 Complete Screen Inventory

The portal SHALL contain the following major screens.

Public Screens

- Home
- Login
- Forgot Password
- Reset Password

- Unauthorized
 - 404 Page
 - 500 Error Page
-

Dashboard

- Dashboard Home
-

Reports

- Report List
 - Report Details
 - Timeline
 - AI Verification
 - Assignment
 - Resolution
 - History
-

Workers

- Worker List
 - Create Worker
 - Worker Details
 - Edit Worker
-

Departments

- Department List
 - Department Details
 - Create Department
 - Edit Department
-

Department Administrators

- Administrator List
- Create Administrator

- Administrator Details
 - Pending Invitations
-

City Administrators

- City Administrator List
 - Create Administrator
 - Details
-

Notifications

- Notification Center
 - Notification Details
-

Analytics

- Dashboard
 - Department Analytics
 - Worker Analytics
 - AI Analytics
 - Executive Reports
-

Profile

- Overview
 - Edit Profile
 - Change Password
-

Settings

- General
 - Notifications
 - Language
 - Appearance
 - Security
-

Platform Administration

- Platform Dashboard
- AI Dataset
- Audit Center
- System Health
- Platform Statistics

28.4 Navigation Tree

Public

|

└─ Home

└─ Login

|



Dashboard

|

└─ Reports

└─ Workers

└─ Departments

└─ Department Administrators

└─ City Administrators

└─ Notifications

└─ Analytics

└─ Profile

└─ Settings

└─ Platform Administration

Every navigation item SHALL correspond to an authorized feature module.

28.5 Route Inventory

The frontend SHOULD maintain a centralized route registry.

Illustrative route structure:

/

/login

/dashboard

/reports

/reports/:id

/workers

/workers/new

/workers/:id

/departments

/departments/:id

/department-admins

/city-admins

/notifications

/analytics

/profile

/settings

/platform

Route names SHALL remain human-readable and REST-inspired where practical.

28.6 Layout Inventory

The application SHALL contain standardized layouts.

Layout	Purpose
Public Layout	Landing Pages
Authentication Layout	Login Pages
Portal Layout	Main Application
Error Layout	Error Screens

Layouts SHALL remain independent from business logic.

28.7 Shared Component Catalog

The Shared Component Library SHALL include reusable UI components.

Examples:

Navigation

- Header
- Sidebar
- Breadcrumb
- Footer

Forms

- Input
- Textarea
- Select
- Checkbox
- Radio
- Switch
- Date Picker

Feedback

- Alert
- Toast
- Dialog
- Modal
- Drawer
- Confirmation Dialog

Display

- Card
- Badge
- Table
- Tabs
- Timeline
- Chart
- Statistic Card

- Empty State
- Skeleton Loader

Utility

- Pagination
- Search Bar
- Filter Panel
- Status Badge
- Avatar
- QR Code Display

Duplicate components SHOULD be avoided.

28.8 RBAC Matrix

Module	Department Admin	City Admin	Super Admin
Dashboard	✓	✓	✓
Reports	✓	✓	✓
Workers	✓	View	View
Departments	Own	✓	✓
Department Administrators	✗	✓	✓
City Administrators	✗	✗	✓
Notifications	✓	✓	✓
Analytics	Own	✓	✓
Profile	Own	Own	Own
Settings	Own	Partial	✓
Platform Administration	✗	✗	✓

Note: The frontend SHALL use this matrix only for interface rendering. The backend SHALL remain the authoritative source for authorization decisions.

28.9 API-to-Module Mapping

Frontend Module	Backend API Group
Authentication	Authentication APIs
Dashboard	Dashboard APIs
Reports	Report APIs
Workers	Worker APIs
Departments	Department APIs
Department Administrators	Department Admin APIs
City Administrators	City Admin APIs
Notifications	Notification APIs
Analytics	Analytics APIs
Profile	User APIs
Settings	Configuration APIs
Platform Administration	Platform APIs

Detailed endpoint documentation SHALL be maintained separately in the Backend API Documentation.

28.10 State Management Reference

Global State

- Authentication
- User
- Theme
- Sidebar

Server State

- Reports
- Workers
- Departments
- Analytics
- Notifications

Local State

- Forms
 - Dialogs
 - Filters
 - Selected Items
-

28.11 Icon Reference

The portal SHALL use a consistent icon library.

Examples:

Dashboard → Layout Dashboard

Reports → File Text

Workers → Users

Departments → Building

Analytics → Bar Chart

Notifications → Bell

Settings → Settings

Profile → User

Platform → Shield

Icons SHALL originate from the approved icon library.

28.12 Design Token Reference

The Design System SHALL define reusable design tokens.

Examples include:

Typography

- Font Families
- Font Sizes
- Font Weights

Spacing

- Padding
- Margin
- Gap

Border Radius

Elevation

Color Palette

Animation Duration

Breakpoints

Tokens SHALL be centralized rather than duplicated.

28.13 Naming Reference

Recommended naming conventions:

Components

WorkerTable

ReportCard

DashboardLayout

Hooks

useReports

useDashboard

Services

workerService

reportService

Types

Worker

Report

Department

Naming SHALL remain descriptive and consistent.

28.14 Error Code Reference

The frontend SHALL recognize backend error categories such as:

- Authentication Error
- Authorization Error
- Validation Error
- Not Found
- Conflict
- Server Error

The frontend SHALL display user-friendly messages while preserving backend semantics.

28.15 Glossary

Term	Meaning
------	---------

CSCRS	Crowdsourced Civic Issue Reporting & Resolution System
-------	--

RBAC	Role-Based Access Control
------	---------------------------

KPI	Key Performance Indicator
-----	---------------------------

API	Application Programming Interface
-----	-----------------------------------

JWT	JSON Web Token
-----	----------------

AI	Artificial Intelligence
----	-------------------------

GIS	Geographic Information System
-----	-------------------------------

UAT	User Acceptance Testing
-----	-------------------------

SLA	Service Level Agreement
-----	-------------------------

PWA	Progressive Web Application
-----	-----------------------------

Additional glossary entries MAY be added as the platform evolves.

28.16 Acronyms

Acronym	Expansion
---------	-----------

UI	User Interface
----	----------------

UX	User Experience
----	-----------------

SPA	Single Page Application
-----	-------------------------

Acronym Expansion

CRUD	Create, Read, Update, Delete
REST	Representational State Transfer
CSP	Content Security Policy
WCAG	Web Content Accessibility Guidelines
OWASP	Open Worldwide Application Security Project

28.17 Cross-Reference Index

This document SHALL maintain cross-references between related chapters.

Examples:

- Dashboard → Chapter 10
- Reports → Chapter 11
- Workers → Chapter 12
- Security → Chapter 23
- Testing → Chapter 24
- AI Development → Chapter 27

This index SHALL simplify navigation within the SRS.

28.18 Future Reference Material

Future versions MAY extend this catalog with:

- UI Wireframes
- Component Screenshots
- API Sequence Diagrams
- Database Relationship Overview
- Design Tokens Specification
- Accessibility Checklist
- Localization Reference

These additions SHALL complement, rather than replace, existing chapters.

28.19 Chapter Summary

The Master Reference Catalog provides a centralized reference for the structure and organization of the CSCRS Administrative Web Portal.

It consolidates:

- Module inventory.
- Screen inventory.
- Navigation hierarchy.
- Route registry.
- Component catalog.
- RBAC matrix.
- API mappings.
- Naming conventions.
- Design references.
- Glossary and acronyms.

This chapter SHALL function as the primary quick-reference guide for developers, QA engineers, designers, technical writers, and AI-assisted development tools.

Chapter 29 — Project Governance, Version History & Product Vision

29.1 Overview

This chapter establishes the governance framework, long-term product vision, document management process, and future evolution strategy for the CSCRS Administrative Web Portal.

The purpose of this chapter is to ensure that future development remains consistent with the architectural principles and business objectives established throughout this Software Requirements Specification.

This chapter SHALL guide future developers, architects, project managers, and AI-assisted development tools.

29.2 Governance Objectives

Project governance SHALL ensure:

- Architectural consistency.
- Long-term maintainability.
- Controlled feature growth.
- Stable development practices.
- Proper documentation.

- Predictable release management.
 - Sustainable project evolution.
-

29.3 Product Vision

The CSCRS platform aims to become a comprehensive digital governance platform for municipal civic issue management.

The long-term vision includes:

- AI-assisted civic issue verification.
- Intelligent report management.
- Multi-city deployment.
- District-level administration.
- State-level administration.
- National-scale deployment.
- Smart City integration.
- Data-driven governance.

The frontend architecture SHALL support this vision without requiring fundamental redesign.

29.4 Guiding Principles

Future development SHALL prioritize:

- Simplicity over complexity.
 - Reuse over duplication.
 - Configuration over hardcoding.
 - Modularity over monolithic implementation.
 - Accessibility by default.
 - Security by design.
 - Performance as a feature.
 - Documentation alongside implementation.
-

29.5 Architecture Governance

Architectural changes SHALL follow formal review.

Major architectural decisions SHALL include:

- Business justification.
- Technical impact.
- Migration strategy.
- Compatibility analysis.
- Documentation updates.

No architectural redesign SHALL occur without documented approval.

29.6 Change Management

Changes SHALL be categorized.

Change Type Examples

Patch	Bug Fixes
Minor	New Features
Major	Architectural Changes

Each change SHALL include:

- Description
 - Reason
 - Impact Analysis
 - Testing Summary
 - Documentation Update
-

29.7 Version History

The Software Requirements Specification SHALL maintain revision history.

Illustrative format:

Version	Date	Summary
1.0	Initial Frontend SRS	First Complete Release
1.1	Future	Minor Improvements
2.0	Future	Major Functional Expansion

Document revisions SHALL remain traceable.

29.8 Roadmap

The product roadmap SHALL evolve in phases.

Phase 1

- Landing Portal
 - Authentication
 - Dashboard
 - Reports
 - Workers
 - Departments
 - Analytics
-

Phase 2

- Notification Improvements
 - AI Insights
 - Better Reporting
 - Improved Analytics
-

Phase 3

- GIS Integration
 - Interactive Maps
 - Heat Maps
 - Ward Analytics
-

Phase 4

- Mobile Administrative Portal
 - AI Assistant
 - Smart Recommendations
 - Offline Support
-

Phase 5

- Multi-City Deployment
- District Administration

- State Administration
- National Deployment

Future phases SHALL remain compatible with earlier architectural decisions.

29.9 Future Platform Expansion

The architecture SHALL support future modules including:

- GIS Dashboard
- Citizen Analytics
- Worker Attendance
- Vehicle Tracking
- IoT Integration
- Smart Sensors
- Predictive Maintenance
- Resource Planning
- Budget Analytics
- Public Transparency Dashboard
- Disaster Management Support

These modules SHALL integrate without redesigning existing functionality.

29.10 AI Roadmap

Future AI capabilities MAY include:

- AI Copilot for Administrators
- Predictive Issue Detection
- Automated Report Prioritization
- AI Resolution Suggestions
- AI Risk Forecasting
- AI Dashboard Summaries
- AI Chat Interface
- Natural Language Search

The current frontend architecture SHALL remain compatible with future AI integration.

29.11 Mobile Ecosystem

The CSCRS ecosystem SHALL eventually consist of:

Citizen Mobile App

↓

Backend APIs

↓

Administrative Web Portal

↓

Worker Mobile App

↓

Future Public Dashboard

Each client SHALL communicate through the centralized backend architecture.

29.12 Smart City Vision

Future Smart City integration MAY include:

- CCTV Analytics
- IoT Sensors
- Flood Monitoring
- Air Quality Monitoring
- Smart Street Lights
- Intelligent Waste Monitoring
- Traffic Monitoring
- Emergency Response Systems

The frontend SHALL remain extensible for these integrations.

29.13 Assumptions

This specification assumes:

- Backend APIs remain the authoritative business layer.
- PostgreSQL remains the production database.
- REST APIs remain the primary communication mechanism.
- JWT-based authentication remains supported.
- React + TypeScript remain the approved frontend stack.

Changes to these assumptions SHALL require architectural review.

29.14 Constraints

Current project constraints include:

- Browser-based administrative interface.
- Existing backend API contracts.
- Approved frontend technology stack.
- Existing RBAC model.
- Government-oriented usability requirements.

Future enhancements SHALL respect these constraints unless formally revised.

29.15 Risks

Potential project risks include:

- Rapid feature expansion.
- Scope creep.
- Third-party dependency changes.
- Backend API evolution.
- Browser compatibility changes.
- AI-generated code inconsistencies.

These risks SHALL be managed through documented governance processes.

29.16 Roles & Responsibilities

Role	Responsibilities
Product Owner	Business direction and priorities
Solution Architect	Overall architecture decisions
Frontend Developers	Portal implementation
Backend Developers	API and business logic
QA Engineers	Verification and validation
UI/UX Designers	User experience and design consistency
DevOps Engineers	Deployment and infrastructure
AI Assistants	Code generation under human supervision

All contributors SHALL follow the standards defined in this specification.

29.17 Documentation Governance

This SRS SHALL remain a living document.

Whenever major functionality changes:

- Relevant chapters SHALL be updated.
- Version history SHALL be revised.
- Cross-references SHALL be verified.
- Deprecated information SHALL be removed or clearly marked.

Documentation SHALL evolve alongside the product.

29.18 References

This frontend SRS SHALL be read together with the following project documentation:

- Backend API Documentation
- Backend Architecture Documentation
- Database Documentation
- Deployment Documentation
- Security Documentation
- Verification Engine Documentation

Together, these documents SHALL define the complete CSCRS platform.

29.19 Success Criteria

The frontend SHALL be considered successful when it:

- Implements all approved functional requirements.
 - Integrates correctly with backend APIs.
 - Enforces RBAC through the user interface.
 - Meets performance targets.
 - Satisfies accessibility requirements.
 - Passes production acceptance testing.
 - Remains maintainable and extensible.
-

29.20 Long-Term Vision Statement

The CSCRS Administrative Web Portal is envisioned as more than a municipal complaint management interface.

It is intended to evolve into a scalable digital governance platform capable of supporting intelligent civic administration across multiple organizational levels while maintaining security, accessibility, operational transparency, and data-driven decision making.

Every future enhancement SHOULD preserve the architectural integrity established by this specification.

29.21 Final Conclusion

This Software Requirements Specification establishes the functional, technical, architectural, security, quality, and governance requirements for the CSCRS Administrative Web Portal.

It provides a unified reference for:

- Business stakeholders
- Frontend developers
- Backend developers
- UI/UX designers
- QA engineers
- DevOps engineers
- Technical writers
- AI-assisted development tools

Compliance with this specification SHALL ensure that the portal remains:

- Consistent

- Secure
- Scalable
- Maintainable
- Accessible
- Production-ready

while supporting the long-term evolution of the CSCRS platform.

29.22 Document Approval (Template)

Role	Name	Signature	Date
Project Owner	_____	_____	_____
Solution Architect	_____	_____	_____
Frontend Lead	_____	_____	_____
Backend Lead	_____	_____	_____
QA Lead	_____	_____	_____

This section SHALL be completed prior to declaring the specification as the approved baseline for implementation.